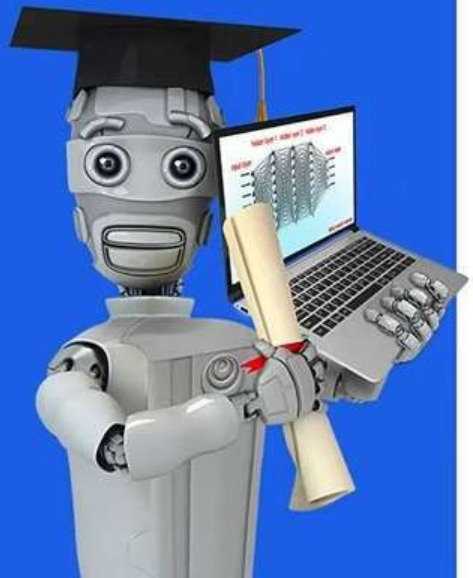
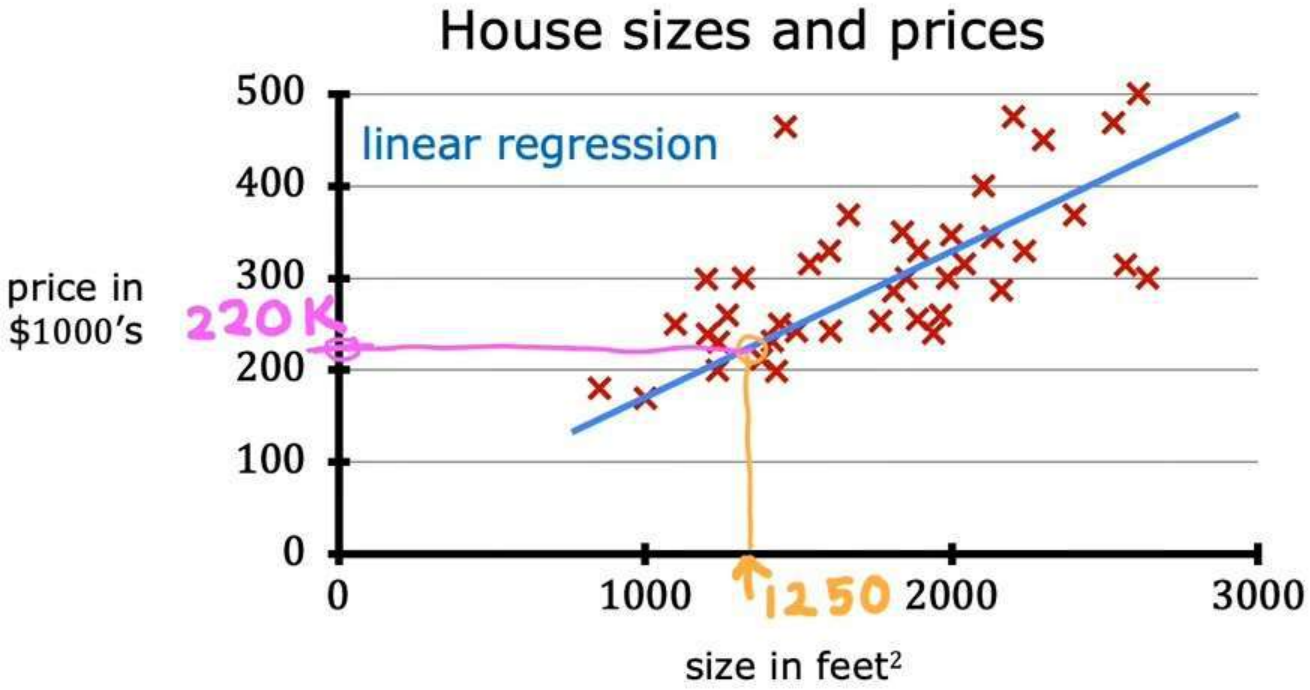
Stanford
ONLINE

DeepLearning.AI



Linear Regression with One Variable

Linear Regression Model Part 1

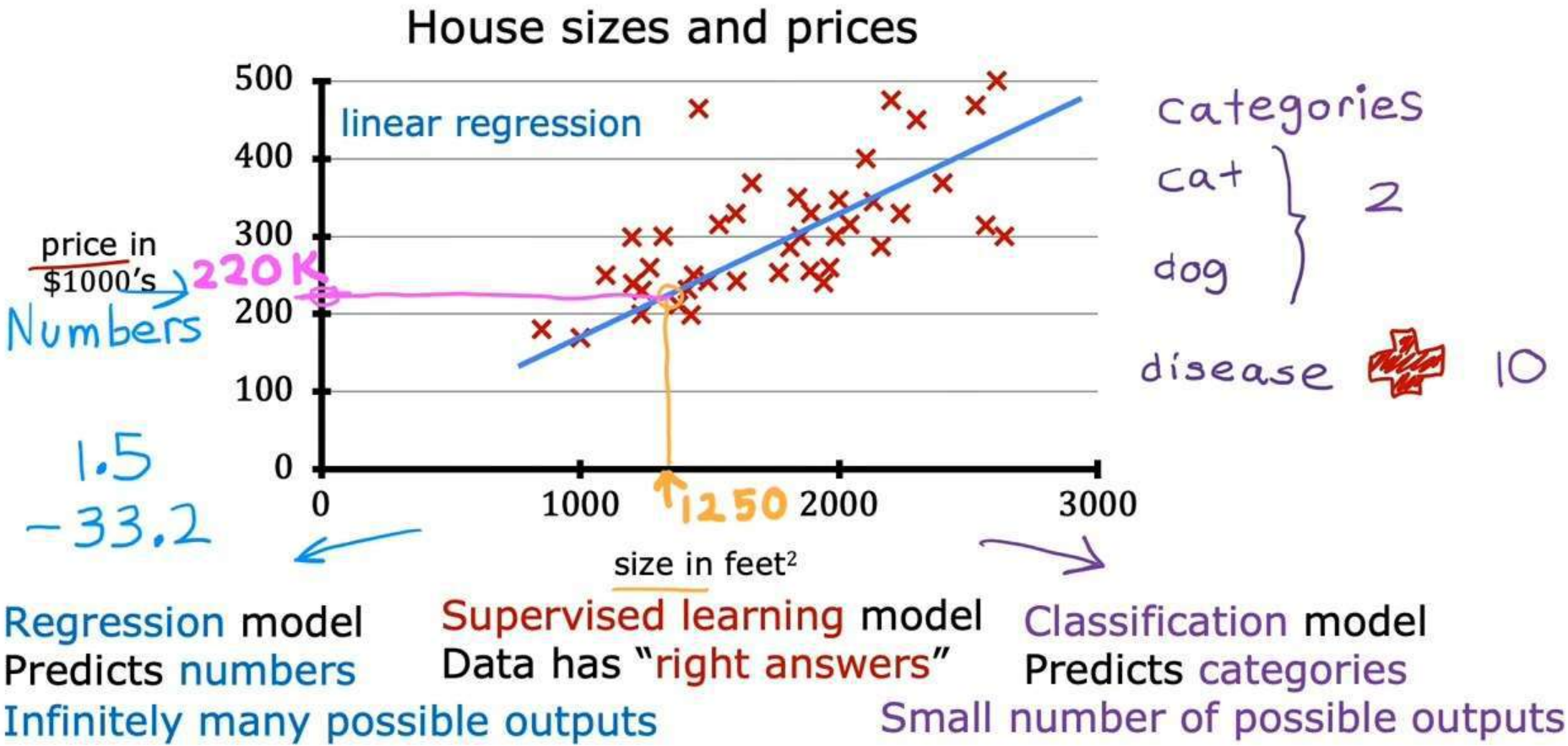

House sizes and prices
linear regression
price in $1000's
500
400
300
220K
200
100
0
0
1000
1250
2000
3000
size in feet²

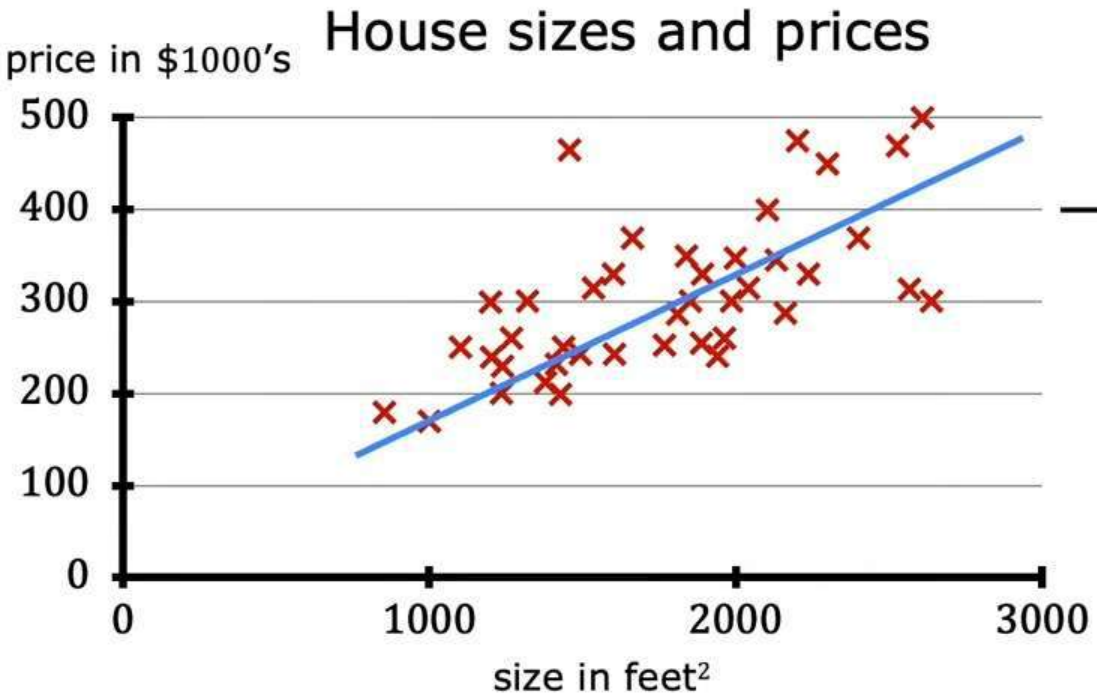


Regression model
Predicts numbers

Supervised learning model
Data has "right answers"

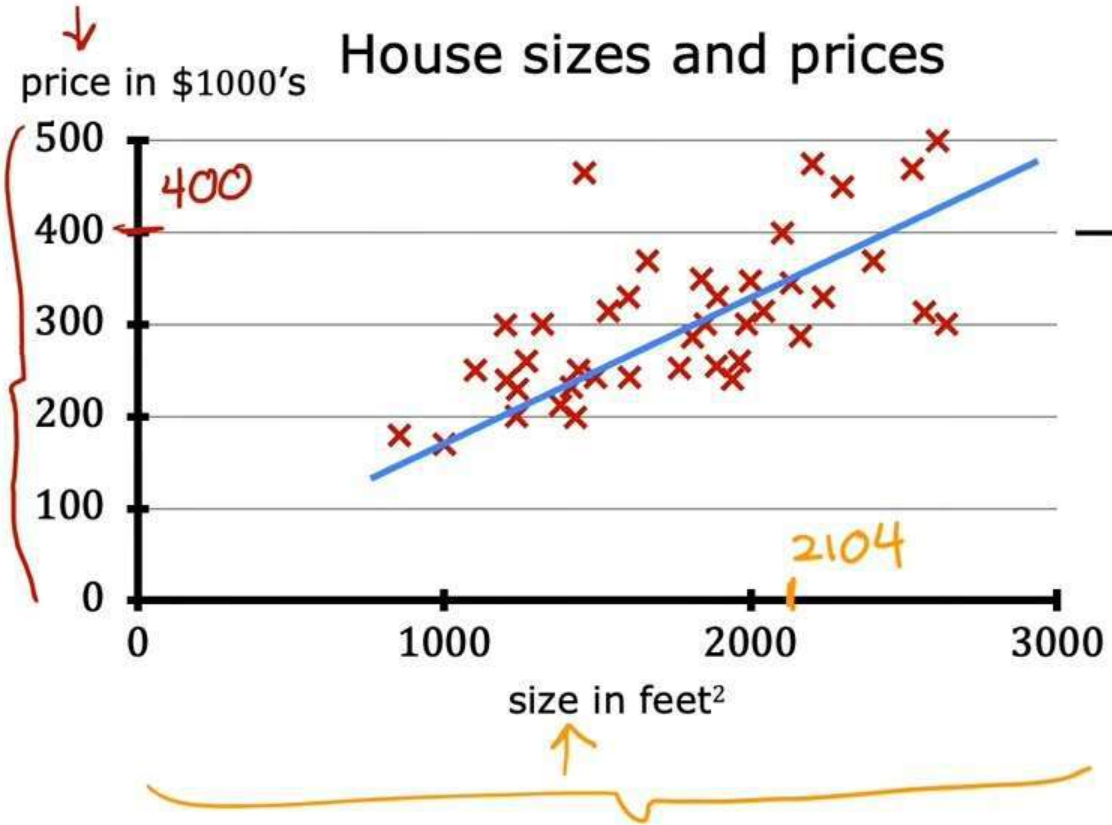
Classification model
Predicts categories


House sizes and prices
linear regression
price in $1000's
Numbers
220K
500
400
300
200
100
0
0
1000
1250
2000
3000
size in feet²
1.5
-33.2
categories
cat
dog
2
disease
10
Regression model
Predicts numbers
Infinitely many possible outputs
Supervised learning model
Data has "right answers"
Classification model
Predicts categories
Small number of possible outputs



Data table

size in feet²	price in $1000's
2104	400
1416	232
1534	315
852	178
...	...
3210	870



Data table

size in feet²	price in $1000's
2104	400
1416	232
1534	315
852	178
...	...
3210	870



Data table

size in feet²	price in $1000's
2104	400
1416	232
1534	315
852	178
...	...
3210	870

Terminology

Training set: Data used to train the model

	x size in feet^2	y price in \$1000's
(1)	2104	400
(2)	1416	232
(3)	1534	315
(4)	852	178
…	…	…
(47)	3210	870

$m = 47$

$x^{(1)} = 2104$ $y^{(1)} = 400$

$(x^{(1)}, y^{(1)}) = (2104, 400)$

Notation:

x = "input" variable
feature

y = "output" variable
"target" variable

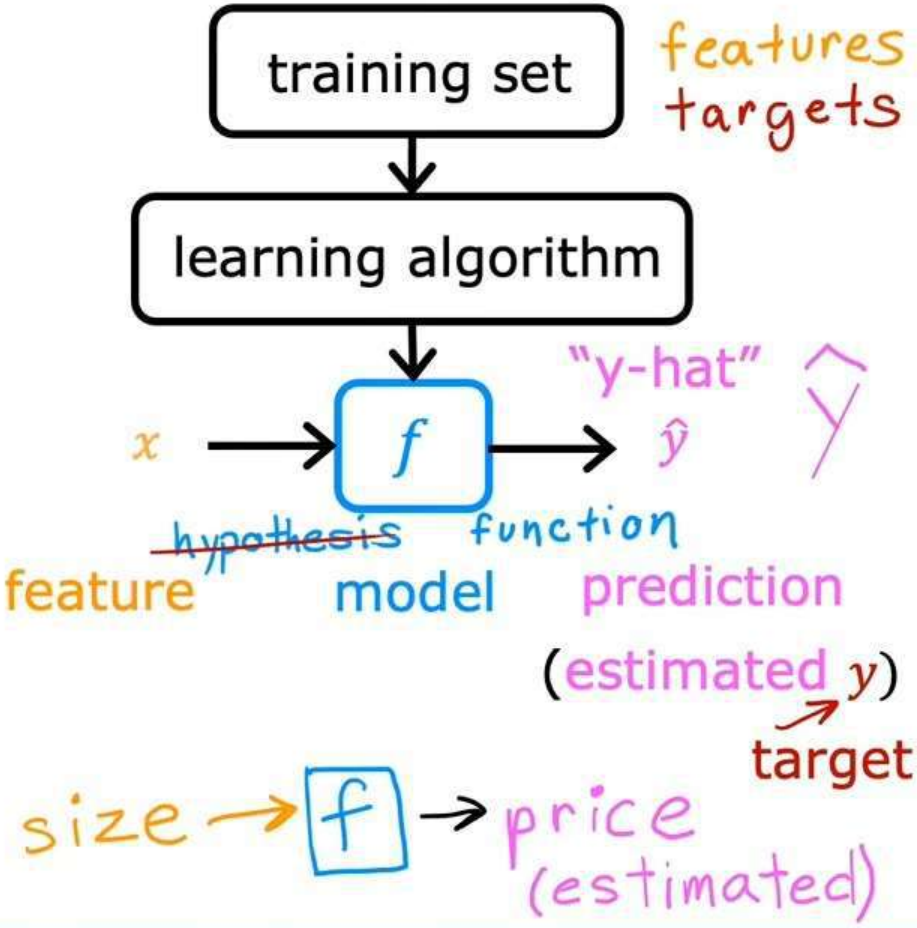
m = number of training examples

(x, y) = single training example

$(x^{(i)}, y^{(i)})$

$(x^{(i)}, y^{(i)})$ = i^{th} training example

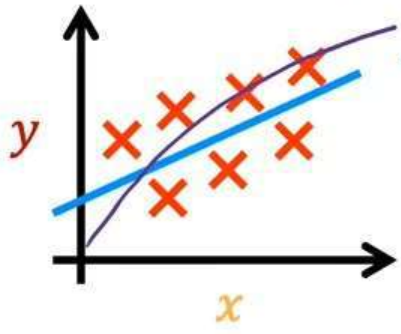
(1st, 2nd, 3rd …)



How to represent f?

$$f_{w,b}(x) = wx + b$$

$$f(x)$$



$$f_{w,b}(x) = wx + b$$

$$f(x) = wx + b$$

linear

single feature x

Linear regression with one variable.

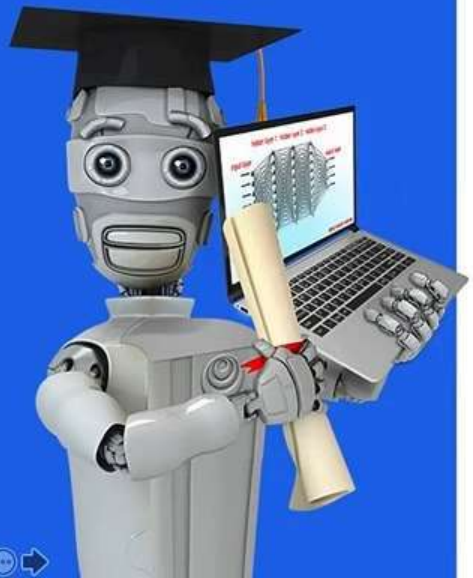
size

Univariate linear regression.

one variable

Stanford
ONLINE

DeepLearning.AI



Linear Regression with One Variable

Cost Function

Training set

features | targets

size in feet² (x)	price \$1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Model: $f_{w,b}(x) = wx + b$

w, b: parameters

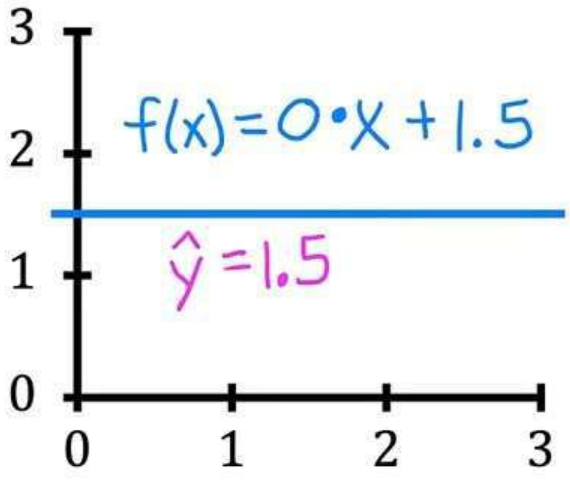
coefficients

weights

What do w, b do?

$$f_{w,b}(x) = wx + b$$

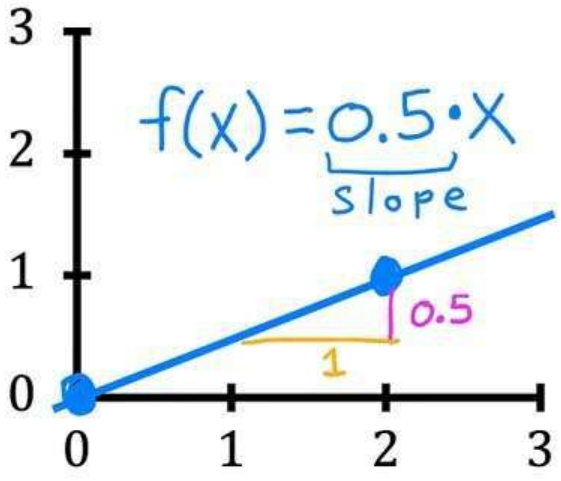
$f(x)$



→ $w = 0$

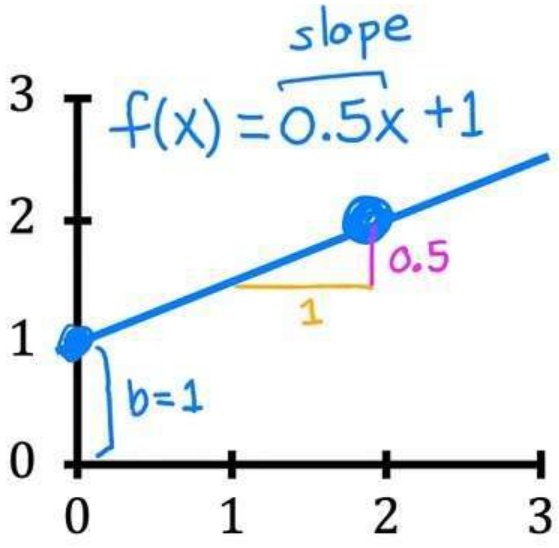
→ $b = 1.5$

(y-intercept)



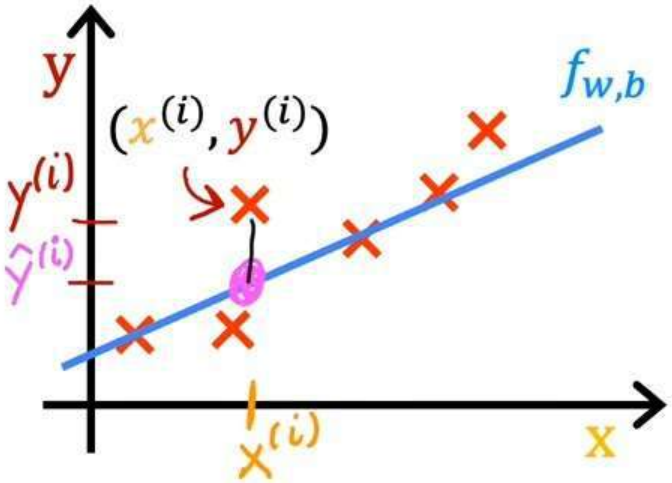
→ $w = 0.5$

→ $b = 0$



→ $w = 0.5$

→ $b = 1$



$$\hat{y}^{(i)} = f_{w,b}\left(x^{(i)}\right)$$

$$f_{w,b}\left(x^{(i)}\right) = wx^{(i)} + b$$

Cost function: Squared error cost function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

($\hat{y}^{(i)}$: error)

m = number of training examples

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}\left(x^{(i)}\right) - y^{(i)}\right)^2$$

Find w, b:

$\hat{y}^{(i)}$ is close to $y^{(i)}$ for all $(x^{(i)}, y^{(i)})$.

Stanford
ONLINE

DeepLearning.AI



Linear Regression with One Variable

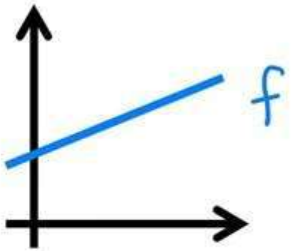
Cost Function Intuition

model:

$$f_{w,b}(x) = wx + b$$

parameters:

w, b



cost function:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

goal:

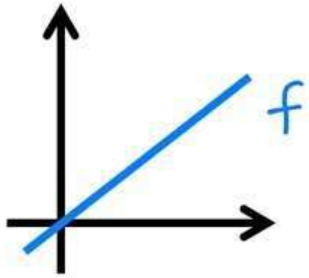
$$\underset{w,b}{\text{minimize}}\, J(w,b)$$

simplified

$$f_w(x) = wx$$

$b = \emptyset$

w



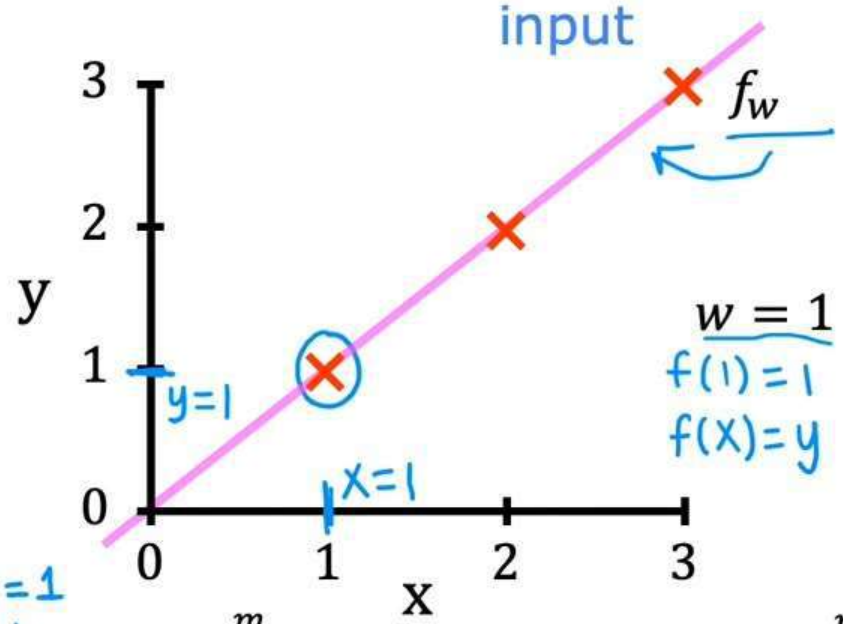
$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^{(i)}) - y^{(i)})^2$$

$wx^{(i)}$

$$\underset{w}{\text{minimize}}\, J(w)$$

$f_w(x)$

(for fixed w, function of x)



$J(w)$

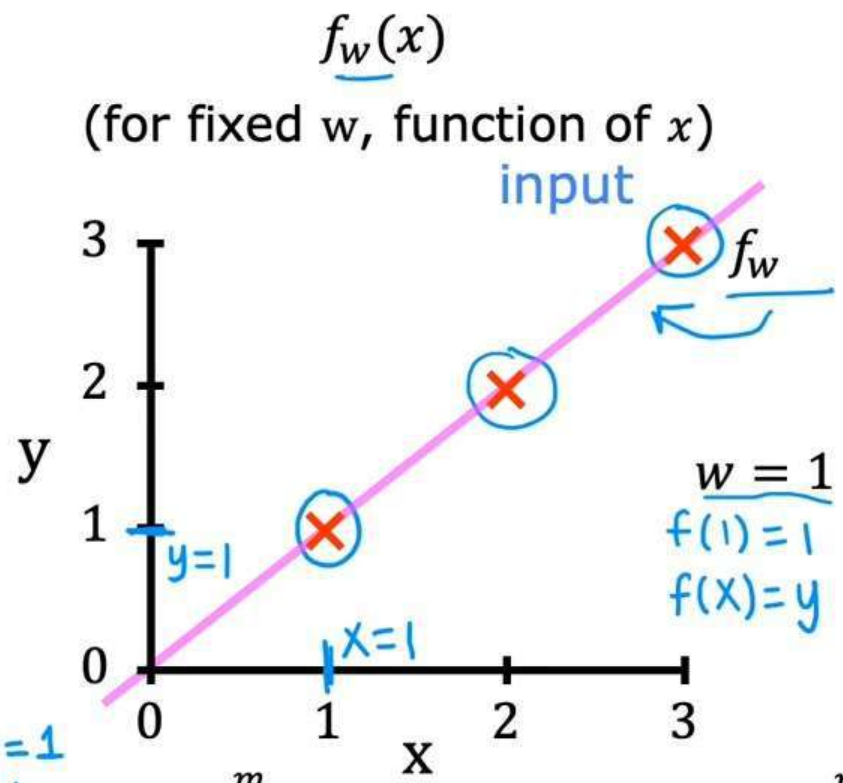
(function of w)

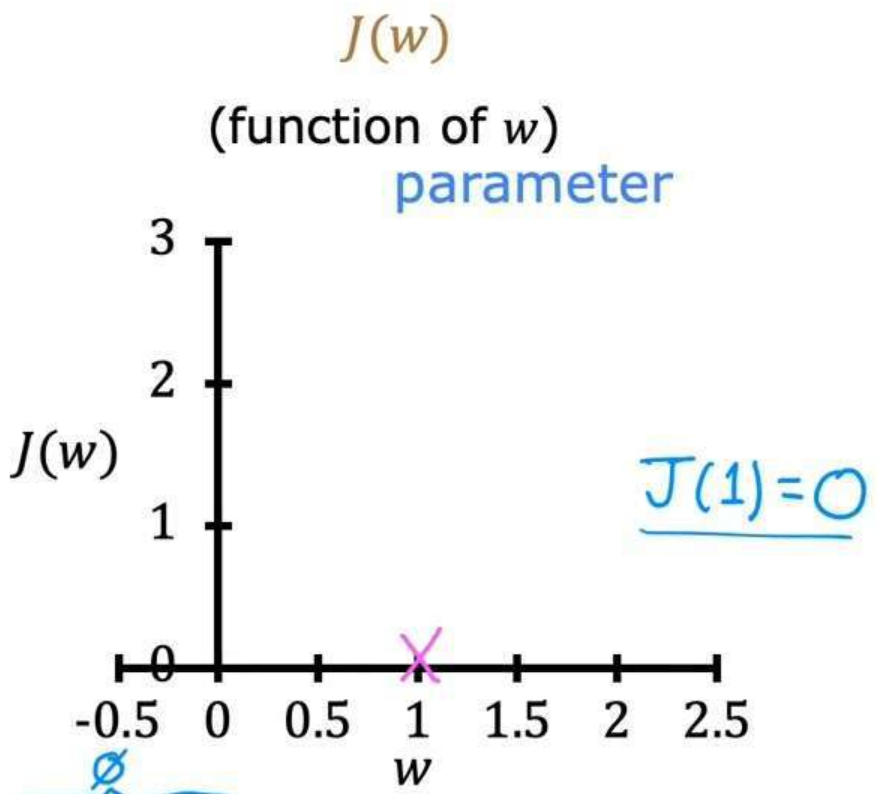
parameter

$w=1$

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m}\sum_{i=1}^{m}(wx^{(i)} - y^{(i)})^2$$

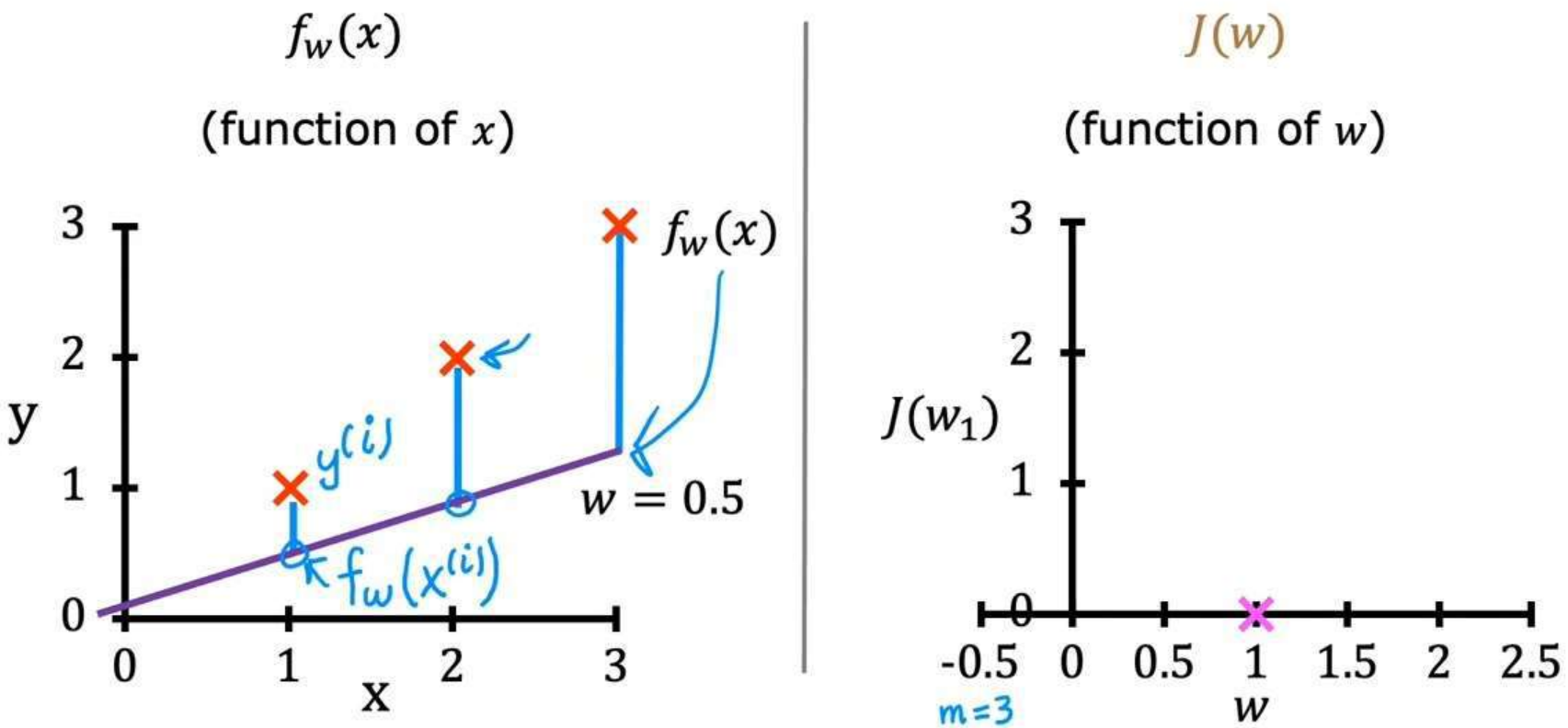
$wx^{(i)}$





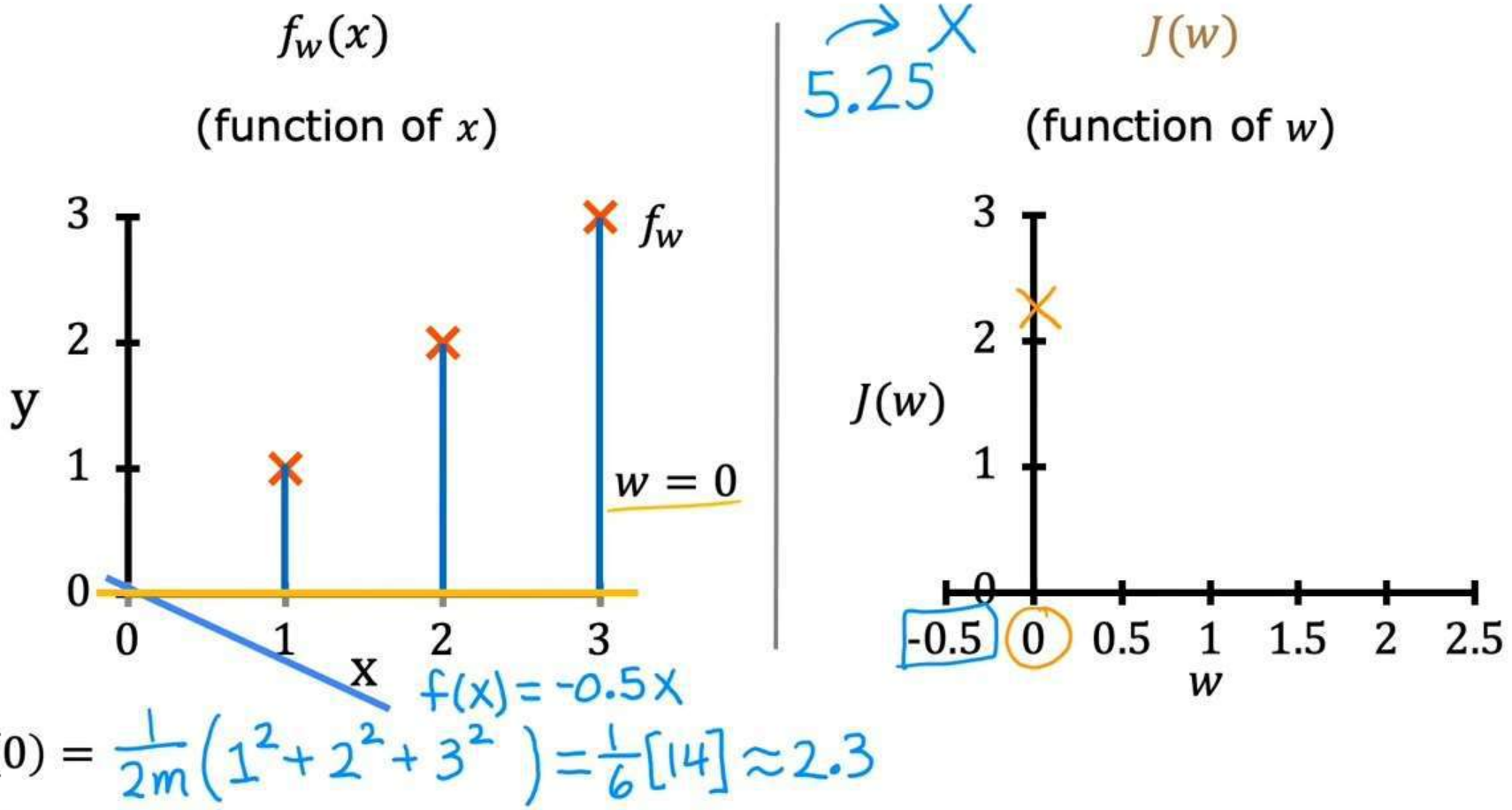
$w = 1$

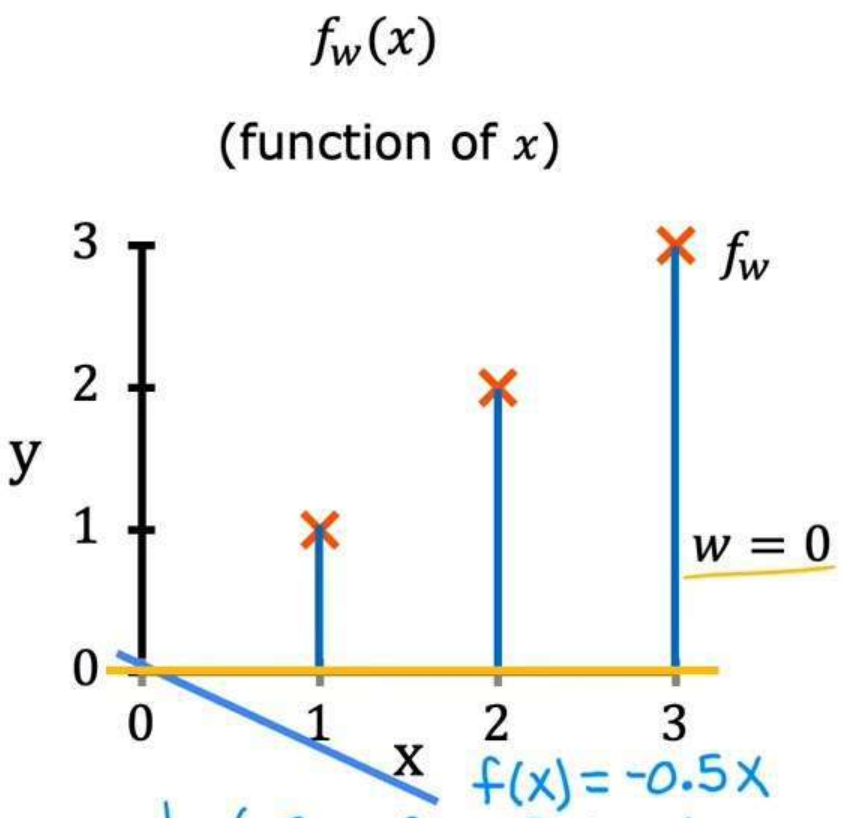
$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(f_w(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m}\sum_{i=1}^{m}(wx^{(i)} - y^{(i)})^2 = \frac{1}{2m}(0^2 + 0^2 + 0^2) = 0$$

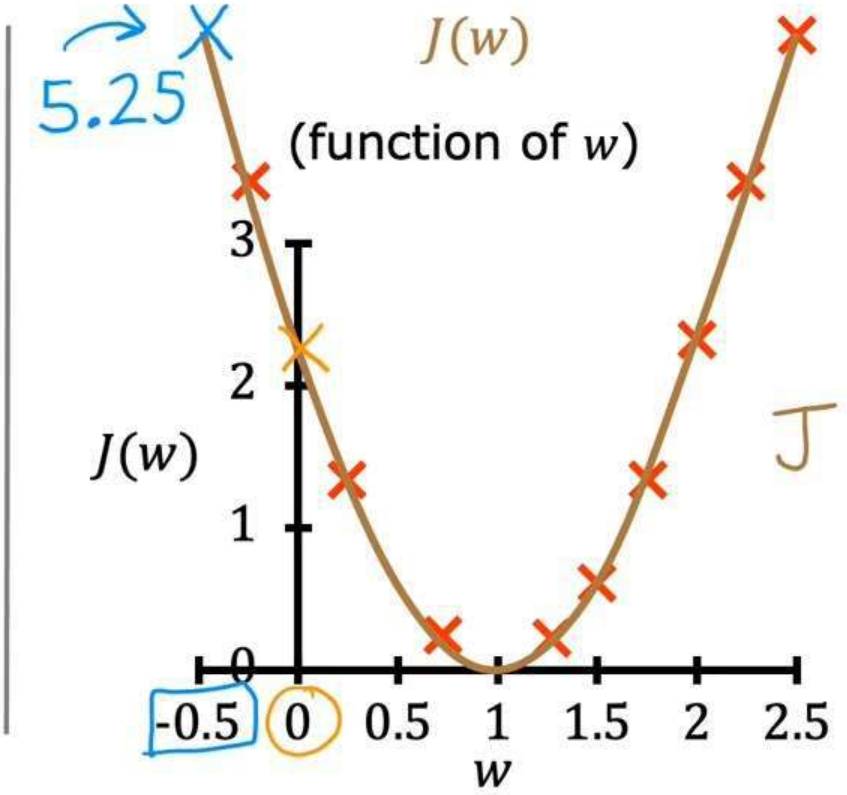


$$J(0.5) = \frac{1}{2m}\left[(0.5-1)^2 + (1-2)^2 + (1.5-3)^2\right] = \frac{1}{2\times 3}[3.5] = \frac{3.5}{6} \approx 0.58$$

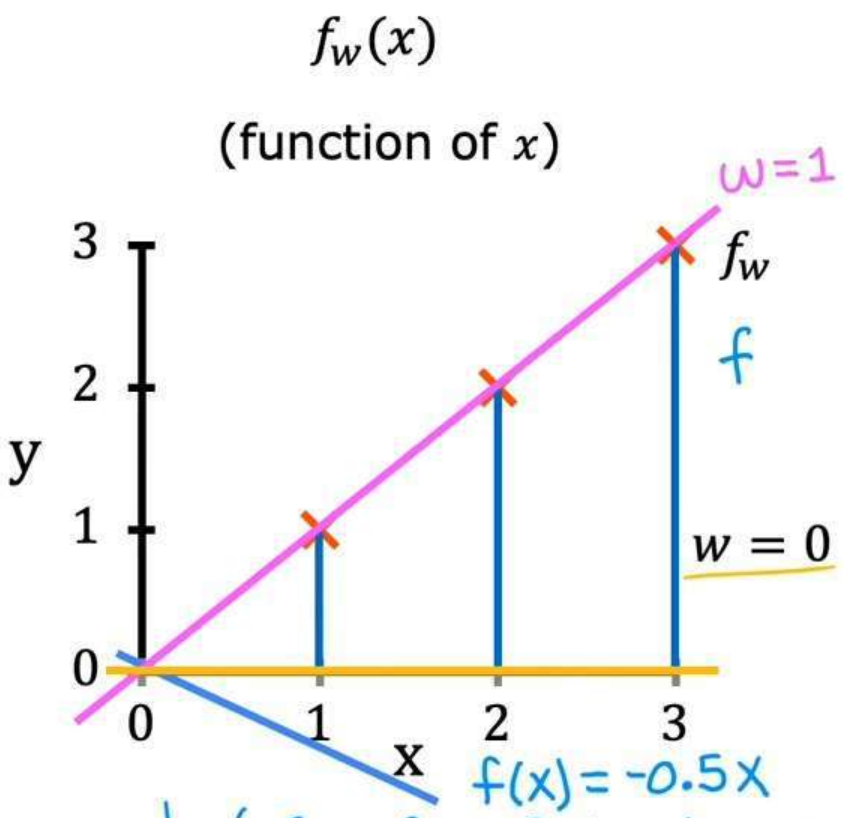
$m = 3$

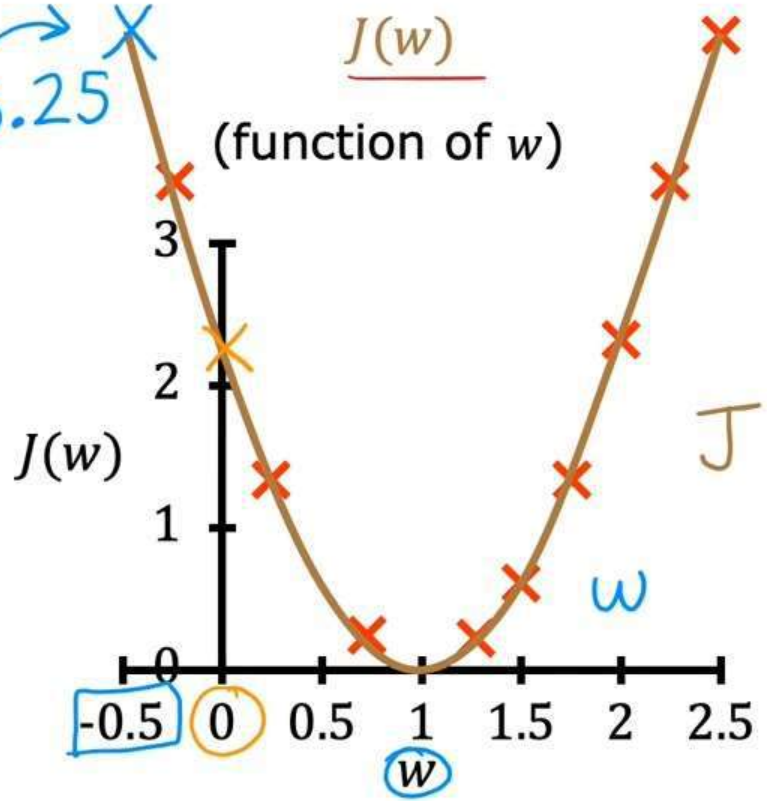

$f_w(x)$
(function of x)
3
2
1
0
y
0
1
2
3
x
f_w
$w = 0$
$f(x) = -0.5x$
$J(w) = \frac{1}{2m}(1^2 + 2^2 + 3^2) = \frac{1}{6}[14] \approx 2.3$
5.25
$J(w)$
(function of w)
3
2
1
0
$J(w)$
-0.5
0
0.5
1
1.5
2
2.5
w



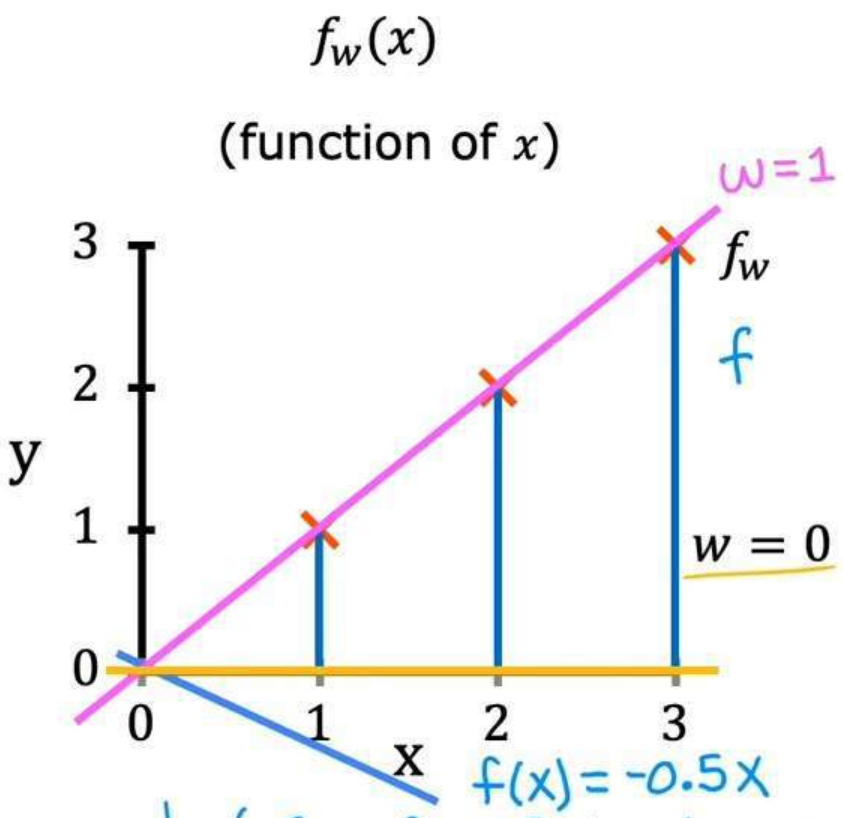


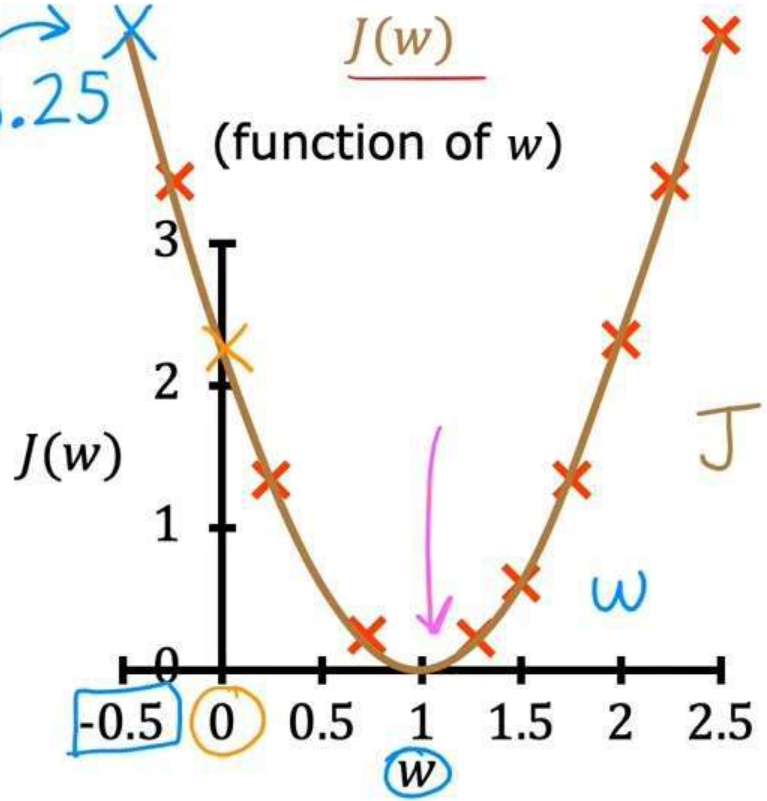
$$J(0) = \frac{1}{2m}\left(1^2 + 2^2 + 3^2\right) = \frac{1}{6}[14] \approx 2.3$$



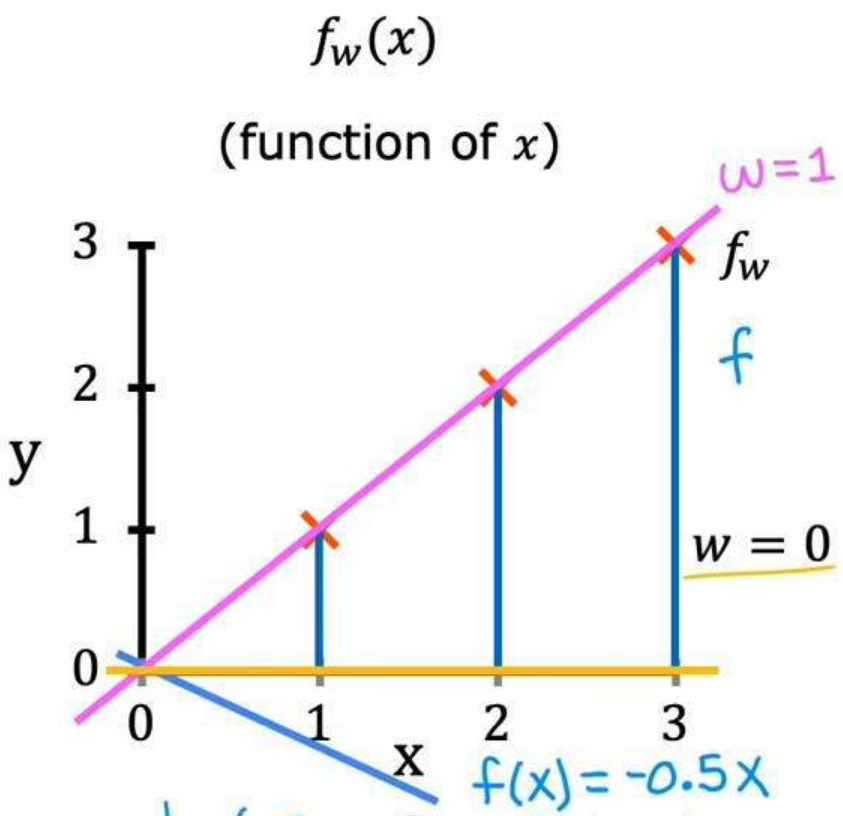


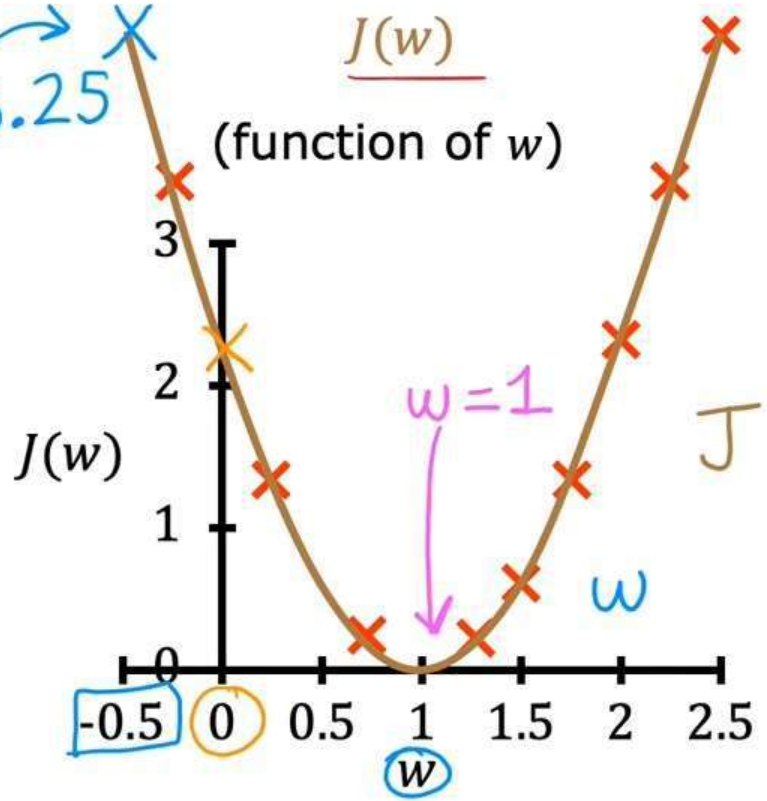
$$J(0) = \frac{1}{2m}\left(1^2 + 2^2 + 3^2\right) = \frac{1}{6}[14] \approx 2.3$$



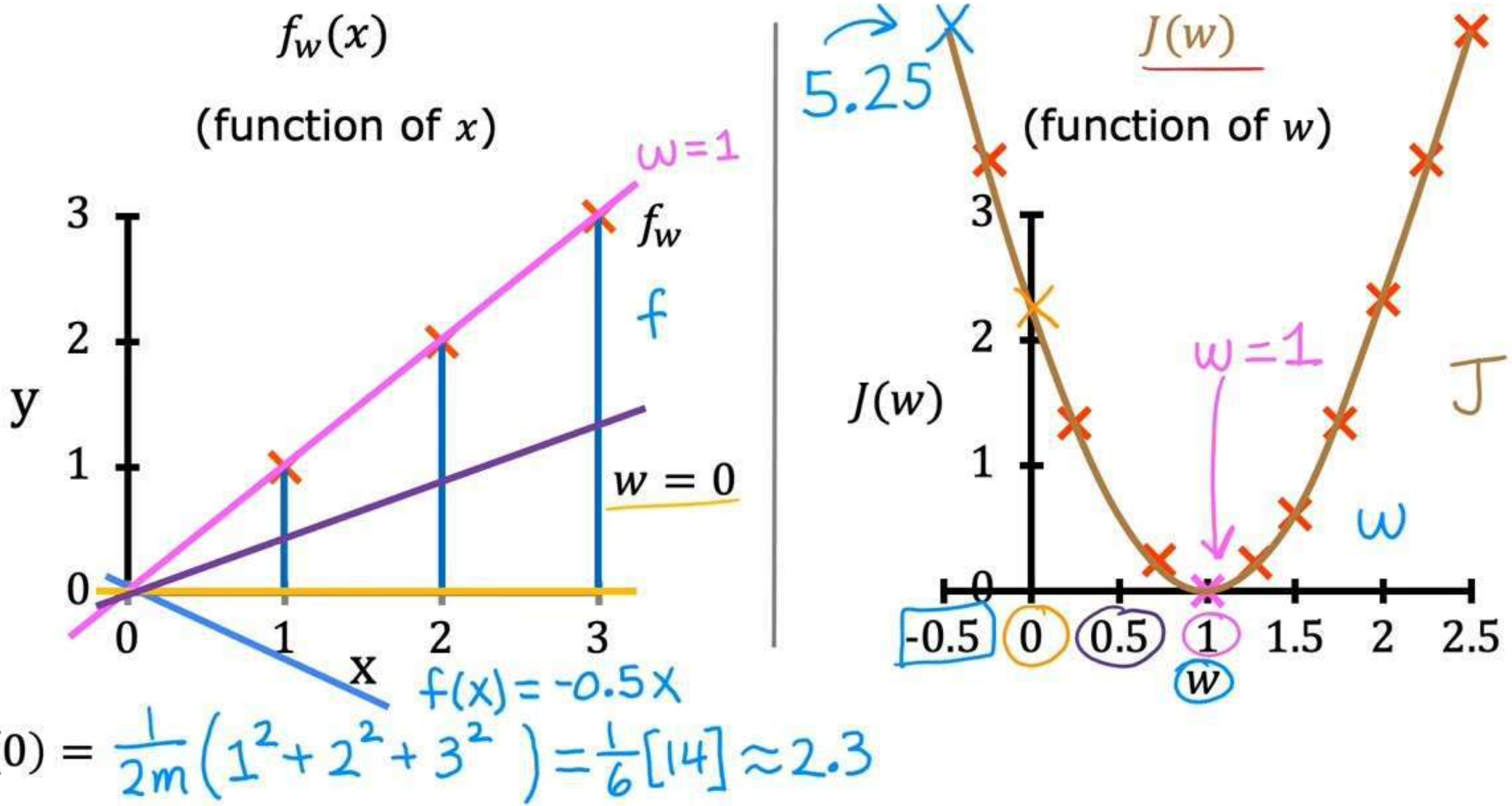


$$J(0) = \frac{1}{2m}\left(1^2 + 2^2 + 3^2\right) = \frac{1}{6}[14] \approx 2.3$$

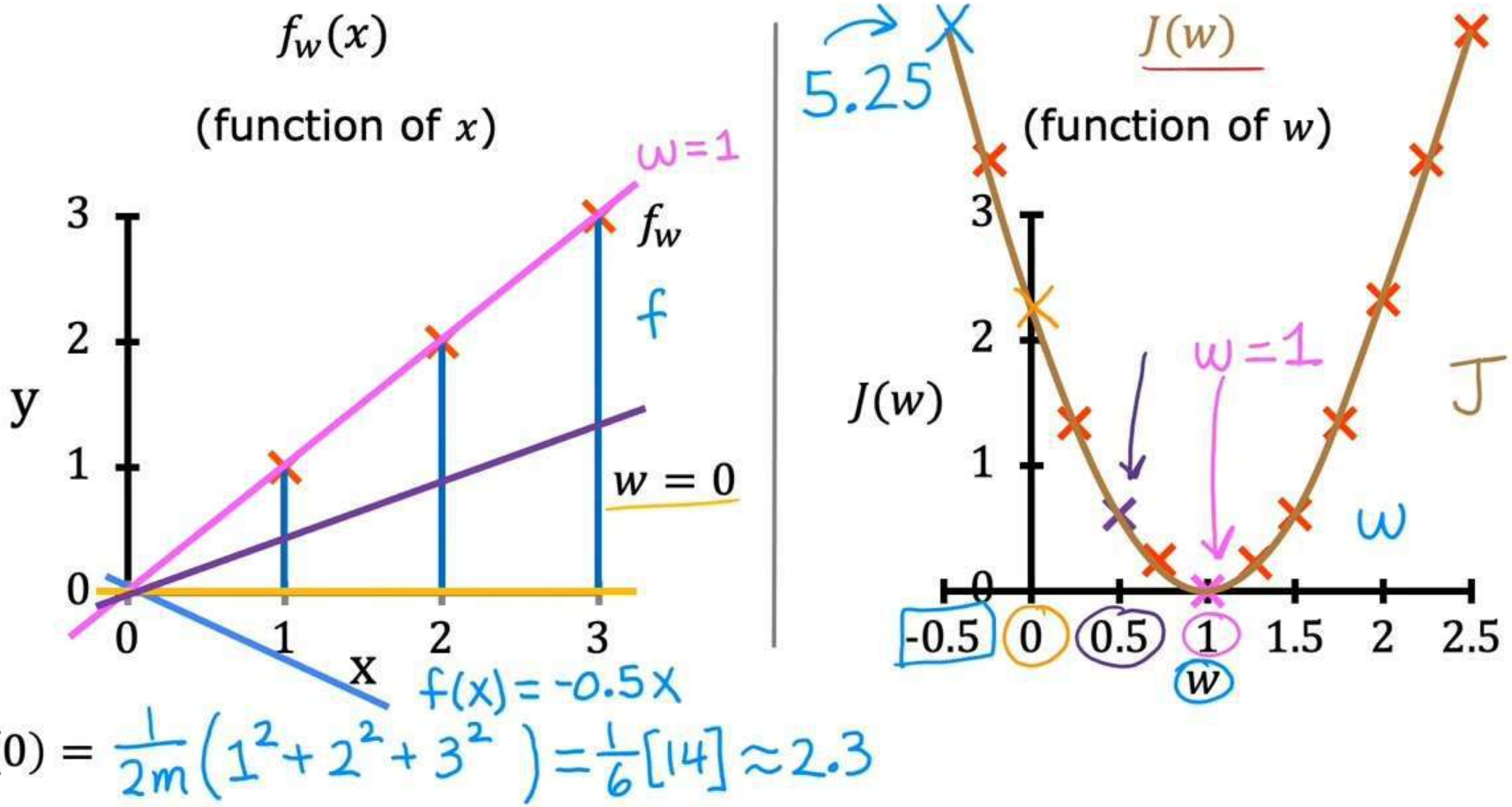




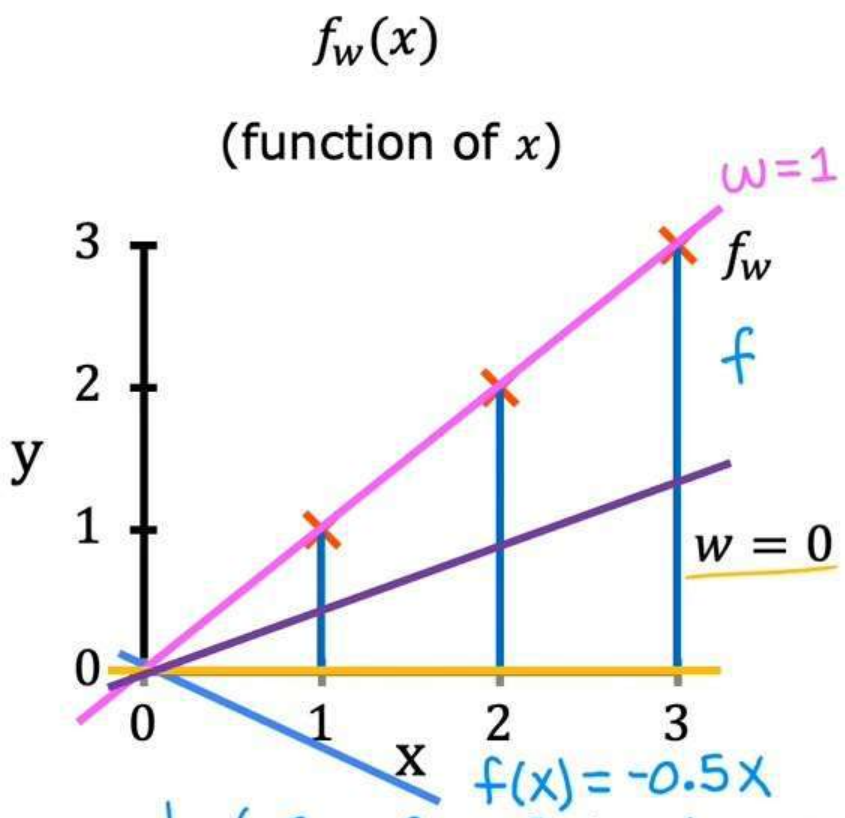
$$J(0) = \frac{1}{2m}\left(1^2+2^2+3^2\right) = \frac{1}{6}[14] \approx 2.3$$

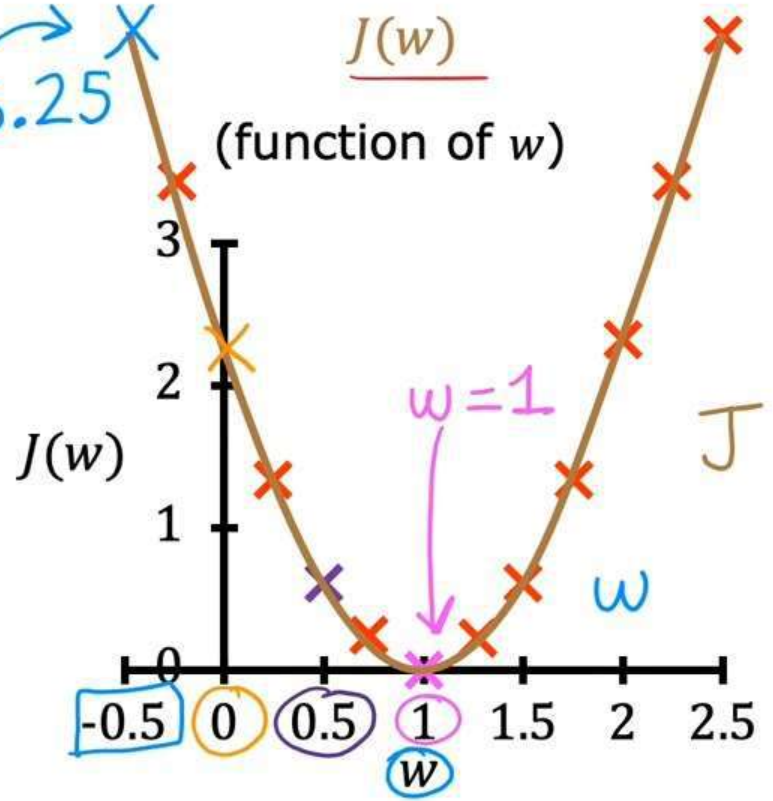


$$J(0) = \frac{1}{2m}\left(1^2+2^2+3^2\right) = \frac{1}{6}[14] \approx 2.3$$



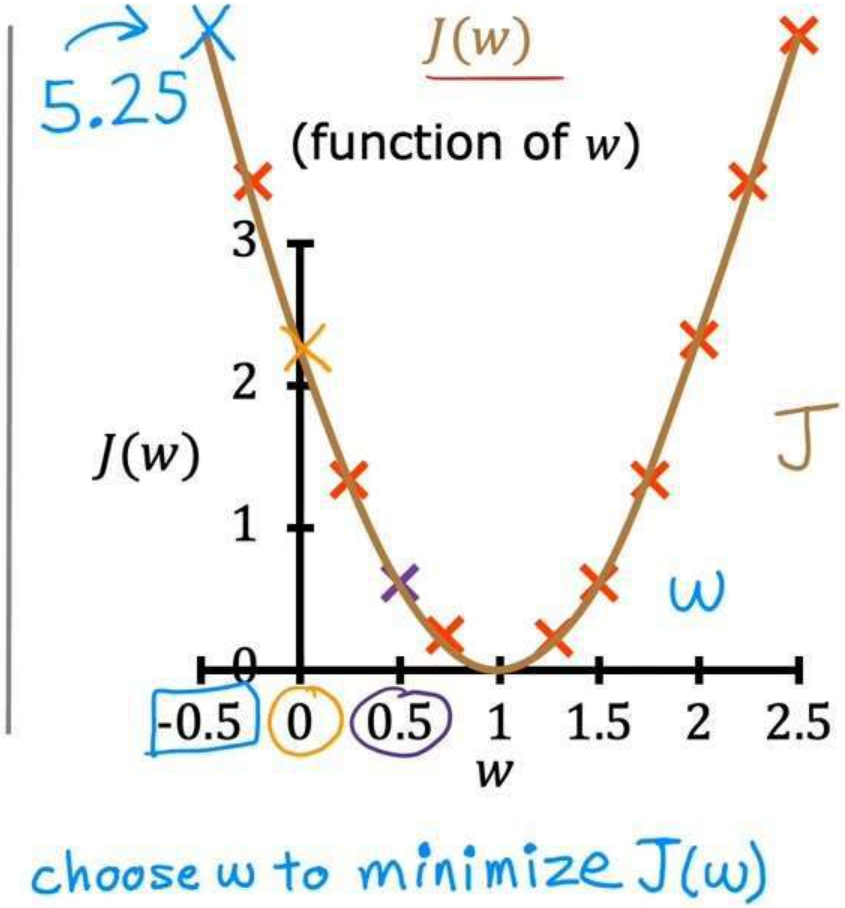
$$J(0) = \frac{1}{2m}\left(1^2+2^2+3^2\right) = \frac{1}{6}[14] \approx 2.3$$





$$J(0) = \frac{1}{2m}\left(1^2 + 2^2 + 3^2\right) = \frac{1}{6}[14] \approx 2.3$$

how to choose w?

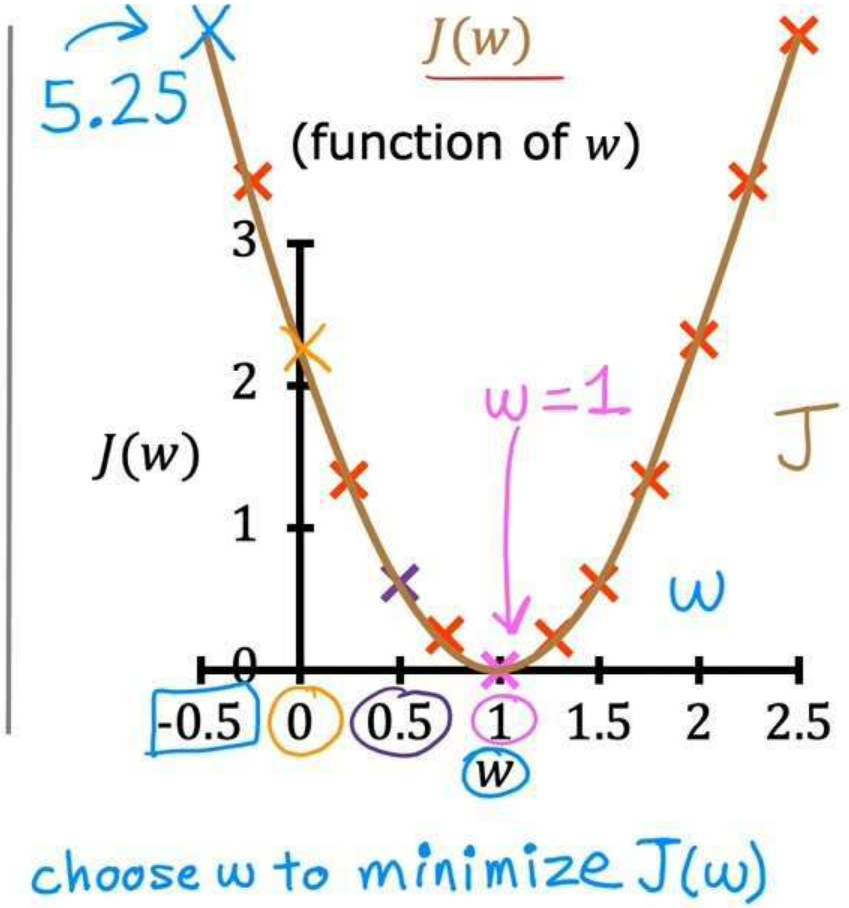

5.25
$J(w)$
(function of w)
3
2
1
0
$J(w)$
J
w
-0.5
0
0.5
1
1.5
2
2.5
w
choose w to minimize J(w)

goal of linear regression:

$$\underset{w}{\text{minimize}}\, J(w)$$

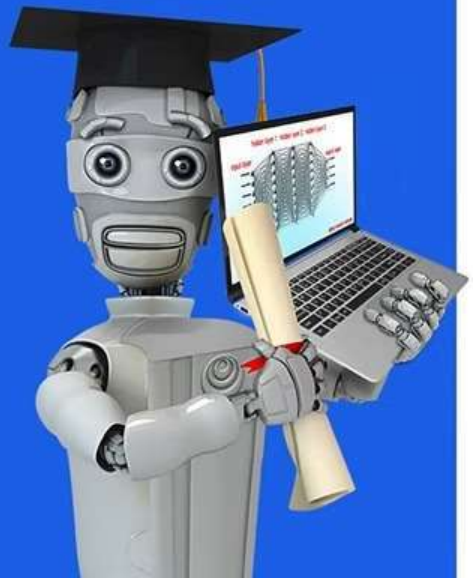
general case:

$$\underset{w,b}{\text{minimize}}\, J(w,b)$$



Stanford
ONLINE

DeepLearning.AI

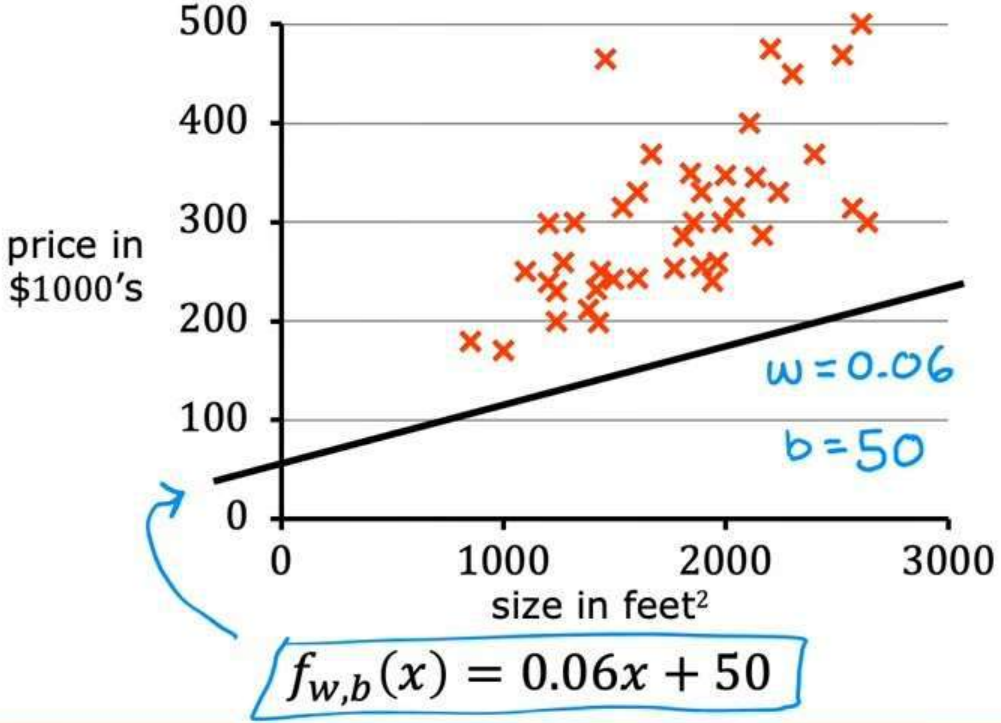


Linear Regression with One Variable

Visualizing the Cost Function

$f_{w,b}$

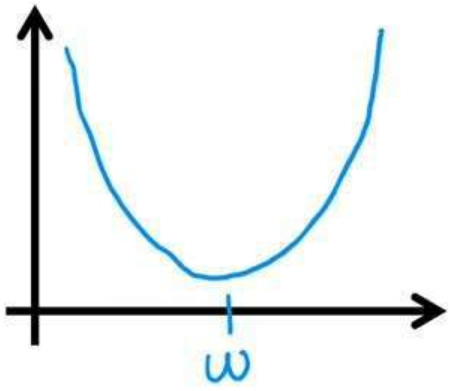
(function of x)



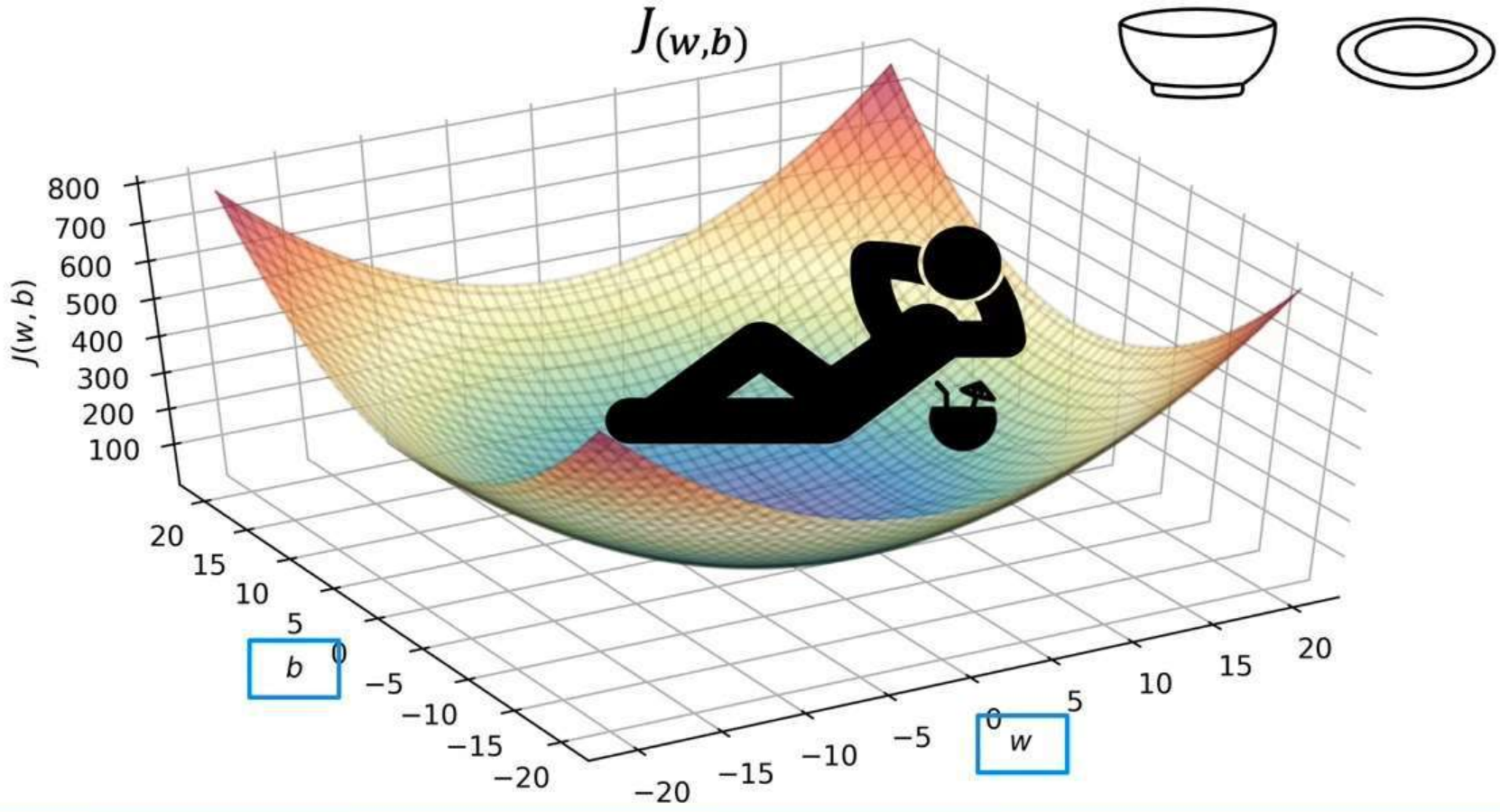
$f_{w,b}(x) = 0.06x + 50$

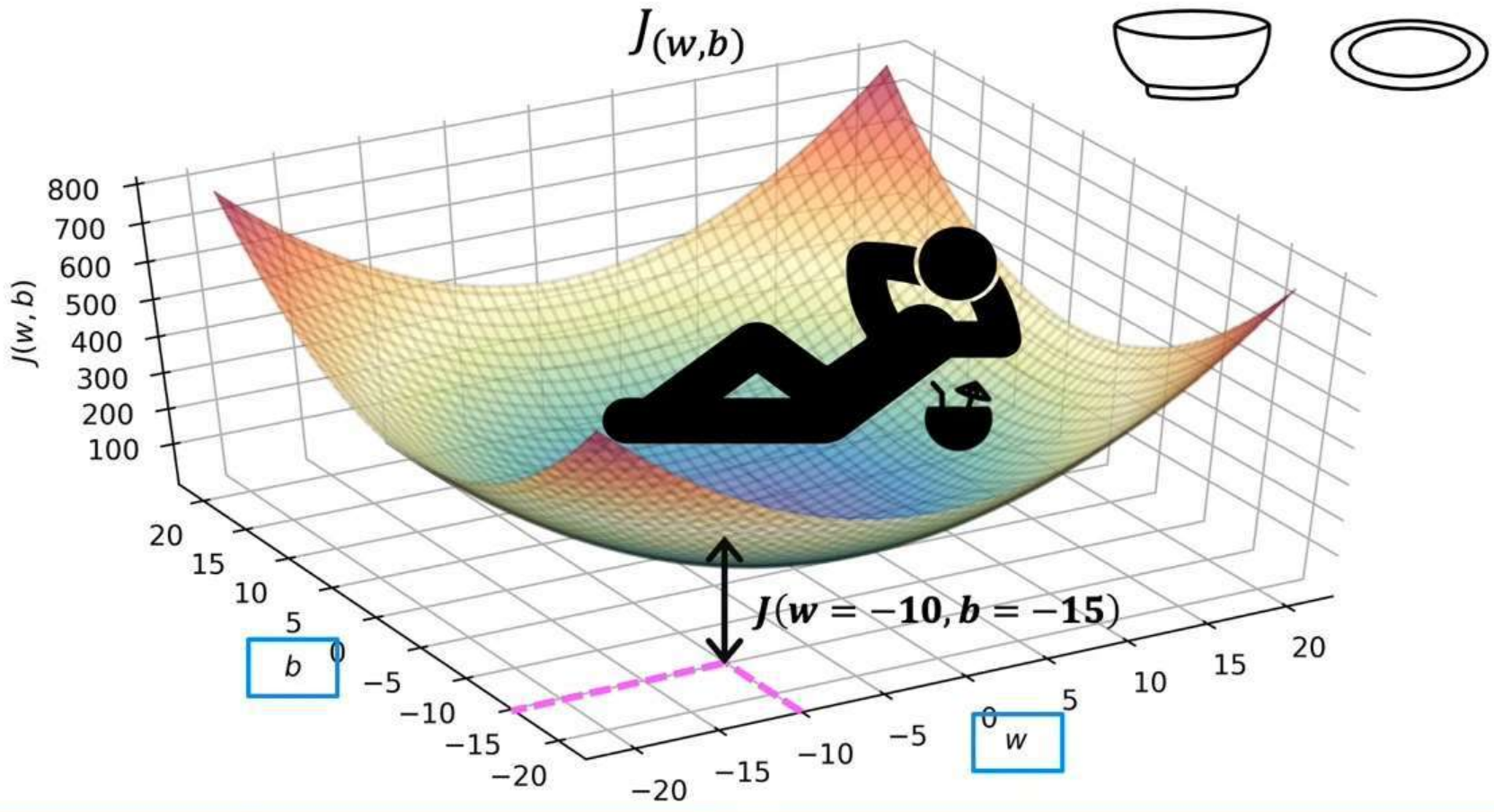
J

(function of w, b)

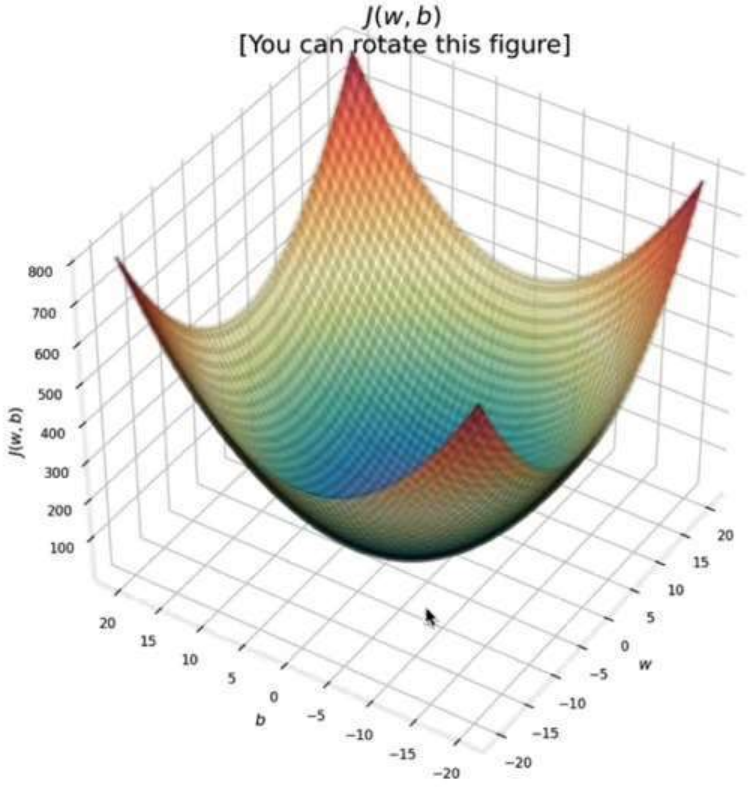


w , b

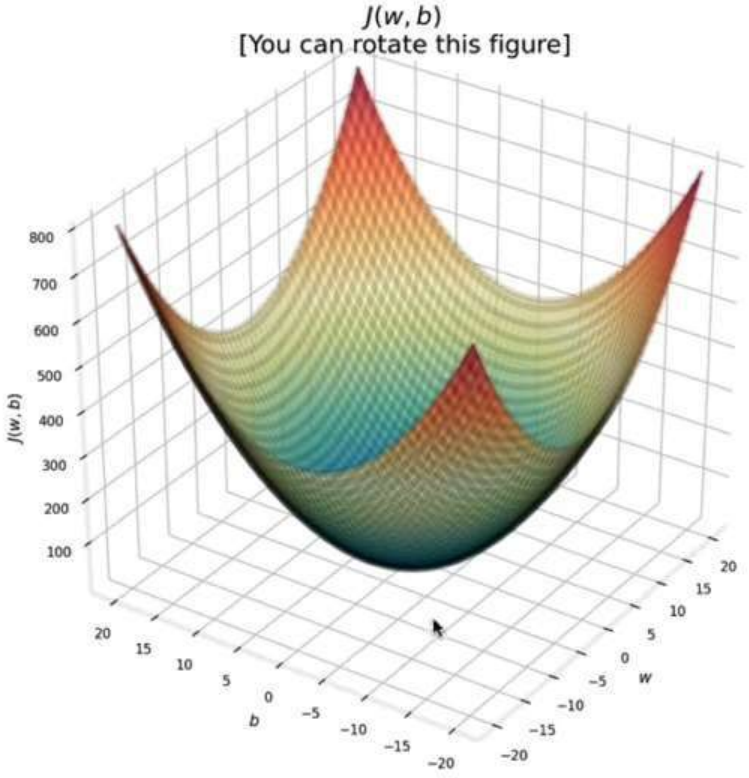

$J_{(w,b)}$
$J(w,b)$
800
700
600
500
400
300
200
100
20
15
10
5
0
−5
−10
−15
−20
b
−20
−15
−10
−5
0
5
10
15
20
w

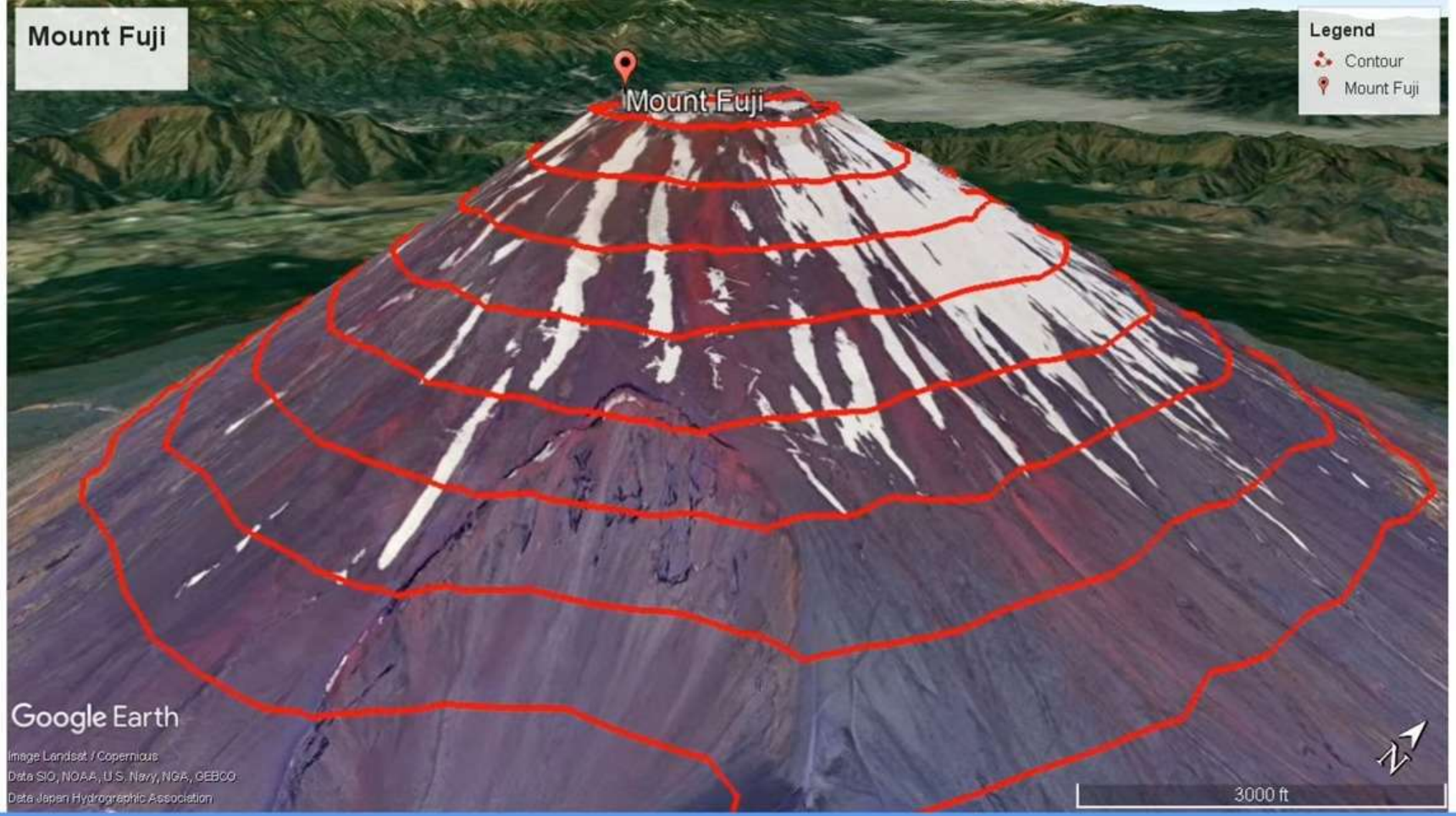

$J_{(w,b)}$
$J(w,b)$
800
700
600
500
400
300
200
100
20
15
10
5
0
−5
−10
−15
−20
b
−20
−15
−10
−5
0
5
10
15
20
w
$J(w=-10, b=-15)$

3D surface plot



3D surface plot




Mount Fuji
Mount Fuji
Legend
Contour
Mount Fuji
Google Earth
Image Landsat / Copernicus
Data SIO, NOAA, U.S. Navy, NGA, GEBCO
Data Japan Hydrographic Association
3000 ft

Stanford
ONLINE

DeepLearning.AI



Training Linear Regression

Gradient Descent

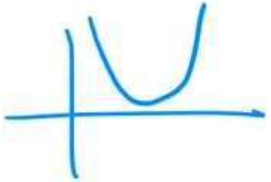
Have some function $J(w, b)$ for linear regression or any function

Want $\min_{w,b} J(w, b)$ $\min_{w_1, \ldots, w_n, b} J(w_1, w_2, \ldots, w_n, b)$

Outline:

- Start with some w, b (set $w=0$, $b=0$)
- Keep changing w, b to reduce $J(w, b)$
- Until we settle at or near a minimum

J not always



may have >1 minimum

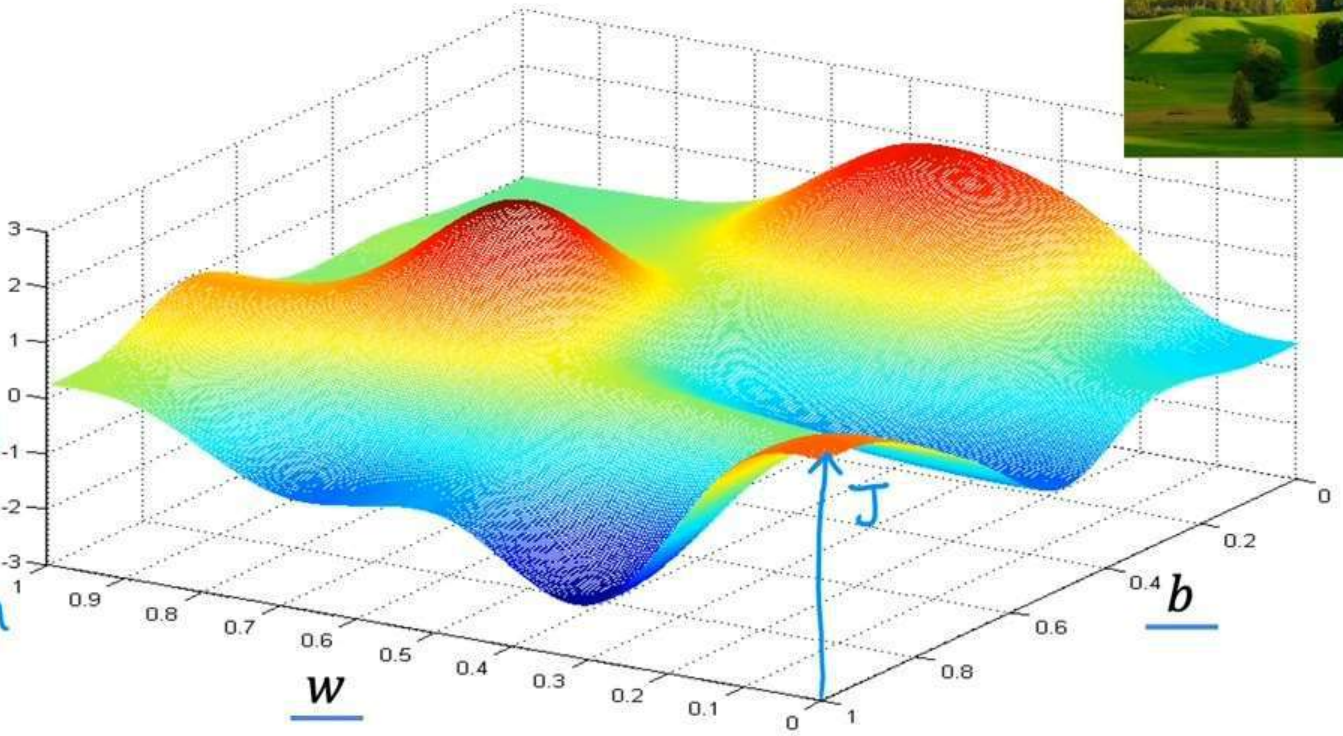
gradient descent



$J(w, b)$

not squared error cost

not linear regression



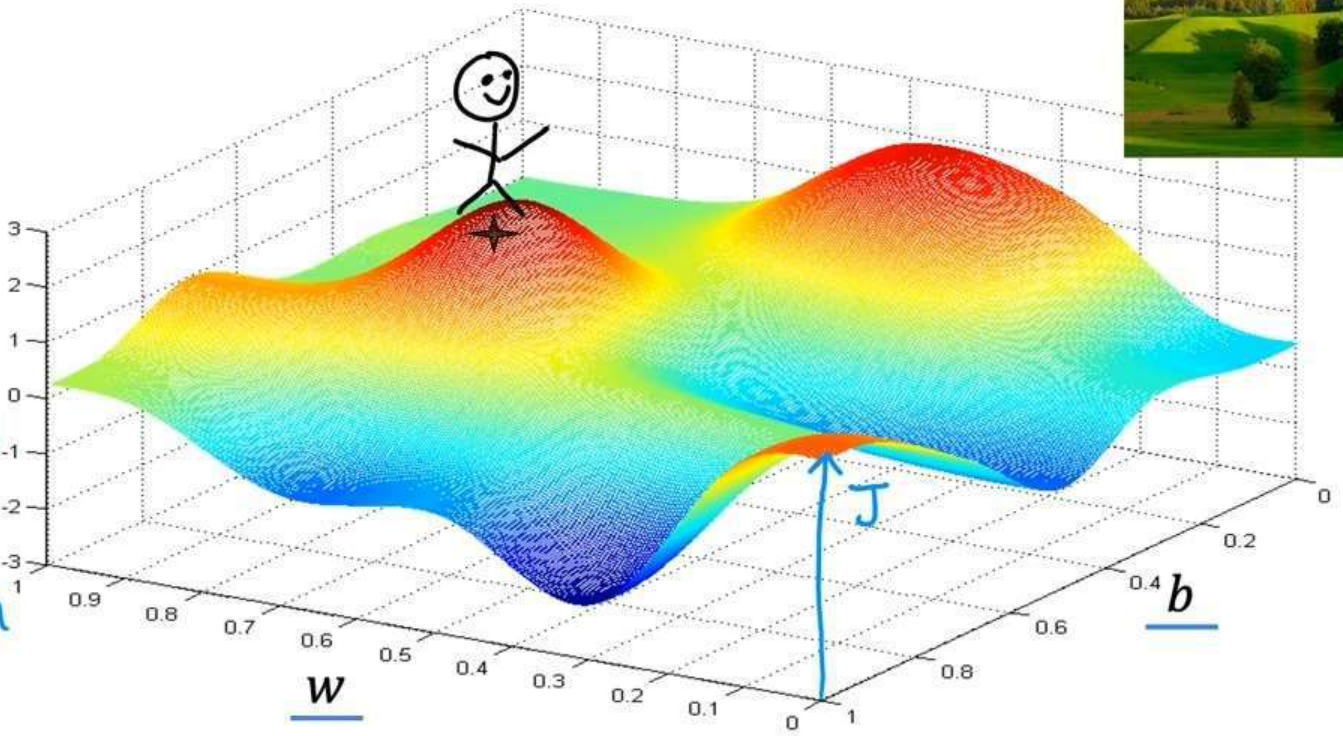
gradient descent



$J(w, b)$

not squared error cost

not linear regression



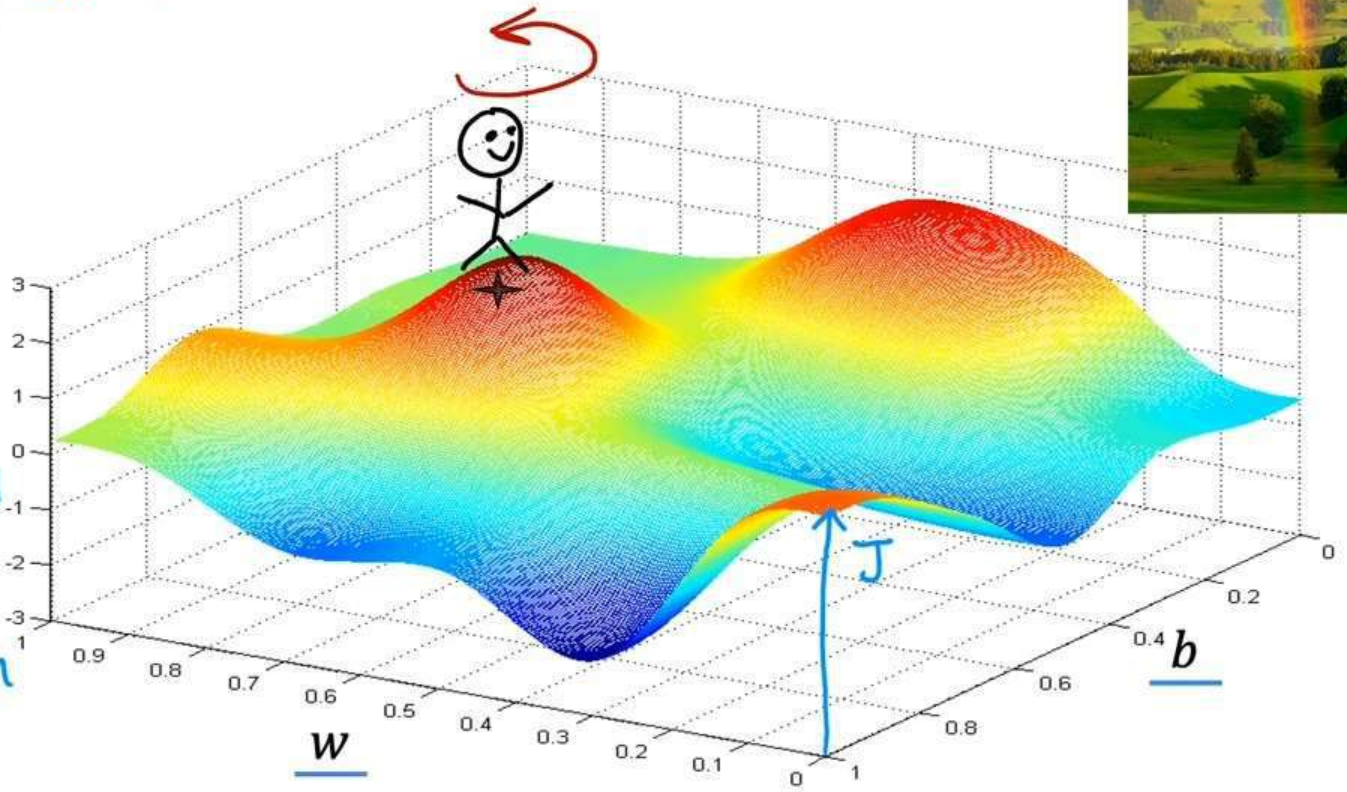
gradient descent



$J(w, b)$

not squared error cost

not linear regression



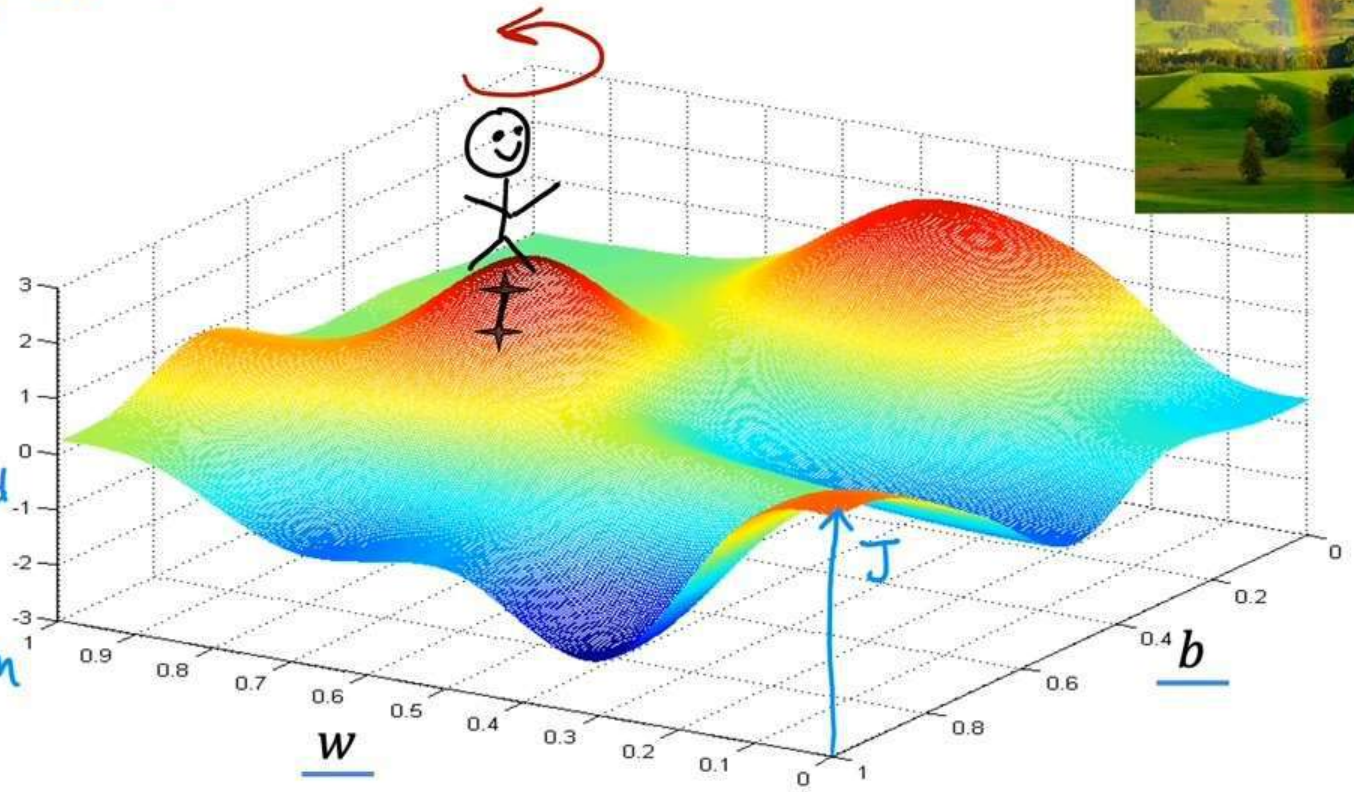
gradient descent



$J(w, b)$

not squared error cost

not linear regression



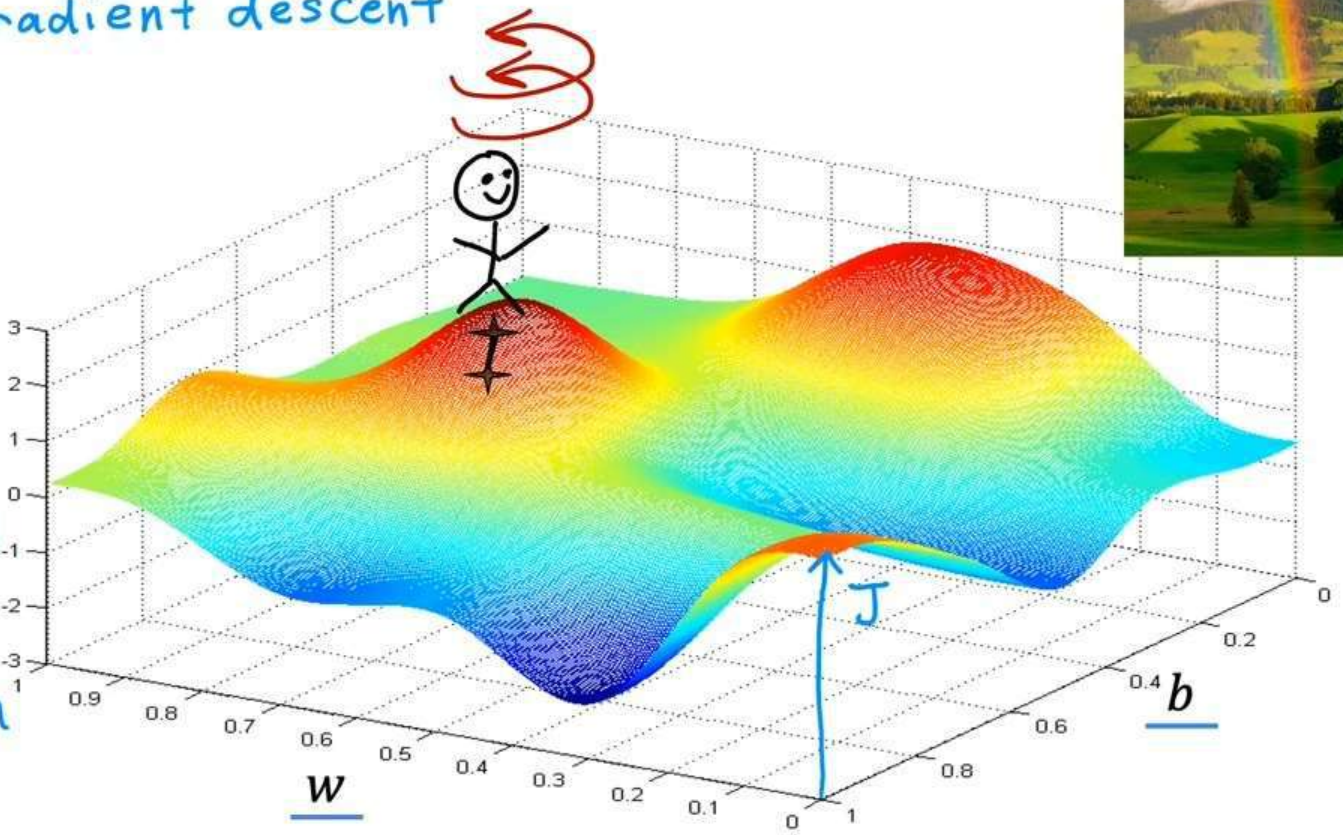
gradient descent



$J(w,b)$

not squared error cost

not linear regression



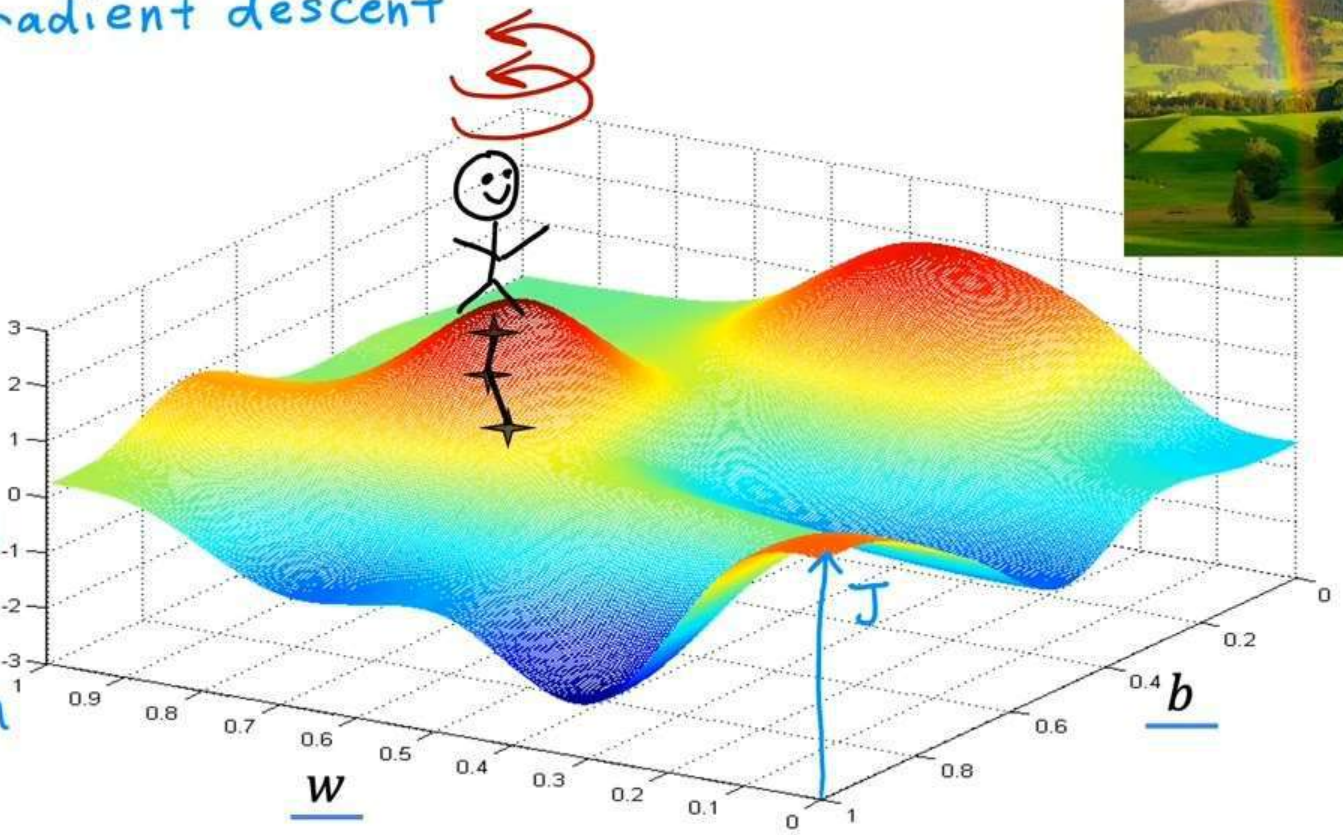
gradient descent



$J(w, b)$

not squared error cost

not linear regression



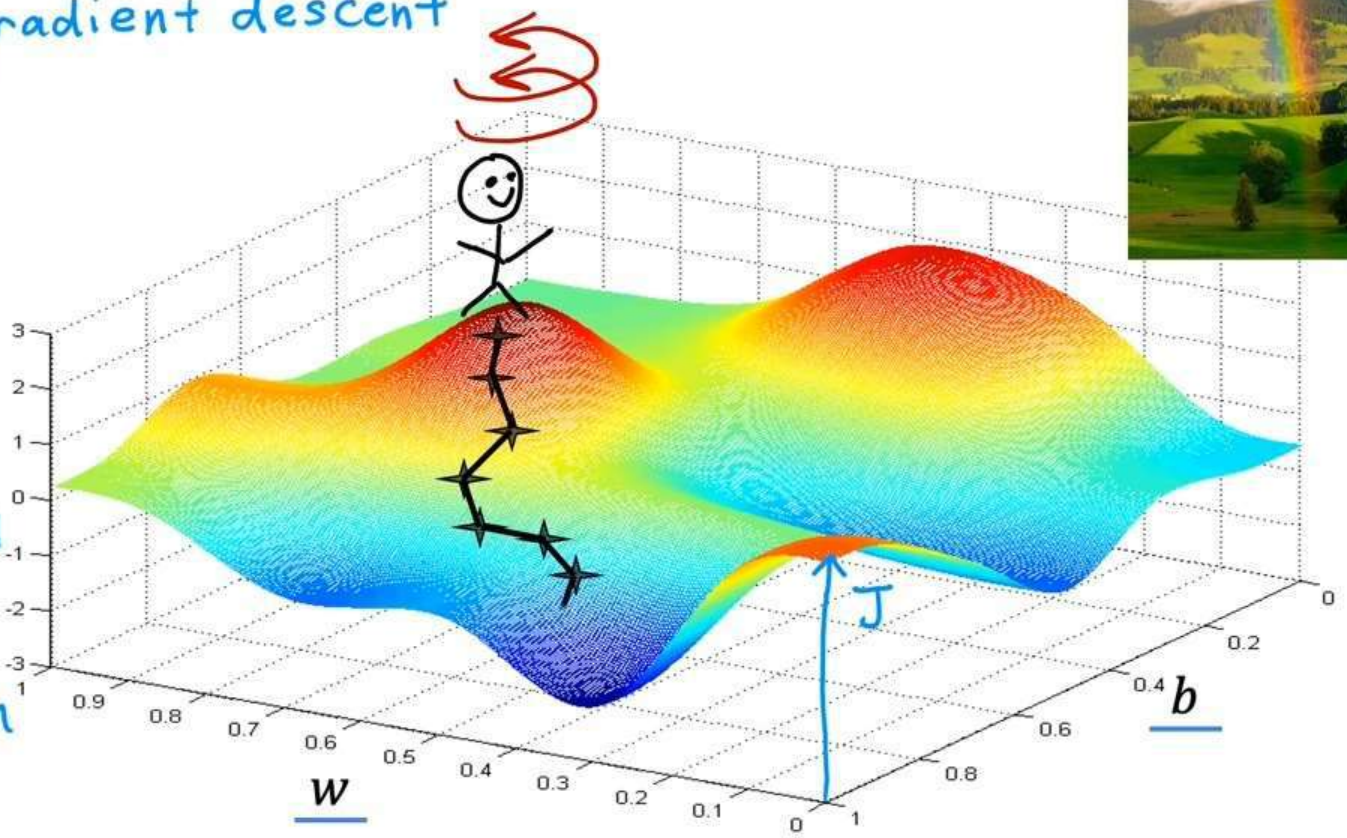
gradient descent



$J(w, b)$

not squared error cost

not linear regression



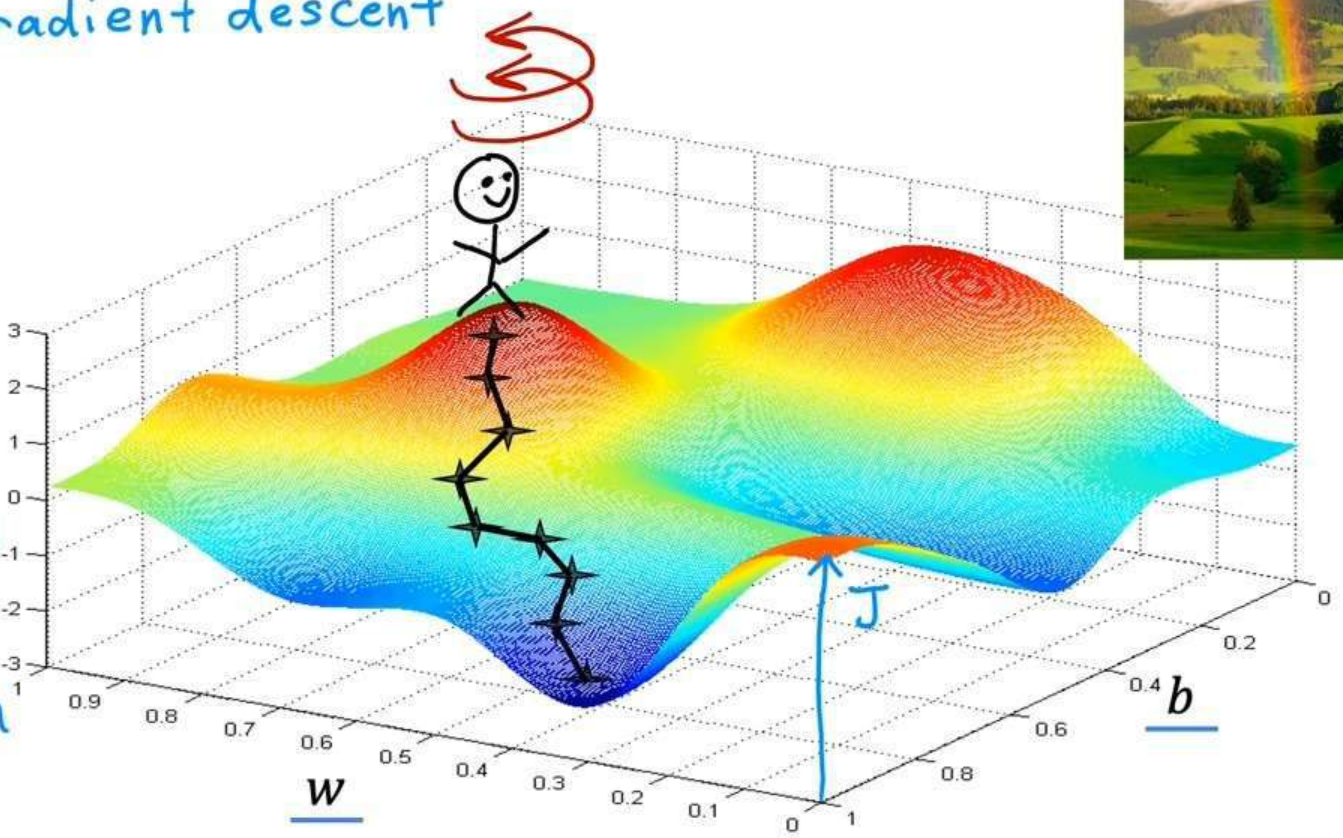
gradient descent



$J(w, b)$

not squared error cost

not linear regression



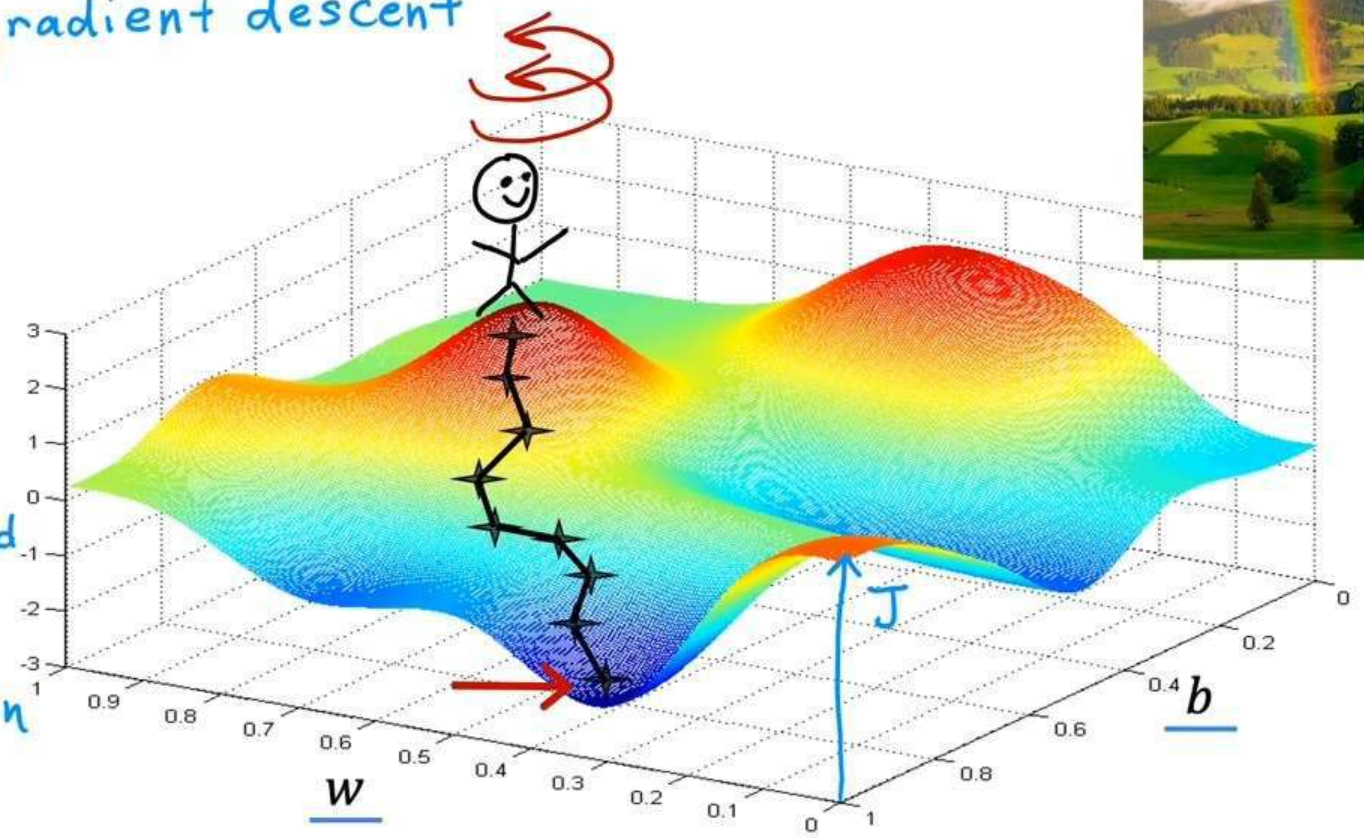
gradient descent



$J(w, b)$

not squared error cost

not linear regression



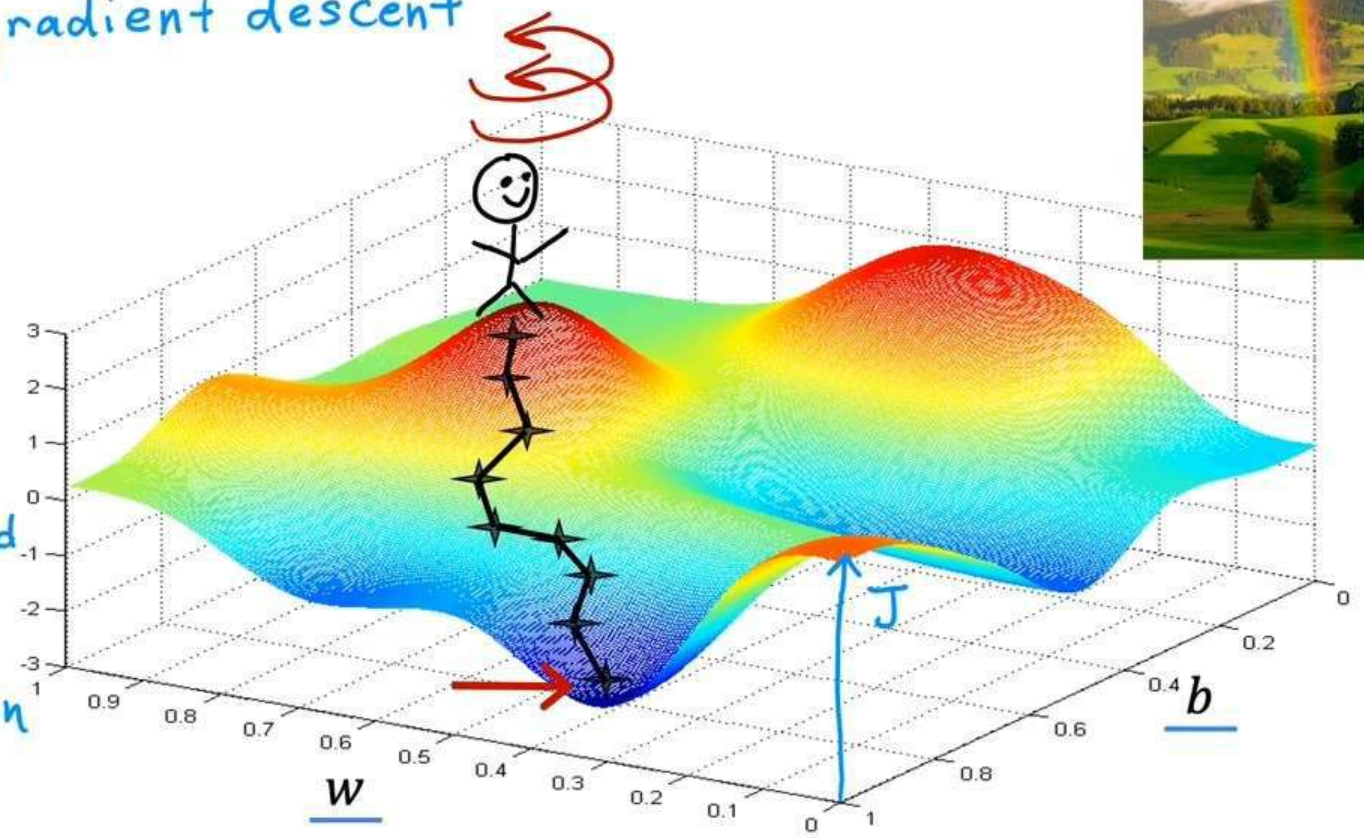
gradient descent



$J(w, b)$

not squared error cost

not linear regression



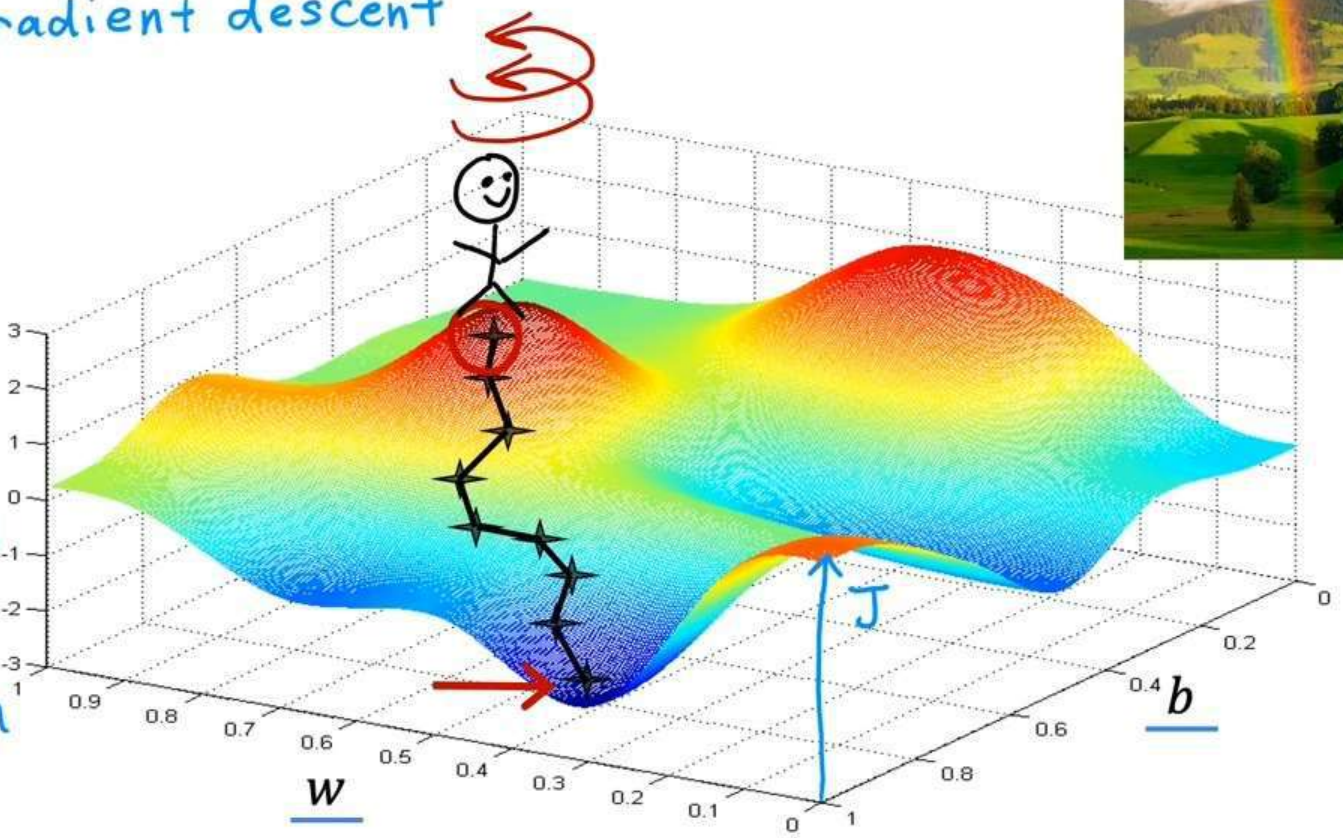
gradient descent



$J(w, b)$

not squared error cost

not linear regression



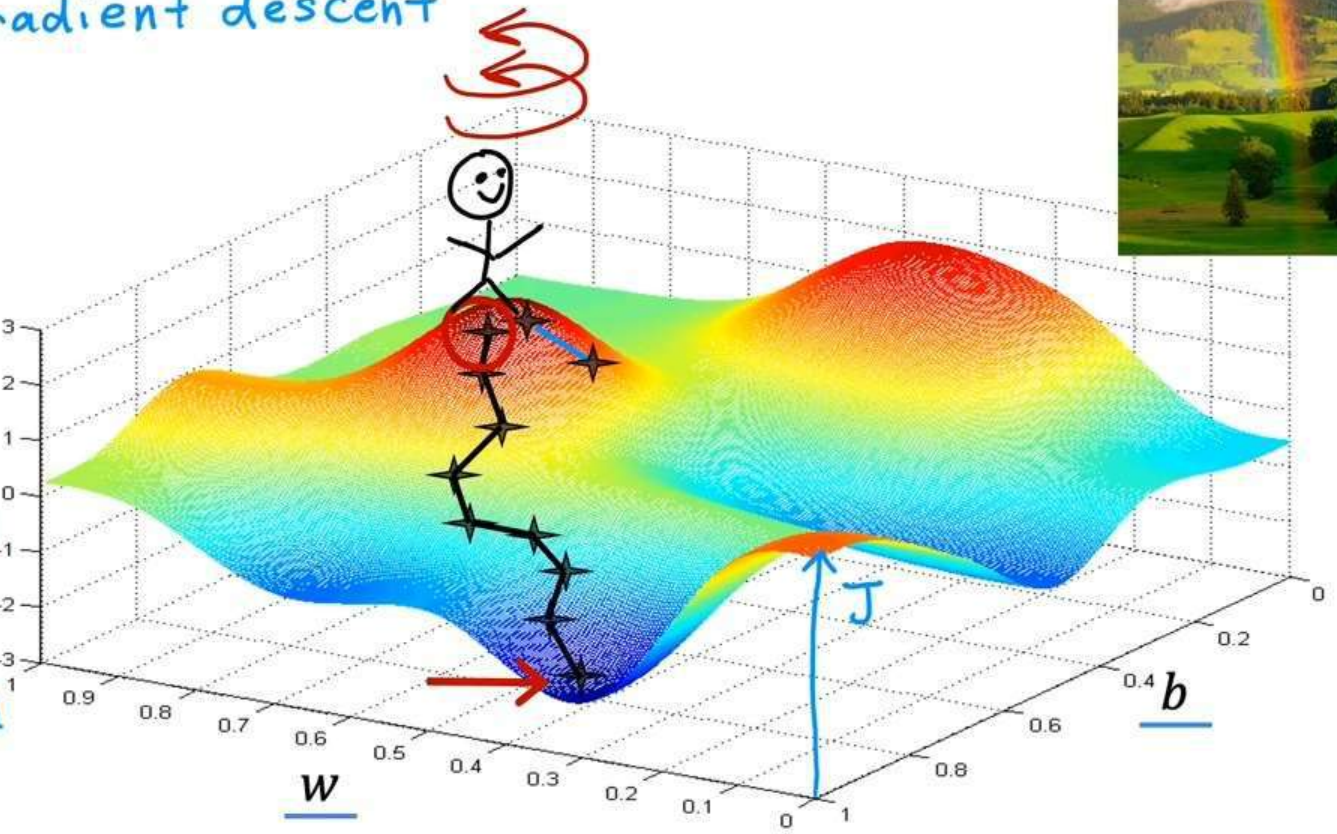
gradient descent



$J(w, b)$

not squared error cost

not linear regression



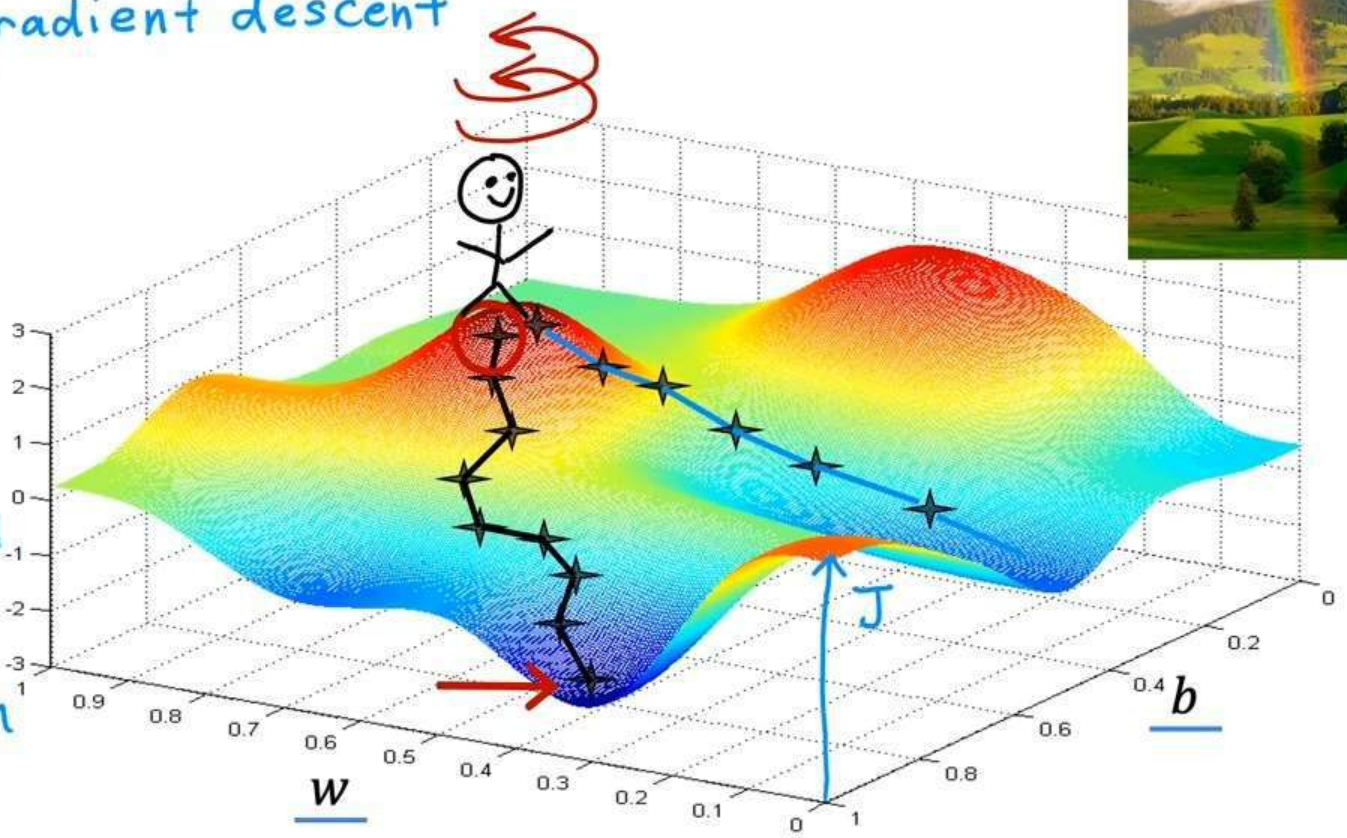
gradient descent



$J(w, b)$

not squared error cost

not linear regression



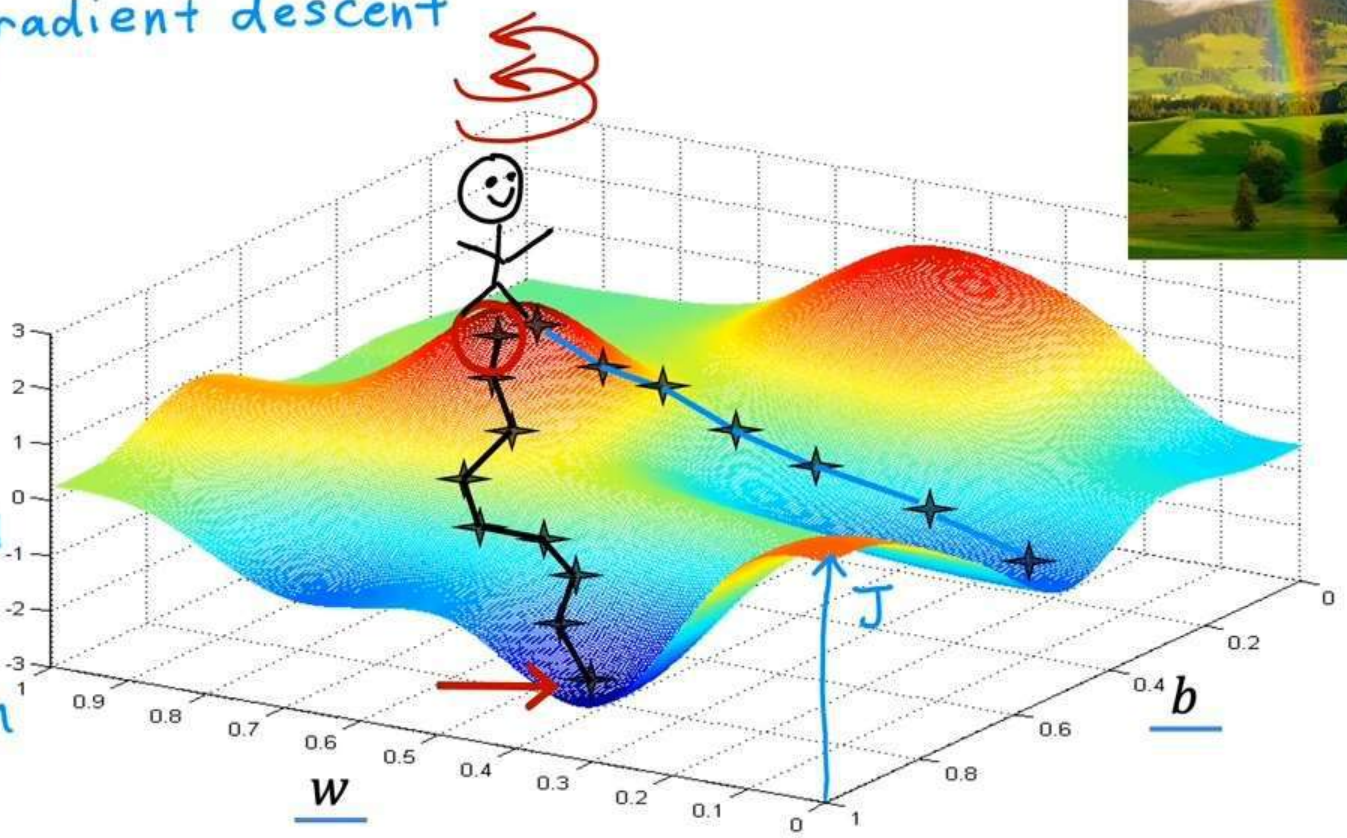
gradient descent



$J(w, b)$

not squared error cost

not linear regression



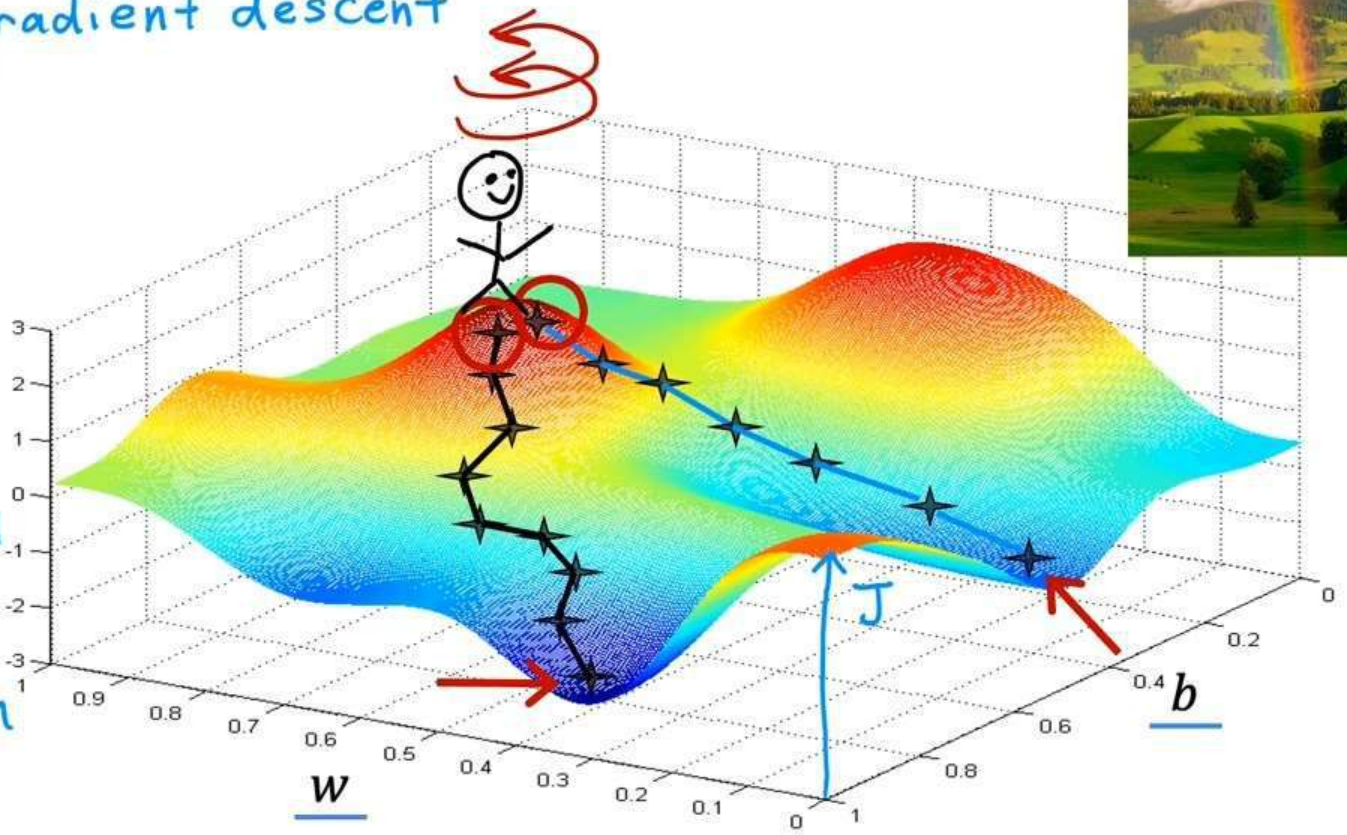
gradient descent



$J(w, b)$

not squared error cost

not linear regression



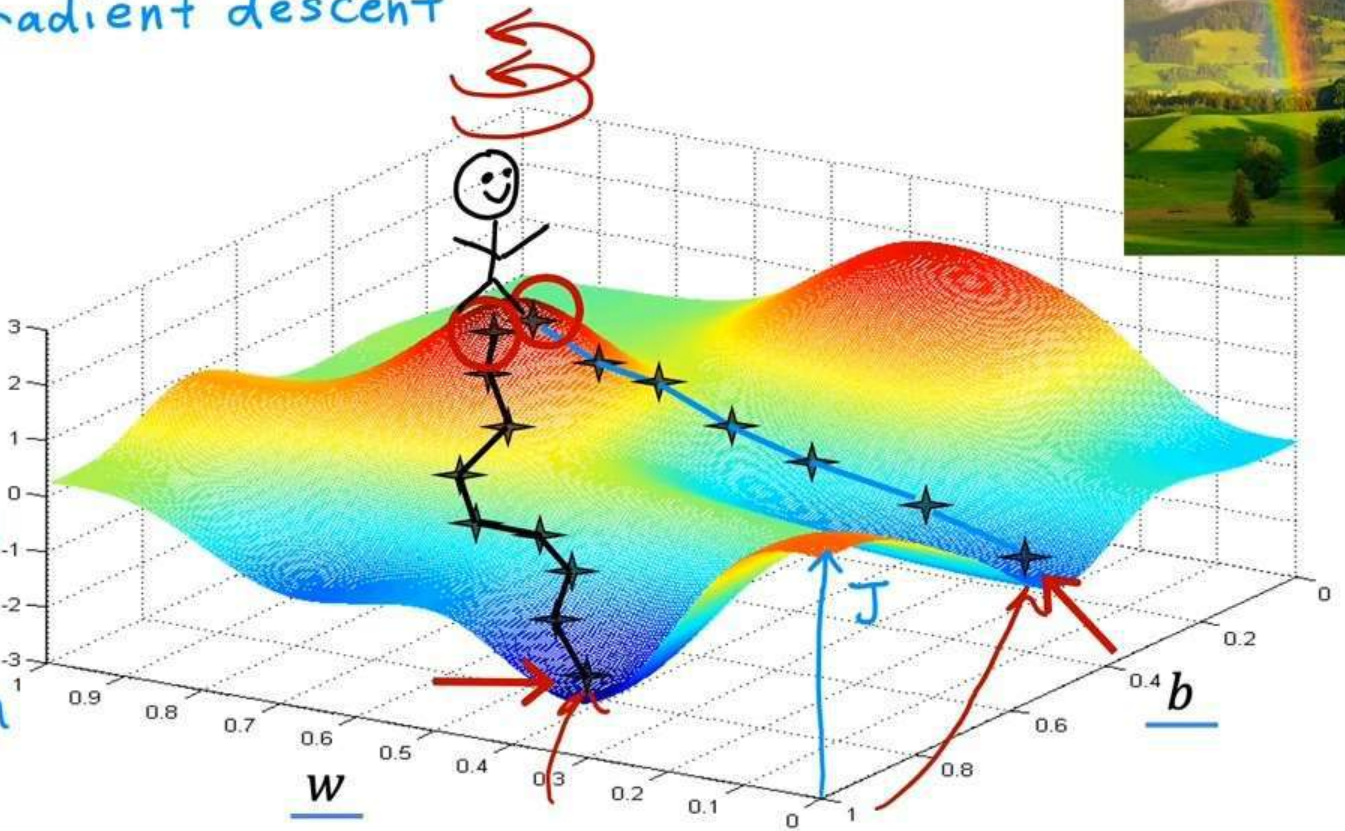
gradient descent

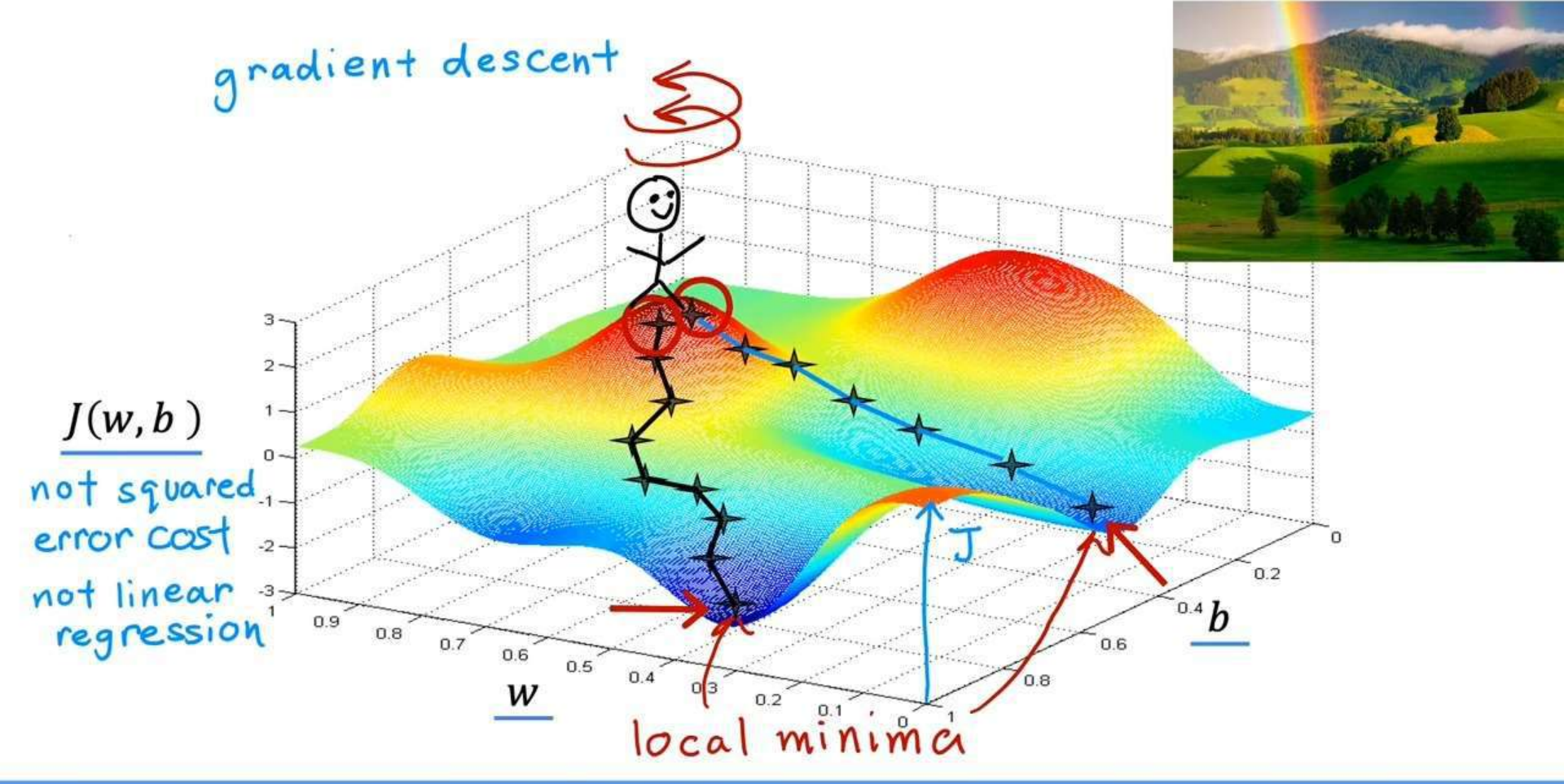


$J(w, b)$

not squared error cost

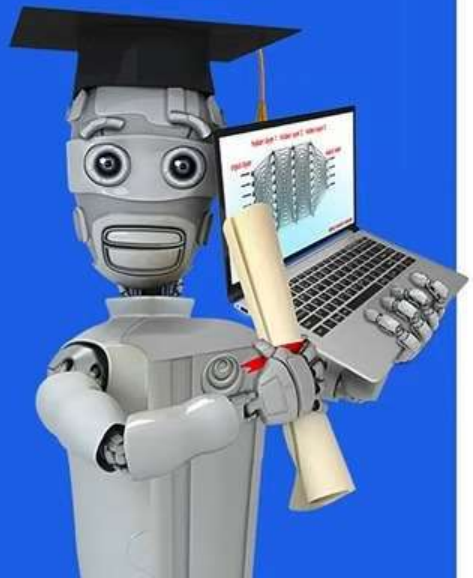
not linear regression




gradient descent
$J(w,b)$
not squared error cost
not linear regression
J
w
b
local minima

Stanford
ONLINE

DeepLearning.AI



Training Linear Regression

Implementing Gradient Descent

Gradient descent algorithm

$$w := w - \alpha \frac{d}{dw} J(w,b)$$

Assignment	Truth assertion
$a = c$	$a = c$
$a = a + 1$	$a = a + 1$
Code	Math

Gradient descent algorithm

$$w := w - \alpha \frac{d}{dw} J(w,b)$$

Learning rate

Assignment

$a = c$

$a = a + 1$

Code

Truth assertion

$a = c$

$a = a + 1$

Math

a==c

Gradient descent algorithm

Repeat until convergence

$$w = w - \alpha \frac{d}{dw} J(w,b)$$

$$b = b - \alpha \frac{d}{db} J(w,b)$$

Learning rate

Derivative

Simultaneously update w and b

Assignment

$a = c$

$a = a + 1$

Code

Truth assertion

$a = c$

$a = a + 1$

Math

a==c

Gradient descent algorithm

Repeat until convergence

$$w = w - \alpha \frac{d}{dw} J(w,b)$$

$$b = b - \alpha \frac{d}{db} J(w,b)$$

Learning rate

Derivative

Simultaneously update w and b

Assignment

$a = c$

$a = a + 1$

Code

Truth assertion

$a = c$

$a = a + 1$ ✗

Math

a==c

Correct: Simultaneous update

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$w = tmp_w$$

$$b = tmp_b$$

Incorrect

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

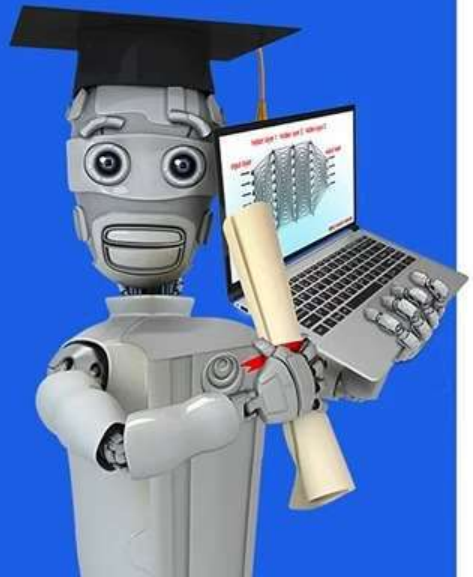
$$w = tmp_w$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

$$b = tmp_b$$

Stanford
ONLINE

DeepLearning.AI



Training Linear Regression

Gradient Descent Intuition

Gradient descent algorithm

repeat until convergence {

learning rate

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

derivative

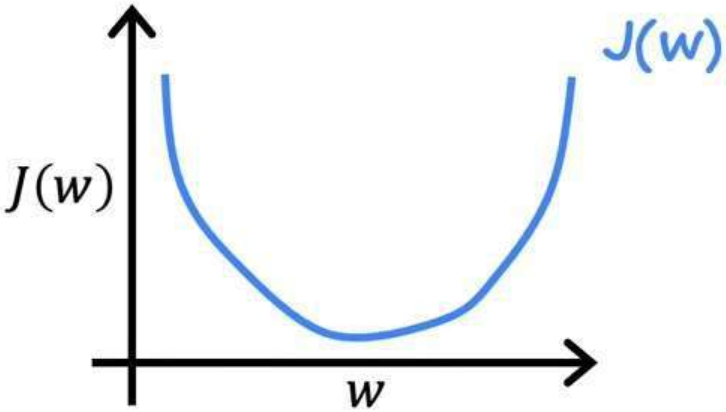
$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

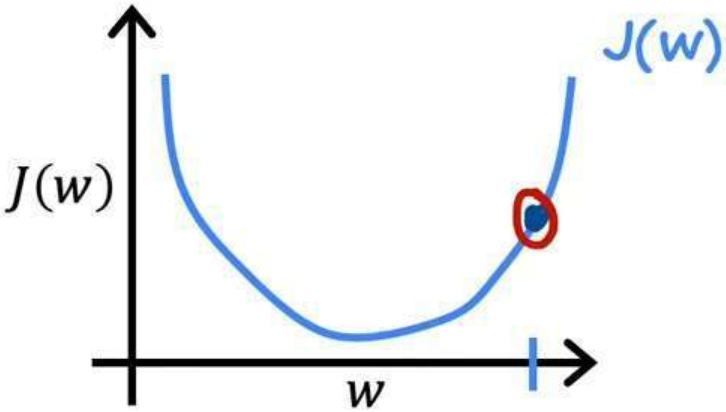
}

$J(w)$

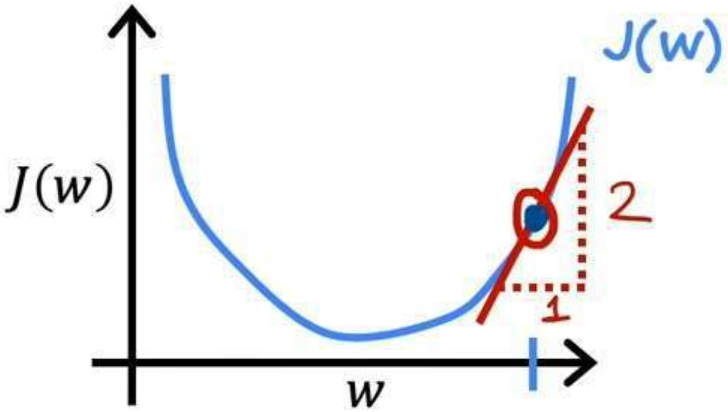
$$w = w - \alpha \frac{\partial}{\partial w} J(w)$$

$$\min_{w} J(w)$$

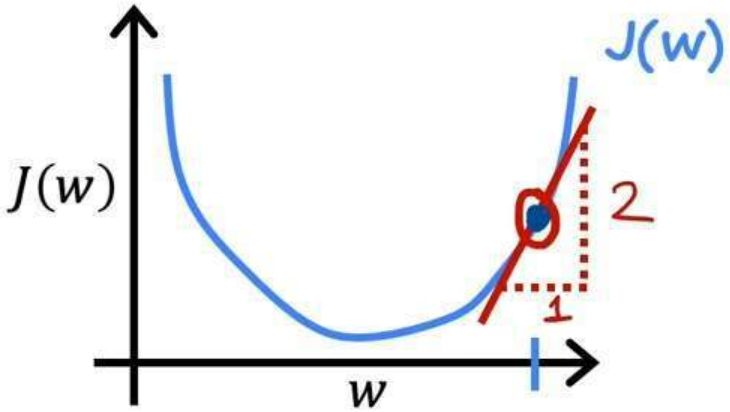

J(w)
J(w)
w



$$w = w - \alpha \frac{d}{dw} J(w)$$

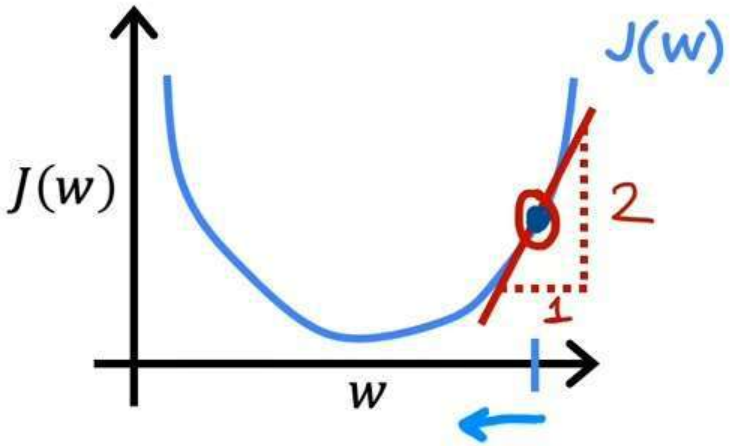


$$w = w - \alpha \frac{d}{dw} J(w)$$



$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

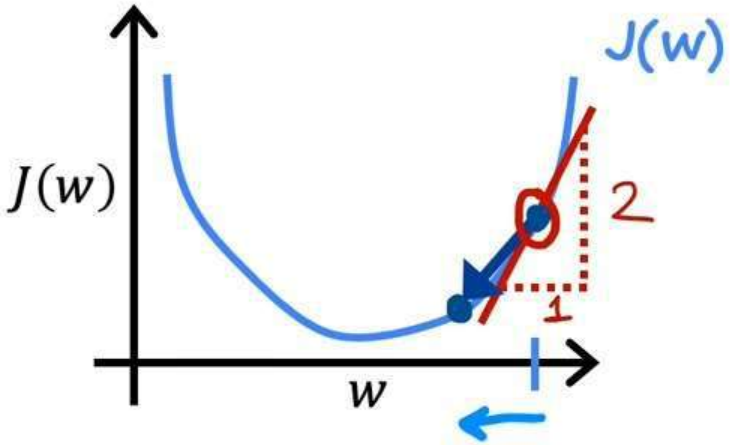
$$w = w - \alpha \cdot (positive\ number)$$



$$w = w - \alpha \frac{d}{dw} J(w)$$

$$> 0$$

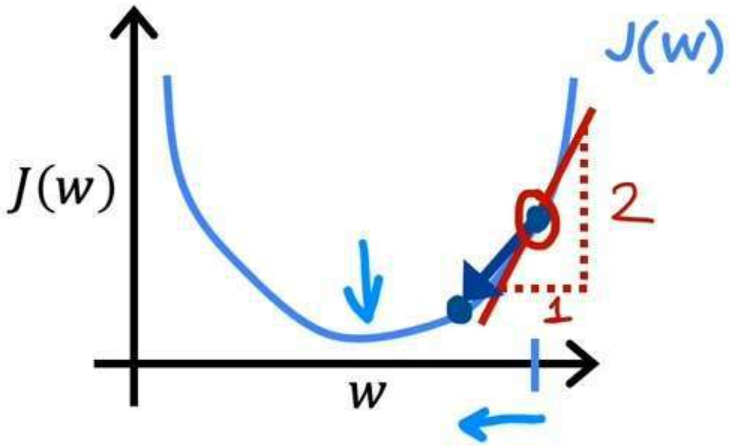
$$w = w - \alpha \cdot (positive\ number)$$



$$w = w - \alpha \frac{d}{dw} J(w)$$

$$> 0$$

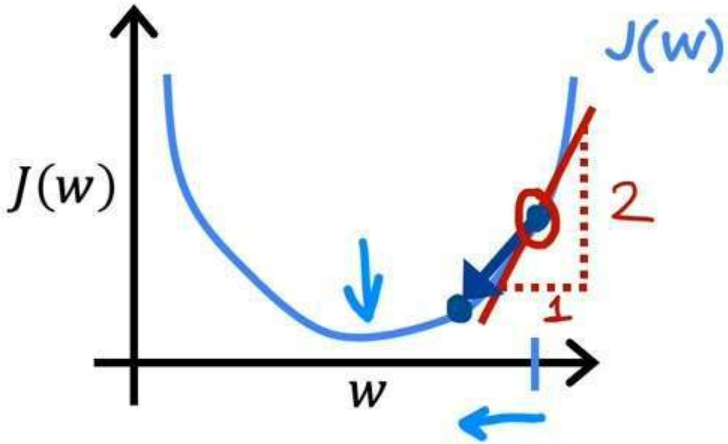
$$w = w - \alpha \cdot (positive\ number)$$



$$w = w - \alpha \frac{d}{dw} J(w)$$

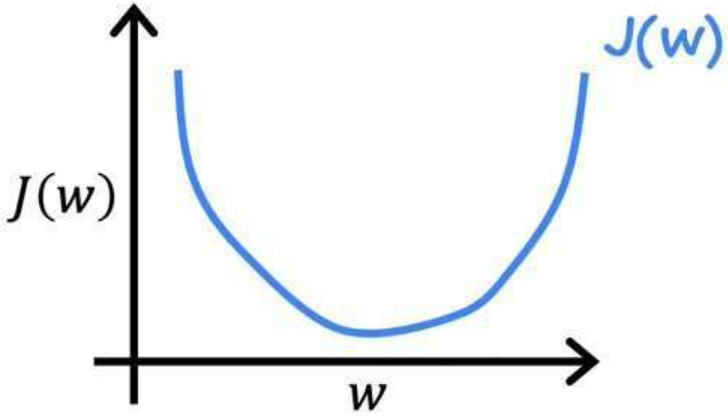
$$> 0$$

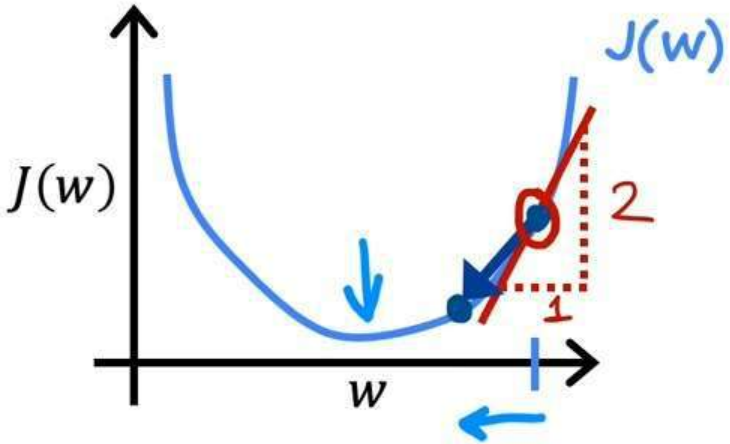
$$w = w - \alpha \cdot (positive\ number)$$



$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \alpha \cdot (positive\ number)$$

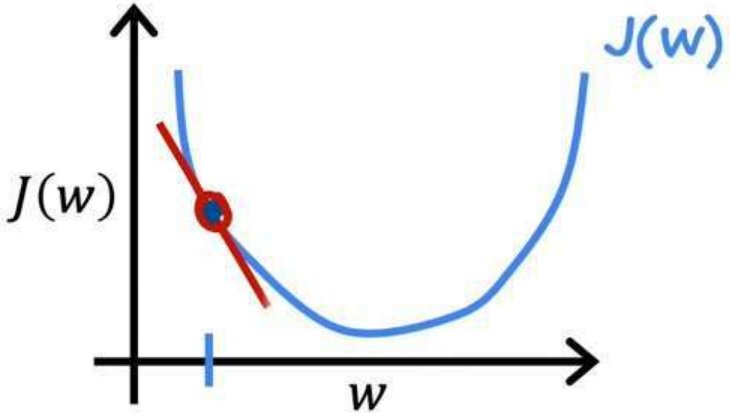


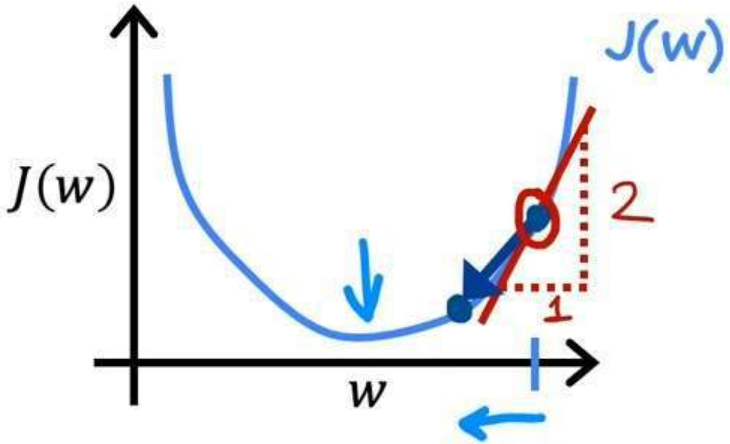


$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \alpha \cdot (positive\ number)$$

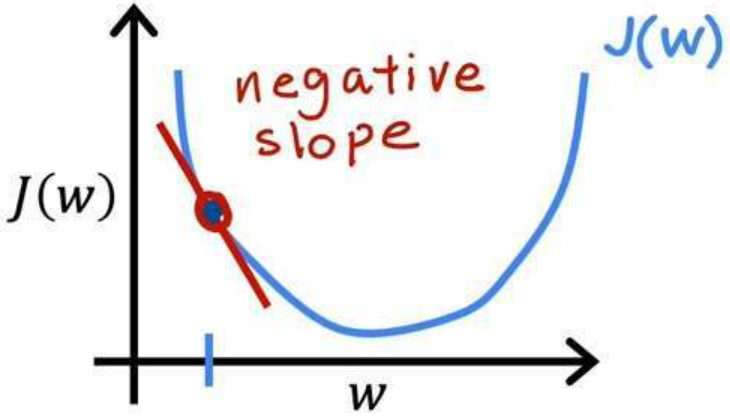
$$\frac{d}{dw} J(w)$$



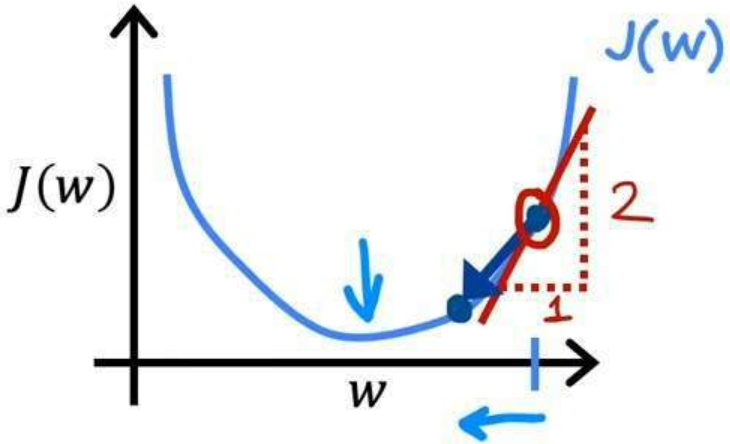


$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

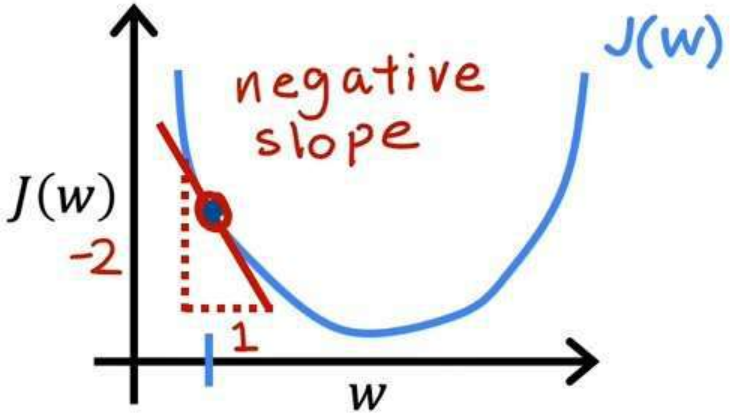
$$w = w - \alpha \cdot (positive\ number)$$



$$\frac{d}{dw} J(w)$$

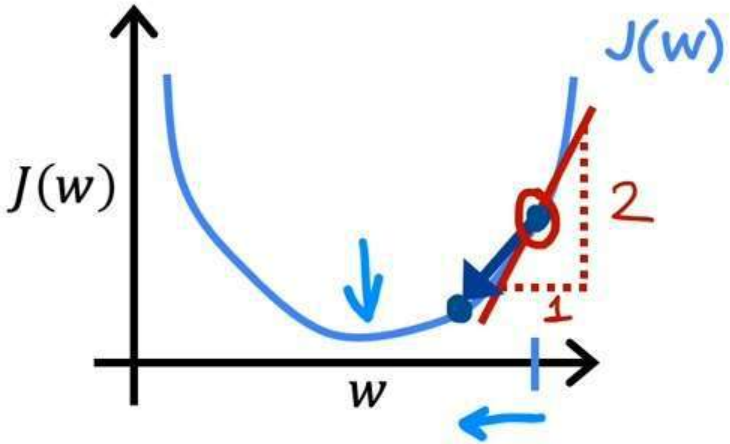


$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$



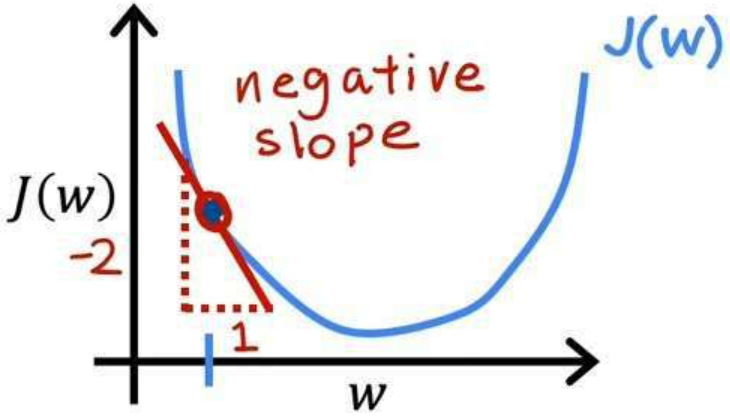
$$w = w - \alpha \cdot (positive\ number)$$

$$\frac{d}{dw} J(w)$$



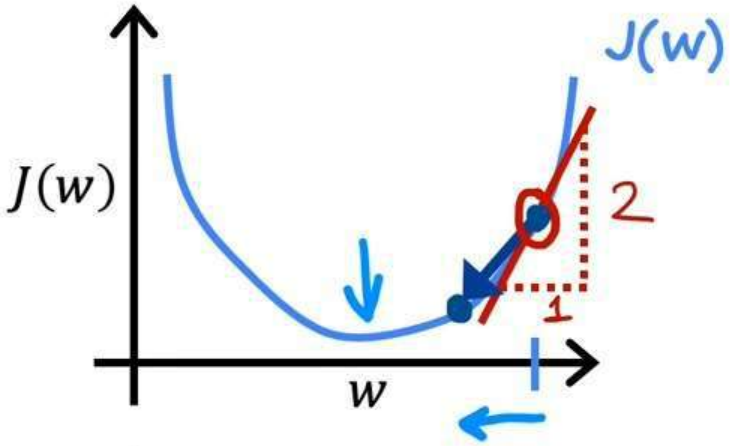
$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \alpha \cdot (\text{positive number})$$



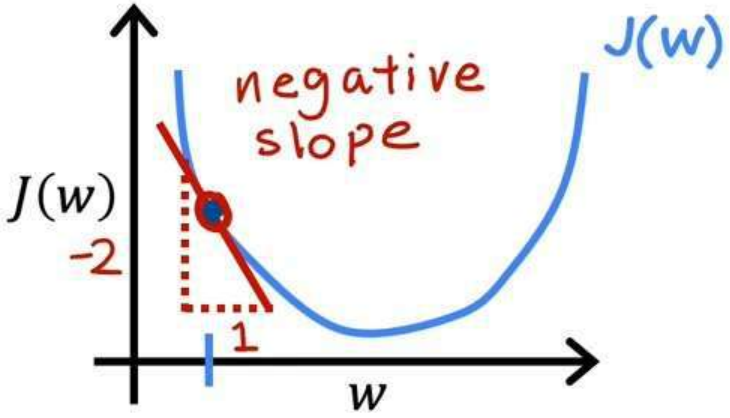
$$\underbrace{\frac{d}{dw} J(w)}_{<0}$$

$$w = w - \alpha \cdot (\text{negative number})$$



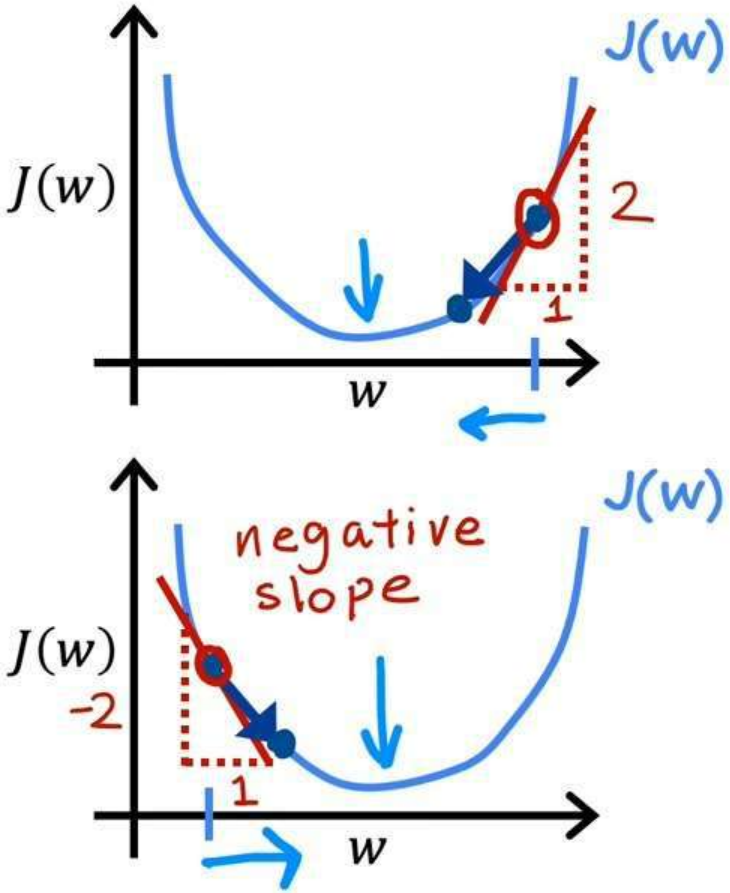
$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

$$w = w - \alpha \cdot (\text{positive number})$$



$$\underbrace{\frac{d}{dw} J(w)}_{<0}$$

$$w = w - \alpha \cdot (\text{negative number})$$



$$w = w - \alpha \underbrace{\frac{d}{dw} J(w)}_{>0}$$

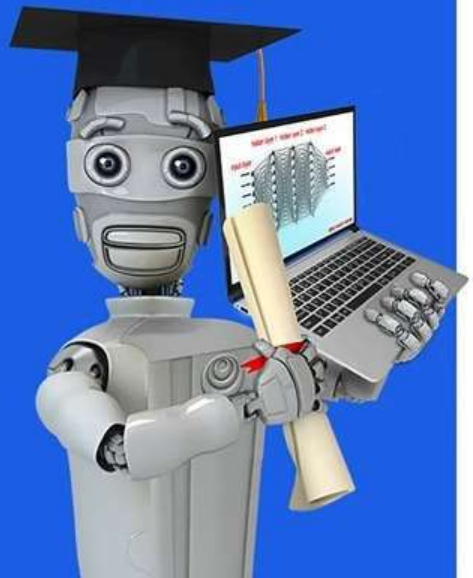
$$w = w - \alpha \cdot (\text{positive number})$$

$$\underbrace{\frac{d}{dw} J(w)}_{<0}$$

$$w = w - \alpha \cdot (\text{negative number})$$

Stanford
ONLINE

DeepLearning.AI



Training Linear Regression

Learning Rate

$$w = w - \alpha \frac{d}{dw} J(w)$$

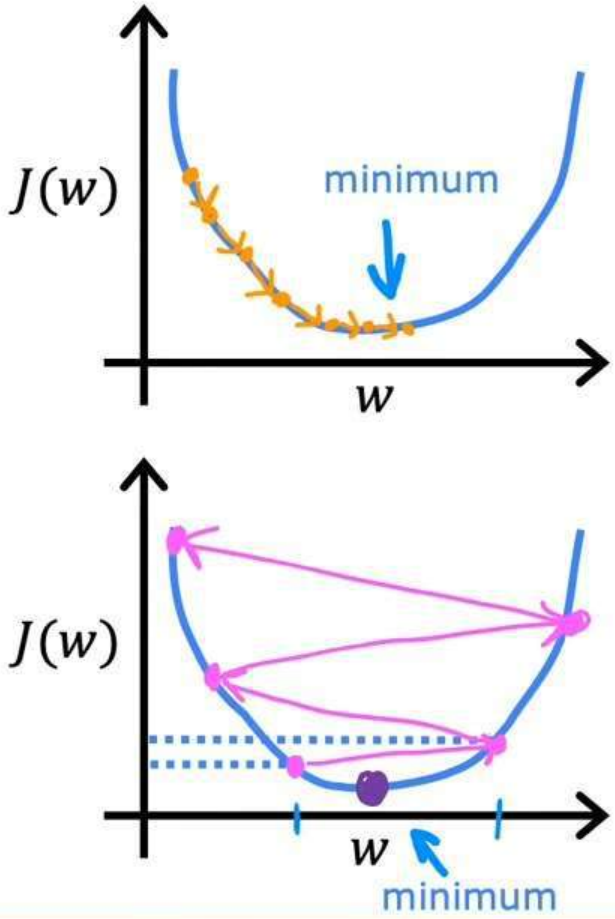
If α is too small…

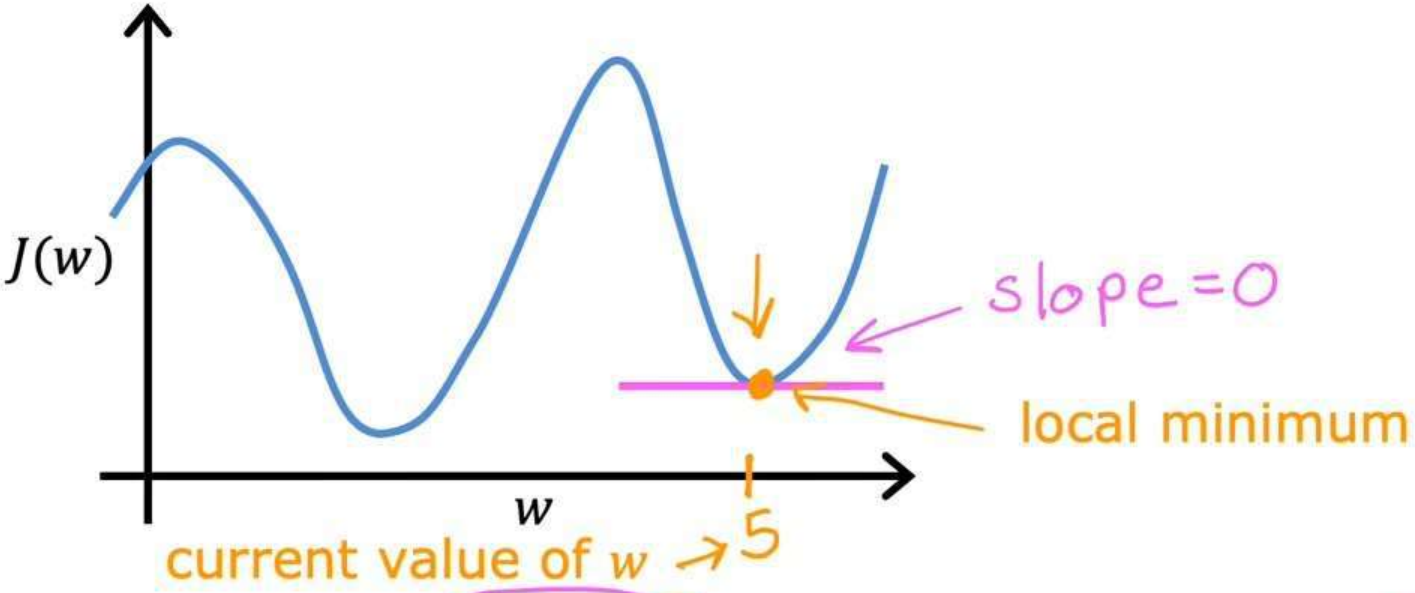
Gradient descent may be slow.

If α is too large…

Gradient descent may:

- Overshoot, never reach minimum
- Fail to converge, diverge



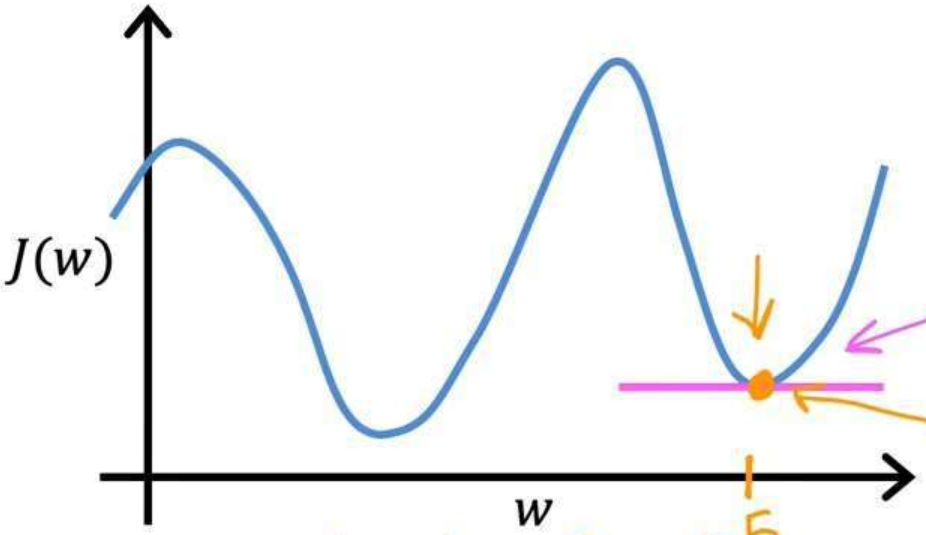


$$w = w - \alpha \frac{d}{dw} J(w)$$

=0

$$w = w - \alpha \cdot 0$$

$$w = w$$



$$w = w - \alpha \frac{d}{dw} J(w)$$

0.1

$=0$

$5 - 0.1 \cdot 0$

$w = w - \alpha \cdot 0$

5

$w = w$

Can reach local minimum with fixed learning rate

$$w = w - \alpha \frac{d}{dw} J(w)$$

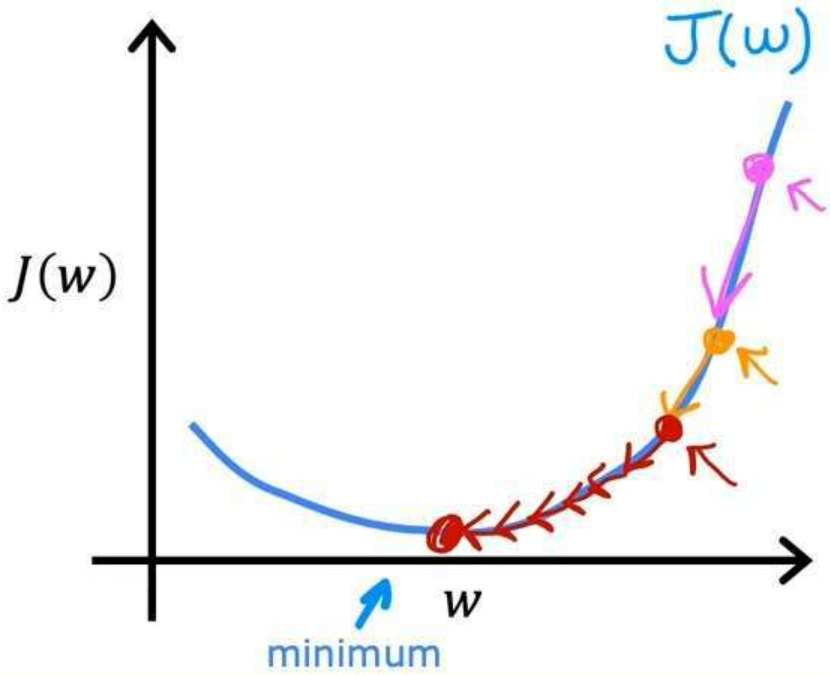
smaller

not as large

large

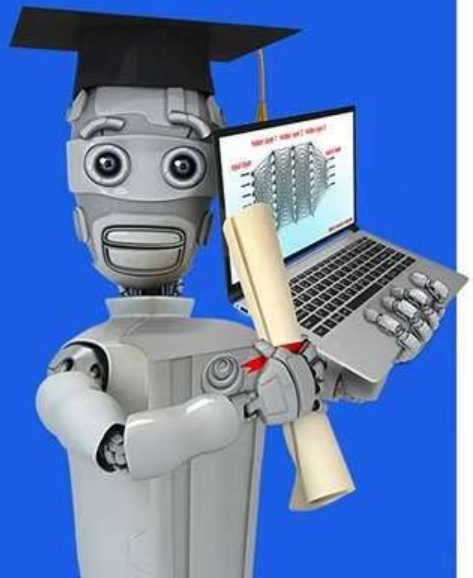
Near a local minimum,
- Derivative becomes smaller
- Update steps become smaller

Can reach minimum without decreasing learning rate α



Stanford
ONLINE

DeepLearning.AI



Training Linear Regression

Gradient Descent for Linear Regression

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

}

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$$

}

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w,b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w,b) \rightarrow \frac{1}{m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})$$

}

(Optional)

$$\frac{\partial}{\partial w} J(w,b) = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 = \frac{d}{dw} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right)^2$$

$$= \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right) 2x^{(i)} = \boxed{\frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right) x^{(i)}}$$

$$\frac{\partial}{\partial b} J(w,b) = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 = \frac{d}{db} \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right)^2$$

$$= \frac{1}{2m} \sum_{i=1}^{m} \left(wx^{(i)} + b - y^{(i)}\right) 2 = \boxed{\frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)}\right)}$$

no $x^{(i)}$

Gradient descent algorithm

$\frac{\partial}{\partial w} J(w,b)$

repeat until convergence {

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)}) \quad x^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{w,b}(x^{(i)}) - y^{(i)})$$

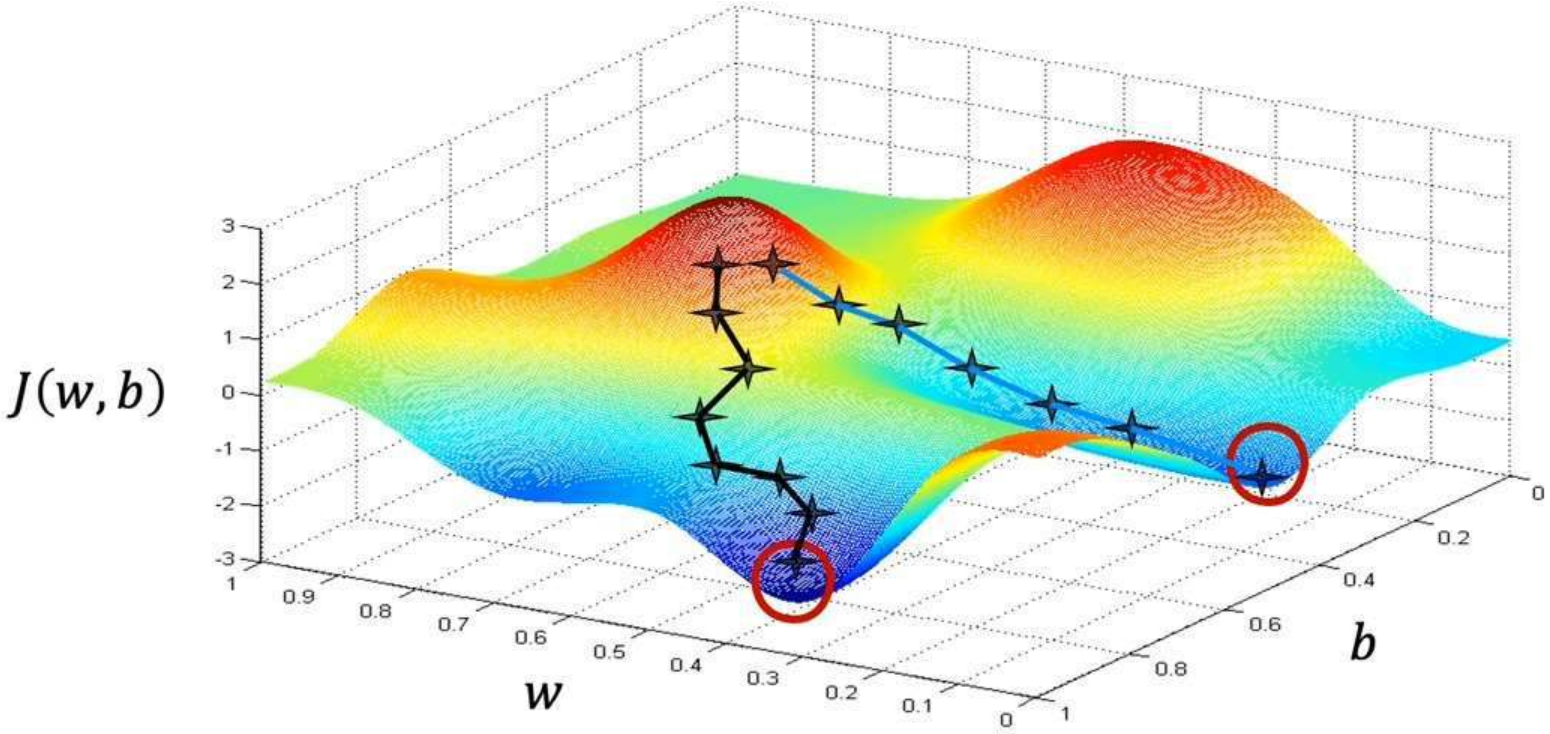
}

Update w and b simultaneously

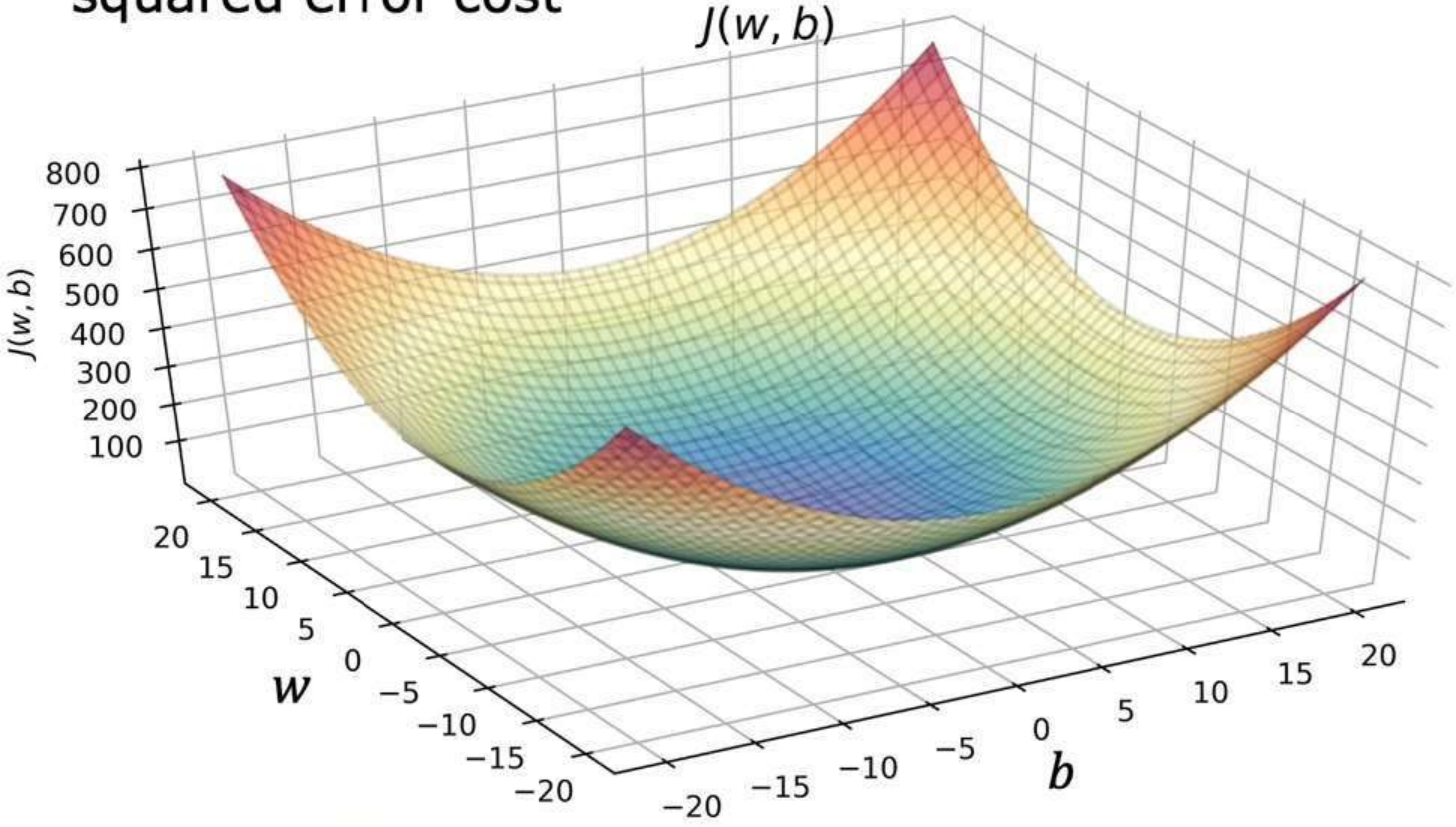
$\frac{\partial}{\partial b} J(w,b)$

$f_{w,b}(x^{(i)}) = wx^{(i)} + b$

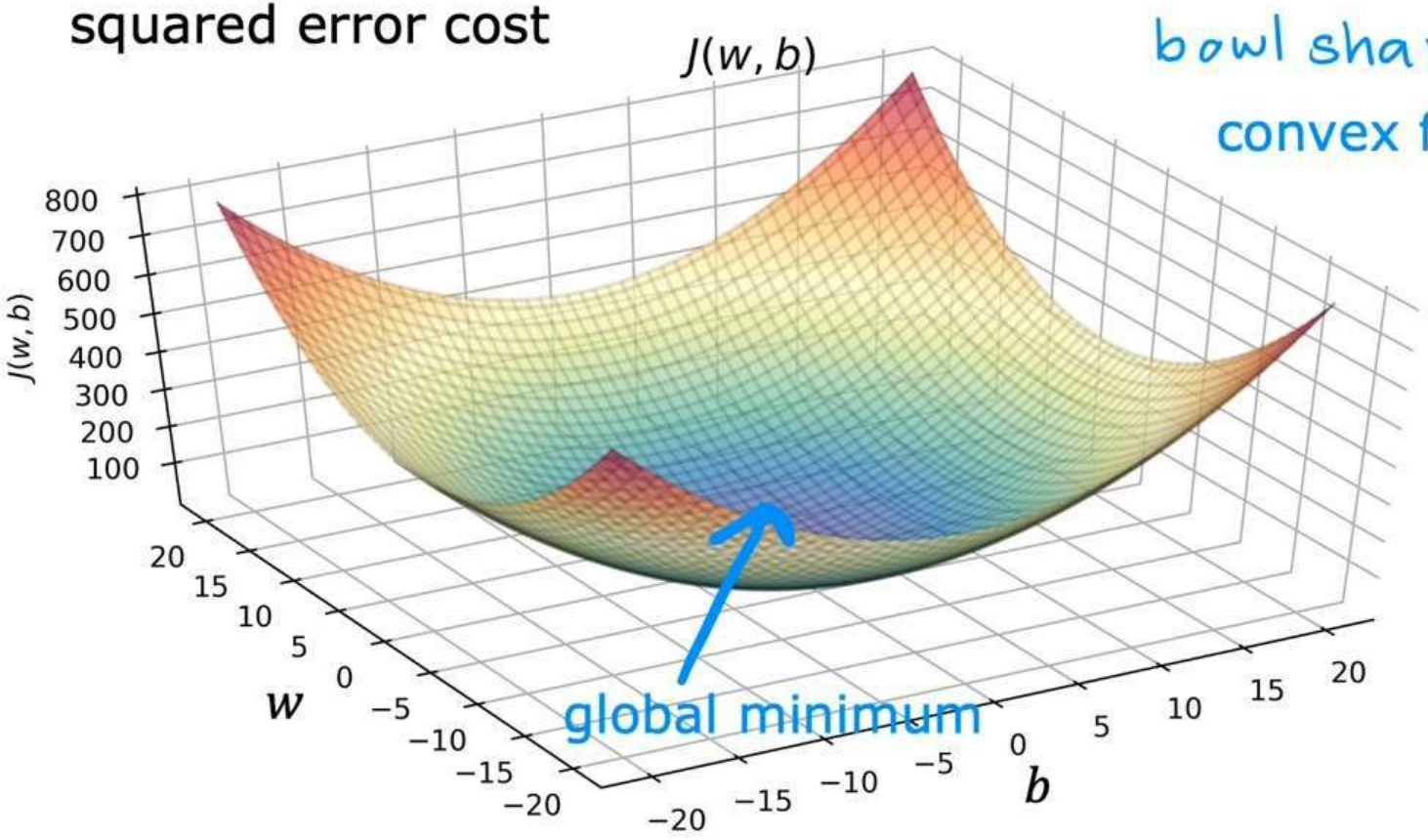
More than one local minimum



squared error cost

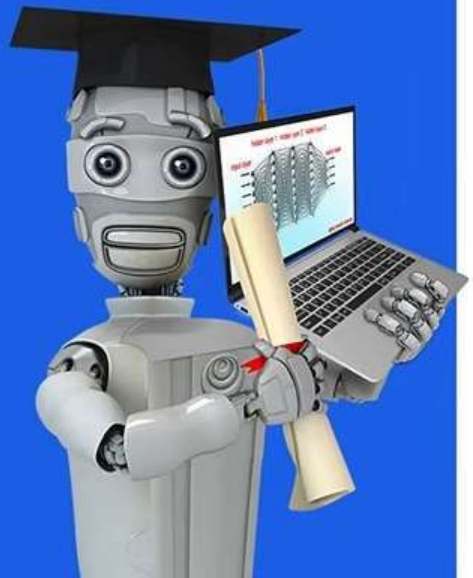


squared error cost



Stanford
ONLINE

DeepLearning.AI



Linear Regression with Multiple Variables

Multiple Features

Multiple features (variables)

one feature →

Size in feet² (x)	Price ($) in 1000's ($y$) ←
2104	400
1416	232
1534	315
852	178
...	...

$$f_{w,b}(x) = wx + b$$

Multiple features (variables)

Size in feet^2	Number of bedrooms	Number of floors	Age of home in years	Price ($) in $1000's
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

Multiple features (variables)

Size in feet²	Number of bedrooms	Number of floors	Age of home in years	Price ($) in $1000's
x_1	x_2	x_3	x_4	
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

$j = 1 \ldots 4$

$n = 4$

$i = 2$

x_j = j^{th} feature

n = number of features

$\vec{x}^{(i)}$ = features of ith training example

$x_j^{(i)}$ = value of feature j in i^{th} training example

$\vec{x}^{(2)} = [1416 \quad 3 \quad 2 \quad 40]$

$x_3^{(2)} = 2$

Model:

Previously: $f_{w,b}(x) = wx + b$

$$f_{w,b}(X) = w_1 X_1 + w_2 X_2 + w_3 X_3 + w_4 X_4 + b$$

example

$$f_{w,b}(X) = 0.1 X_1 + 4 X_2 + 10 X_3 + -2 X_4 + 80$$

X_1: size
X_2: #bedrooms
X_3: #floors
X_4: years
80: base price

$$f_{w,b}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

$\vec{w} = [w_1 \; w_2 \; w_3 \ldots w_n]$ parameters of the model

b is a number

vector $\vec{x} = [x_1 \; x_2 \; x_3 \ldots x_n]$

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_n x_n + b$$

dot product

multiple linear regression

Parameters and features

$\vec{w} = [w_1 \quad w_2 \quad w_3]$ $n=3$

b is a number

$\vec{x} = [x_1 \quad x_2 \quad x_3]$

linear algebra: count from 1

NumPy

w[0] w[1] w[2]

```
w = np.array([1.0,2.5,-3.3])
b = 4
x = np.array([10,20,30])
```

x[0] x[1] x[2]

code: count from 0

Without vectorization $n = 100,000$

$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$

```
f = w[0] * x[0] +
    w[1] * x[1] +
    w[2] * x[2] + b
```



Without vectorization

$$f_{\vec{w},b}(\vec{x}) = \left(\sum_{j=1}^{n} w_j x_j\right) + b$$

$\sum_{j=1}^{n} \rightarrow j = 1 \ldots n$ 1, 2, 3

range(0, n) → j = 0 ... n-1

```
f = 0      range(n)
for j in range(0,n):
    f = f + w[j] * x[j]
f = f + b
```



Vectorization

$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

```
f = np.dot(w,x) + b
```



Without vectorization

```
for j in range(0,16):
    f = f + w[j] * x[j]
```

t_0

```
f + w[0] * x[0]
```

t_1

```
f + w[1] * x[1]
```

...

t_{15}

```
f + w[15] * x[15]
```

Vectorization

Without vectorization

```
for j in range(0,16):
    f = f + w[j] * x[j]
```

t_0

```
f + w[0] * x[0]
```

t_1

```
f + w[1] * x[1]
```

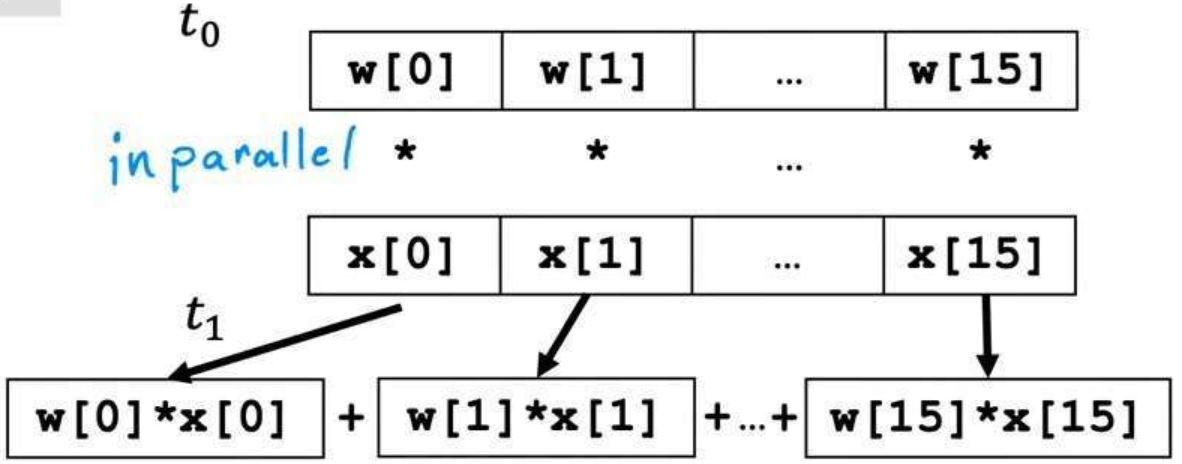
...

t_{15}

```
f + w[15] * x[15]
```

Vectorization

```
np.dot(w,x)
```



efficient → scale to large datasets

Gradient descent

$\vec{w} = (w_1 \quad w_2 \quad \cdots \quad w_{16})$ ~~b~~ parameters

derivatives $\vec{d} = (d_1 \quad d_2 \quad \cdots \quad d_{16})$

```
w = np.array([0.5, 1.3, … 3.4])
d = np.array([0.3, 0.2, … 0.4])
```

learning rate α

compute $w_j = w_j - 0.1 d_j$ for $j = 1 \ldots 16$

Gradient descent

$\vec{w} = (w_1 \quad w_2 \quad \cdots \quad w_{16})$ ~~b~~ parameters

derivatives $\vec{d} = (d_1 \quad d_2 \quad \cdots \quad d_{16})$

```
w = np.array([0.5, 1.3, … 3.4])
d = np.array([0.3, 0.2, … 0.4])
```

learning rate α

compute $w_j = w_j - 0.1 d_j$ for $j = 1 \ldots 16$

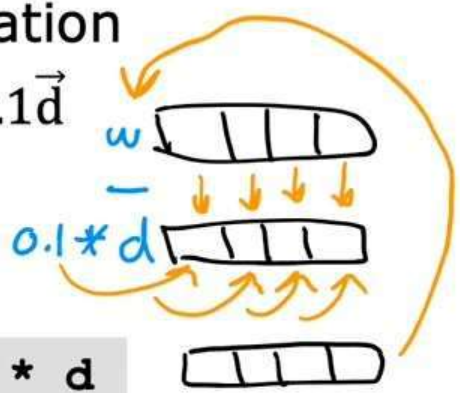
Without vectorization

$$w_1 = w_1 - 0.1 d_1$$
$$w_2 = w_2 - 0.1 d_2$$
$$\vdots$$
$$w_{16} = w_{16} - 0.1 d_{16}$$

```
for j in range(0,16):
    w[j] = w[j] - 0.1 * d[j]
```

With vectorization

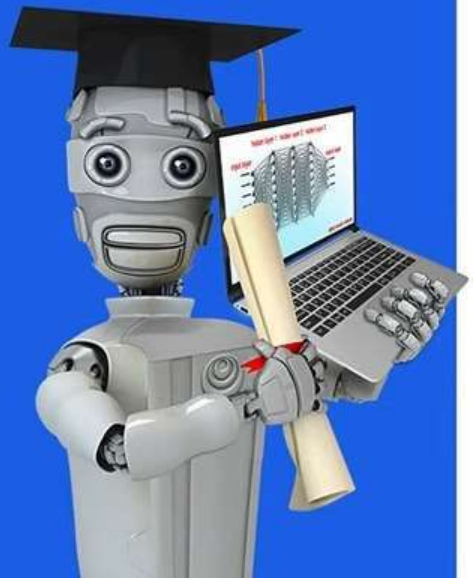
$$\vec{w} = \vec{w} - 0.1\vec{d}$$




```
w = w - 0.1 * d
```

Stanford
ONLINE

DeepLearning.AI



Linear Regression with Multiple Variables

Gradient Descent for Multiple Regression

	Previous notation	Vector notation
Parameters	$w_1, \cdots, w_n$ b	$\vec{w} = [w_1 \quad \cdots \quad w_n]$ ← vector of length n b still a number
Model	$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \cdots + w_n x_n + b$	$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$ ← dot product
Cost function	$J(w_1, \cdots, w_n, b)$	$J(\vec{w}, b)$

Gradient descent

Previous notation

Parameters $w_1, \cdots, w_n$

b

Model $f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \cdots + w_n x_n + b$

Cost function $J(w_1, \cdots, w_n, b)$

Vector notation

$\vec{w} = [w_1 \quad \cdots \quad w_n]$ ← vector of length n

b still a number

$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$ ← dot product

$J(\vec{w}, b)$

Gradient descent

repeat {

$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w_1, \cdots, w_n, b)$

$b = b - \alpha \frac{\partial}{\partial b} J(w_1, \cdots, w_n, b)$

}

repeat {

$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$

$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$

}

Gradient descent

One feature

repeat {

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}\left(x^{(i)}\right) - y^{(i)} \right) x^{(i)}$$

$$\frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}\left(x^{(i)}\right) - y^{(i)} \right)$$

simultaneously update w, b

}

n features ($n \geq 2$)

repeat {

$j = 1$

$$w_1 = w_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)} \right) x_1^{(i)}$$

$$\frac{\partial}{\partial w_1} J(\vec{w}, b)$$

$\vdots$

$j = n$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)} \right) x_n^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}\left(\vec{x}^{(i)}\right) - y^{(i)} \right)$$

simultaneously update w_j (for $j = 1, \cdots, n$) and b

}

An alternative to gradient descent

Normal equation
- Only for linear regression
- Solve for w, b without iterations

Disadvantages
- Doesn't generalize to other learning algorithms.
- Slow when number of features is large (> 10,000)

What you need to know
- Normal equation method may be used in machine learning libraries that implement linear regression.
- Gradient descent is the recommended method for finding parameters w,b

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000

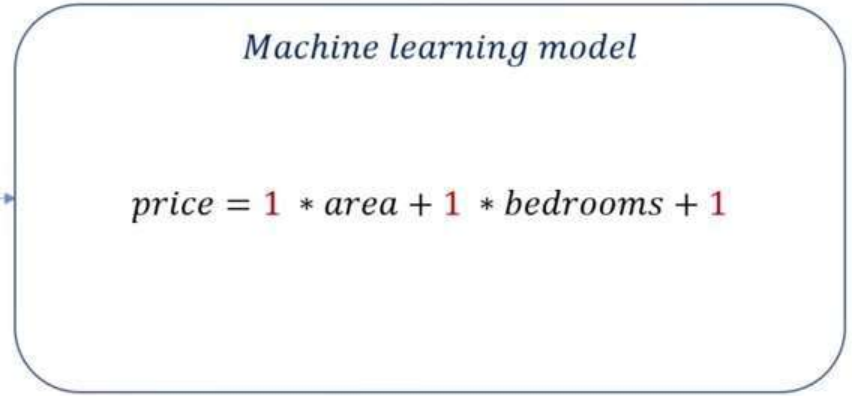
6 *samples*

Machine learning model

$$price = w1 * area + w2 * bedrooms + bias$$

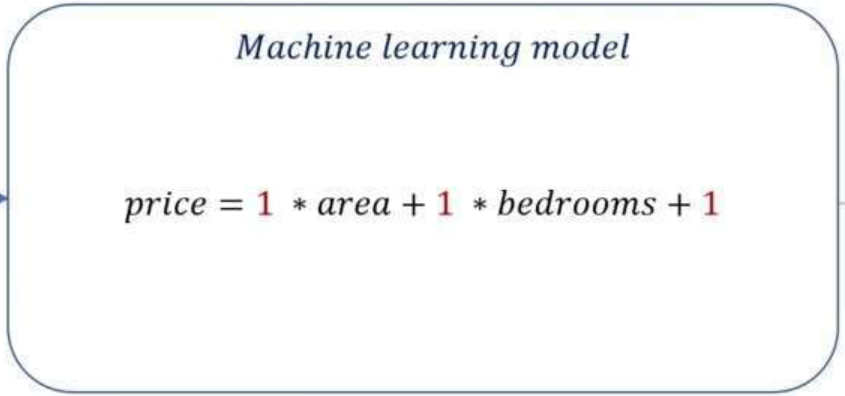
Initialize w1, w2 and bias to be 1

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000



Initialize w1, w2 and bias to be 1

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000

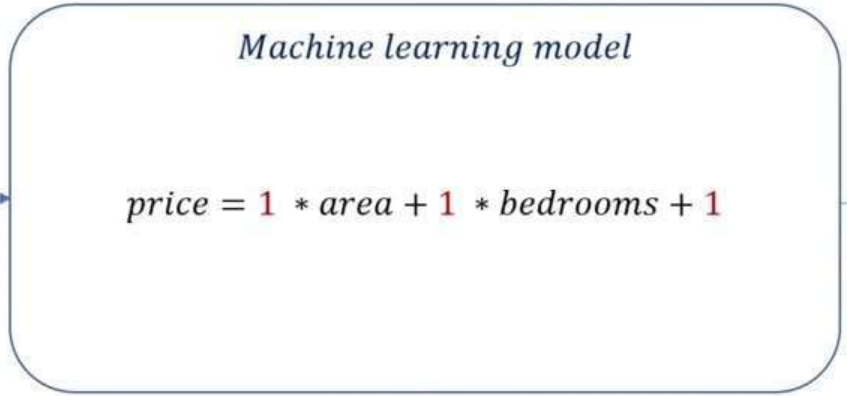


$\widehat{price} = 2604$

$price = 550000$

$error1 = (price - \widehat{price})^2$

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000

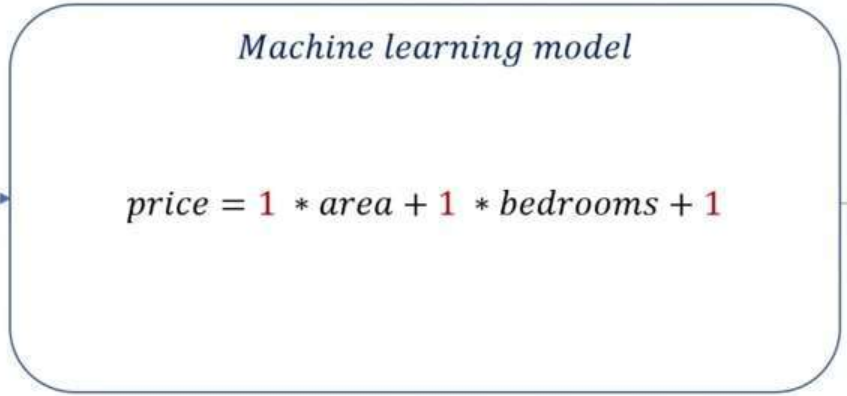


$\widehat{price} = 3005$

$price = 565000$

$error2 = (price - \widehat{price})^2$

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000

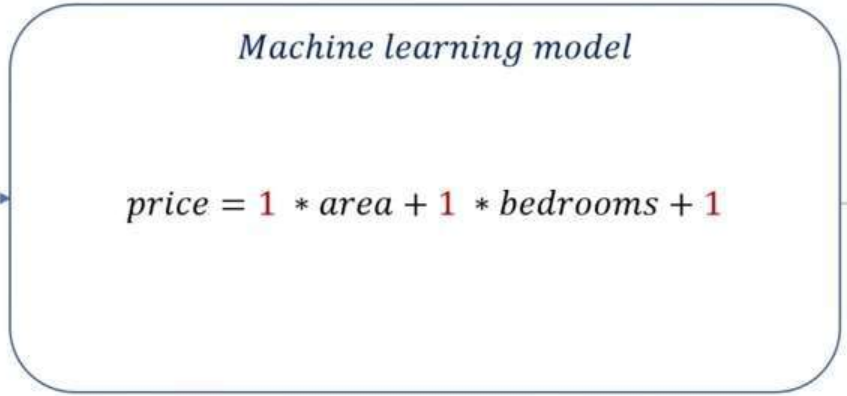


$\widehat{price} = 4107$

$price = 810000$

$error6 = (price - \widehat{price})^2$

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000



$\widehat{price} = 4107$

$price = 810000$

$error6 = (price - \widehat{price})^2$

End of first epoch

$$Total\ Error = \ error1 + error2 + \ \ldots + error6$$

$$Mean\ Squred\ Error(a.k.a.MSE) = \frac{Total\ Error}{6}$$

$$Total\ Error = error1 + error2 + \ldots + error6$$

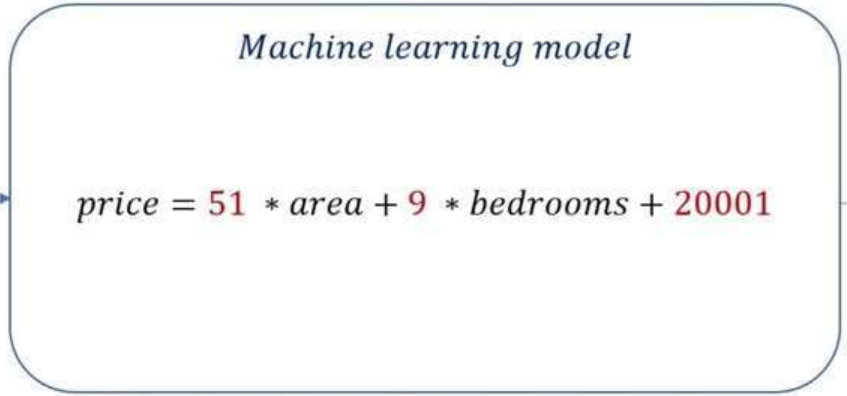
$$Mean\ Squred\ Error(a.k.a.MSE) = \frac{Total\ Error}{6}$$

$$w1 = w1 - learning\ rate * {}^{\partial(\mathrm{MSE})}/_{\partial w1} \qquad w1 = 1 - (-50) = 51$$

$$w2 = w2 - learning\ rate * {}^{\partial(\mathrm{MSE})}/_{\partial w2} \qquad w2 = 1 - (-8) = 9$$

$$b = b - learning\ rate * {}^{\partial(\mathrm{MSE})}/_{\partial b} \qquad bias = 1 - (-20000) = 20001$$

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
4000	5	760000
4100	6	810000



$\widehat{price} = 229155$

$price = 550000$

$error6 = (price - \widehat{price})^2$

End of second epoch

In batch gradient descent,

1) We go through all training samples and calculate cumulative error
2) Now we back propagate and adjust weights

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
...	...	...
4100	6	810000

10 million samples

1) To find cumulative error for first round (epoch) now we need to do a forward pass for 10 million samples
2) We have 2 features (area and bedroom). This requires finding 20 million derivatives

1. *Randomly pick single data training sample*

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
...	...	...
4100	6	810000

10 *million samples*

Machine learning model

$price = 1 * area + 1 * bedrooms + 1$

$\widehat{price} = 3204$

$price = 610000$

$error = (price - \widehat{price})^2$

2. *Adjust weights*

$$w1 = w1 - learning\ rate * {\partial(\text{error})}/{\partial w1} \qquad w1 = 1 - (-13) = 14$$

$$w2 = w2 - learning\ rate * {\partial(\text{error})}/{\partial w2} \qquad w2 = 1 - (-3) = 4$$

$$b = b - learning\ rate * {\partial(\text{error})}/{\partial b} \qquad \text{bias} = 1 - (-1500) = 1501$$

3. *Again randomly pick a training sample*

area	bedrooms	price
2600	3	550000
3000	4	565000
3200	3	610000
3600	3	595000
...	...	...
4100	6	810000

10 *million samples*

Machine learning model

$$price = 14 * area + 4 * bedrooms + 1501$$

$$\widehat{price} = 3204$$

$$price = 37913$$

$$error = (price - \widehat{price})^2$$

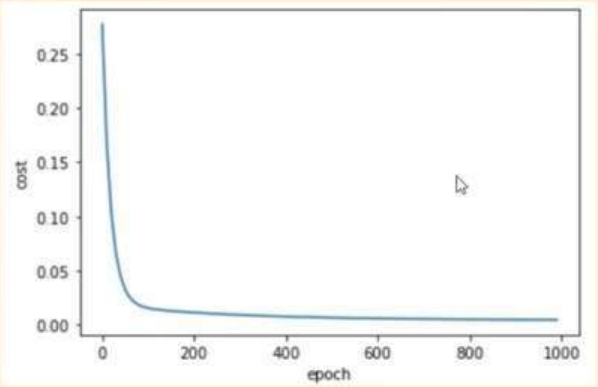
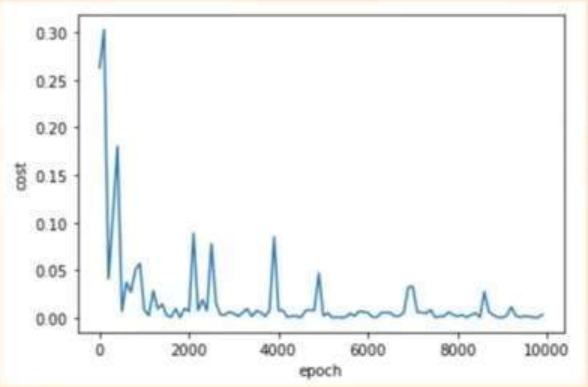
4. Again adjust weights

$$w1 = w1 - learning\ rate * {}^{\partial(\text{error})}/_{\partial w1} \qquad w1 = 14 - (-100) = 114$$

$$w2 = w2 - learning\ rate * {}^{\partial(\text{error})}/_{\partial w2} \qquad w2 = 4 - (-12) = 16$$

$$b = b - learning\ rate * {}^{\partial(\text{error})}/_{\partial b} \qquad bias = 1501 - (-2001) = 3502$$

Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Use **all** training samples for one forward pass and then adjust weights	Use **one** (randomly picked) sample for a forward pass and then adjust weights
Good for small training set	Good when training set is very big and we don't want too much computation

Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Use **all** training samples for one forward pass and then adjust weights	Use **one** (randomly picked) sample for a forward pass and then adjust weights
Good for small training set	Good when training set is very big and we don't want too much computation
	

Mini batch is like SGD. Instead of choosing one randomly picked training sample, you will use a batch of randomly picked training samples.

1. For example I have 20 training samples total.
2. Let's say I use 5 random samples for one forward pass to calculate cumulative error
3. After that I adjust weights

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini batch gradient descent
Use **all** training samples for one forward pass and then adjust weights	Use **one** (randomly picked) sample for a forward pass and then adjust weights	Use **a batch of** (randomly picked) samples for a forward pass and then adjust weights

Stanford
ONLINE

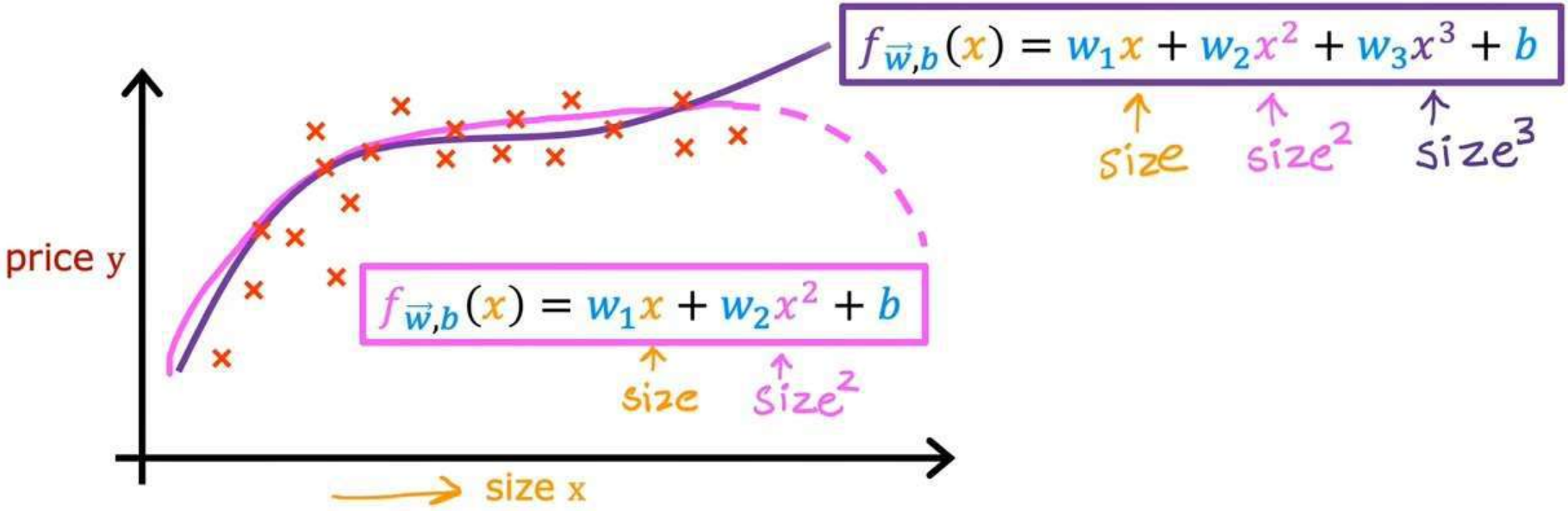
DeepLearning.AI



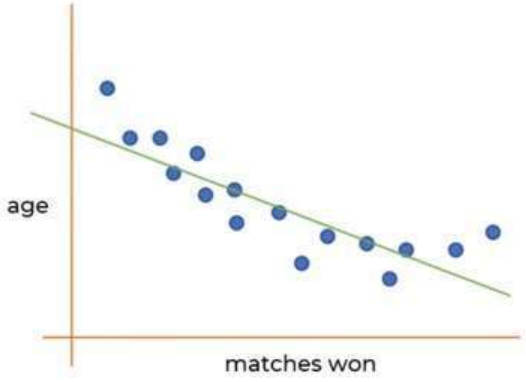
Practical Tips for Linear Regression

Polynomial Regression

Polynomial regression



underfit



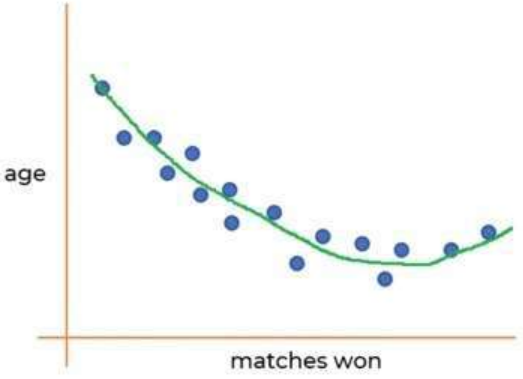
$$\text{match won} = \theta_0 + \theta_1 * age$$

overfit



$$\text{match won} = \theta_0 + \theta_1 * age \quad + \theta_2 * age^2 + \theta_3 * age^3 + \theta_4 * age^4$$

balanced fit

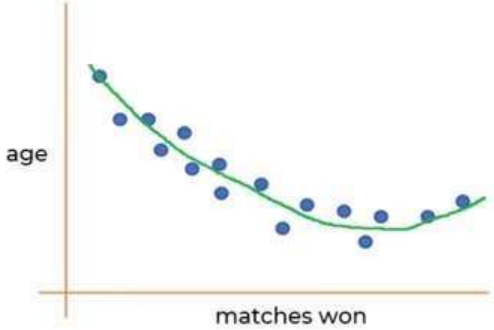


$$\text{match won} = \theta_0 + \theta_1 * age \quad + \theta_2 * age^2$$



$$\text{match won} = \theta_0 + \theta_1 * age \quad + \theta_2 * age^2 + \theta_3 * age^3 + \theta_4 * age^4$$

Try to make θ_3 and $\theta_4$almost close to zero



$$\text{match won} = \theta_0 + \theta_1 * age \quad + \theta_2 * age^2$$

Mean Squared Error

$$mse = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - y_{predicted}\right)^2$$

Mean Squared Error

$$ms\varepsilon = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - h_\theta(x_i)\right)^2$$

$$h_\theta(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

L2 Regularization

$$mse = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - h_\theta(x_i)\right)^2 + \lambda \sum_{i=1}^{n} \theta_i^2$$

$$h_\theta(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$

L1 Regularization

$$mse = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - h_\theta(x_i)\right)^2 + \lambda \sum_{i=1}^{n} |\theta_i|$$

$$h_\theta(x_i) = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3$$